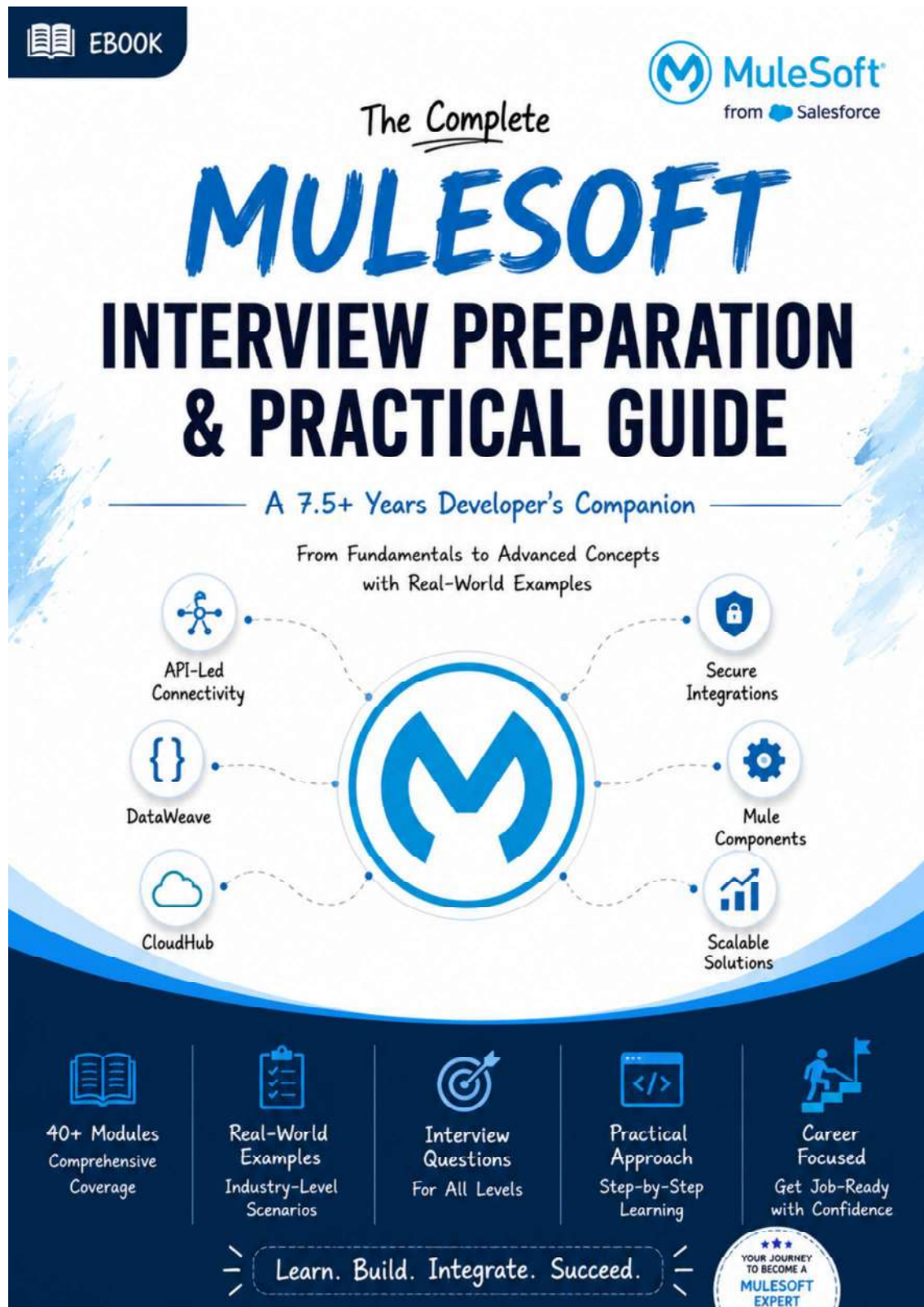




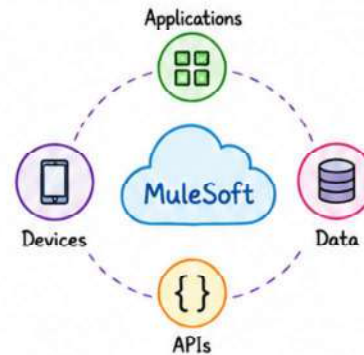
1.1 WHAT IS MULESOFT?

MuleSoft is a leading **integration** and **automation** platform that helps **connect** applications, data, and devices across **cloud** and **on-premises** environments.



KEY POINTS

- MuleSoft is an **iPaaS** (Integration Platform as a Service).
- It enables organizations to build, manage and compose **APIs** and integrations.
- It provides a unified platform - **Anypoint Platform** to design, build, deploy and manage integrations.
- It supports both **cloud** and **on-premises** deployments.
- It helps achieve **real-time integration** and **process automation**.



WHY MULESOFT?



KEY FEATURES

- ✓ API-led connectivity
- ✓ Rich set of connectors
- ✓ Visual development (Anypoint Studio)
- ✓ Data transformation (DataWeave)
- ✓ Management & monitoring
- ✓ Security & governance
- ✓ Scalability & high availability
- ✓ Reusable assets (Exchange)



MULESOFT ECOSYSTEM - ANYPOINT PLATFORM



REAL-WORLD USE CASES



MuleSoft helps organizations to **CONNECT**, **AUTOMATE** and **TRANSFORM** their business with APIs and integrations.



1.2

API-LED CONNECTIVITY

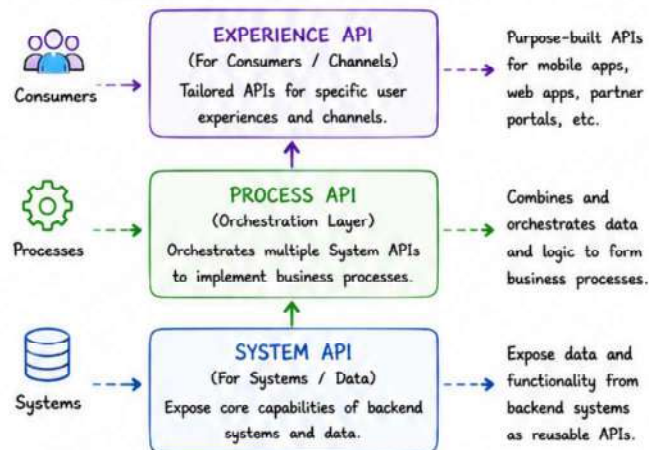


API-Led Connectivity is an architectural approach that uses **APIs** as the **building blocks** to connect applications, data, and services in a **reusable, scalable and secure** way.

WHAT IS IT?

In API-Led Connectivity, we expose and consume capabilities as **APIs** in a layered manner. Each API layer has a **specific purpose** and can be **reused** by other applications.

THREE-LAYER ARCHITECTURE



WHY API-LED CONNECTIVITY?

- Promotes **reusability** of APIs
- Improves **scalability** and **agility**
- Encourages **consistency** and **standardization**
- Enhances **maintainability**
- Speeds up development and **time-to-market**
- Supports **secure** and **governed** integrations

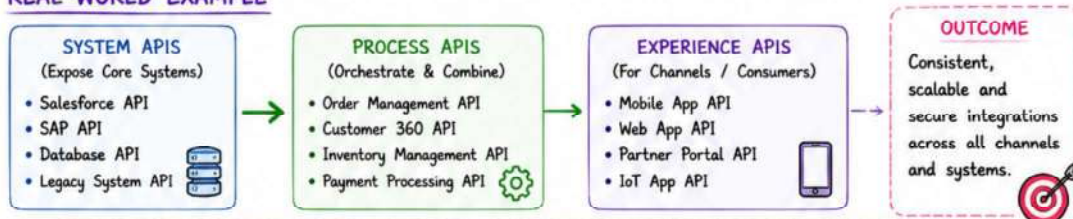
KEY PRINCIPLES

- ✓ Everything is an **API**
- ✓ Design for **reuse**
- ✓ Use **loose coupling**
- ✓ Think at **scale**
- ✓ **Security** and **governance** by design

BENEFITS



REAL-WORLD EXAMPLE



💡 API-Led Connectivity helps organizations build a **flexible** and **future-ready** integration architecture that grows with business needs.

1.3

THREE-LAYER ARCHITECTURE



The **Three-Layer Architecture** is a core principle of **API-Led Connectivity**. It organizes APIs into three logical layers, each with a specific role and purpose.

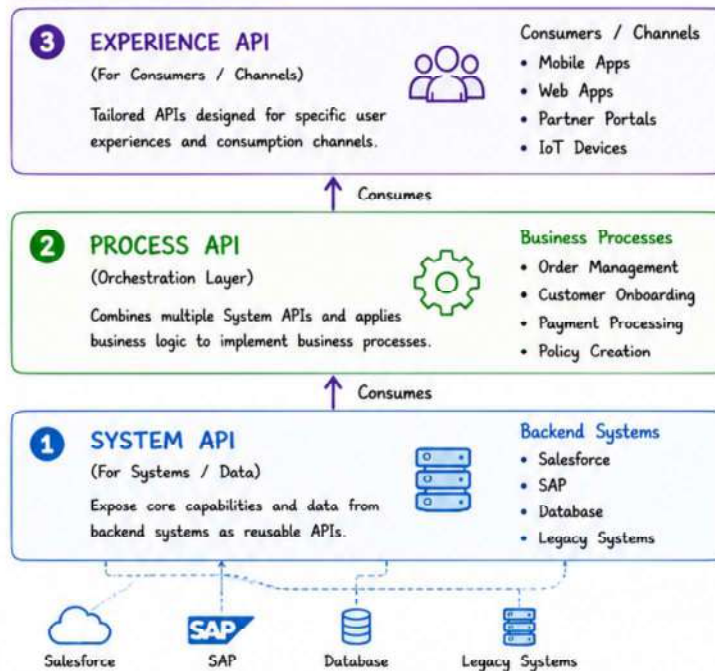
OVERVIEW

This architecture promotes **separation of concerns**, **reusability**, **scalability** and **maintainability** of APIs and integrations.

LAYERS AT A GLANCE

- 1 **System API**
Expose data and capabilities from backend systems.
- 2 **Process API**
Orchestrate and combine System APIs to implement business processes.
- 3 **Experience API**
Expose Process APIs to consumers through different channels.

ARCHITECTURE DIAGRAM



LAYER DETAILS

- 1 **SYSTEM API** (For Systems / Data)
 - Expose data and functions from backend systems.
 - Technology-focused.
 - Reusable across multiple processes.
 - Examples: `GET /customers`, `GET /products`
- 2 **PROCESS API** (Orchestration Layer)
 - Orchestrate and combine multiple System APIs.
 - Implement business logic and rules.
 - Reusable across different experience layers.
 - Examples: `/processOrder`, `/createCustomer`
- 3 **EXPERIENCE API** (For Consumers / Channels)
 - Designed for specific consumer needs.
 - Optimized for user experience.
 - May aggregate multiple Process APIs.
 - Examples: `/mobile/orders`, `/partner/quotes`

KEY BENEFITS

	REUSABILITY	APIs in lower layers are reused across multiple processes and channels.
	SCALABILITY	Changes in one layer have minimal impact on other layers.
	SEPARATION OF CONCERNS	Each layer has a specific responsibility, making it easier to design and maintain.
	FLEXIBILITY	New channels or systems can be added without affecting existing layers.
	GOVERNANCE & SECURITY	Centralized policies, security and governance can be applied.

BEST PRACTICES

- Keep APIs in their appropriate layer.
- Avoid exposing System APIs directly to consumers.
- Design APIs to be reusable and loosely coupled.
- Use consistent naming and versioning strategies.
- Apply security and governance at all layers.



The **Three-Layer Architecture** is the foundation of **API-Led Connectivity**. It helps build **reusable**, **scalable**, **secure** and **future-ready** integrations.



1.4

MULE APPLICATION LIFECYCLE



MuleSoft
from salesforce

The **Mule Application Lifecycle** represents the **end-to-end** journey of a Mule application from initial **design** to ongoing **maintenance** in production.

LIFECYCLE OVERVIEW



DETAILS OF EACH STAGE



KEY POINTS



- The lifecycle is iterative, not always linear.
- Feedback from monitoring helps improve design and development.
- Automation (CI/CD) helps in faster and reliable delivery.
- Following best practices at each stage ensures high-quality and scalable integrations.

TOOLS & COMPONENTS USED



- Design** → Anypoint Design Center
- Development** → Anypoint Studio
- Testing** → MUnit, Postman, DataWeave
- Packaging** → Maven
- Deployment** → Runtime Manager, CloudHub, On-Prem, RTF, Kubernetes
- Monitoring** → Anypoint Monitoring, Runtime Manager

BENEFITS OF FOLLOWING LIFECYCLE



- Ensures a structured and standard approach.
- Improves quality, reliability and performance.
- Supports collaboration across teams.
- Enables faster time-to-market.
- Simplifies maintenance and future enhancements.



Following the **Mule Application Lifecycle** ensures that applications are **well-designed**, **thoroughly tested**, **securely deployed** and **continuously improved** for business success.



1.5 MULE EVENT

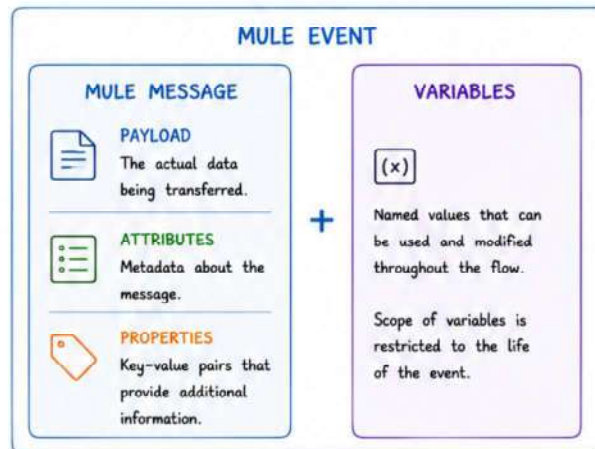


A Mule Event is the **fundamental unit of processing** in Mule runtime. It represents a message that flows through the application from the source to the destination.

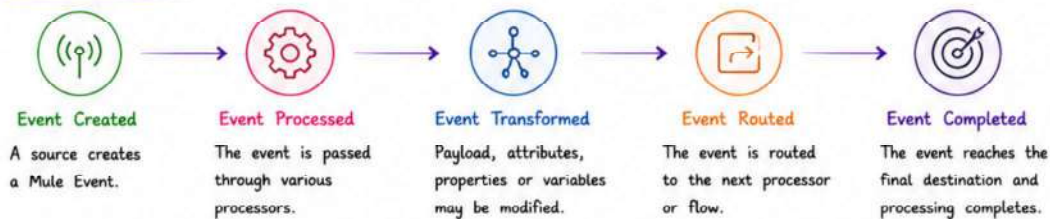
WHAT IS A MULE EVENT?

- It is an object that **carries data** through the flow.
- It consists of a **Mule Message** and associated **variables**.
- It is created by a **source** and processed by **processors** in the flow.
- It is passed from one processor to another until it reaches the **final destination**.

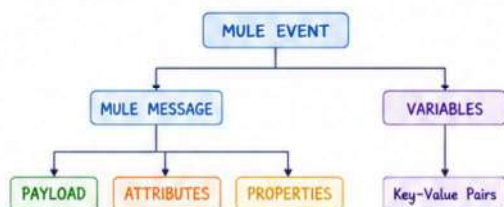
COMPONENTS OF MULE EVENT



EVENT LIFECYCLE



MULE EVENT STRUCTURE



KEY POINTS

- ✓ Mule Event flows through the application and holds all the data.
- ✓ It is immutable in structure but content can be modified.
- ✓ Each processor receives an event, operates on it and passes it on.
- ✓ Variables live only for the lifetime of the event.
- ✓ Understanding Mule Event is key to understanding Mule applications.

EXAMPLE

Consider an HTTP Listener receives a request.

```

Mule Event
{
  "message": {
    "payload": { "id": 101, "name": "John" },
    "attributes": {
      "httpStatus": 200,
      "method": "GET",
      "headers": { "content-type": "application/json" }
    },
    "properties": {
      "source": "HTTP Listener",
      "correlationId": "12345"
    }
  },
  "variables": {
    "userId": 101,
    "isPremium": true
  }
}
  
```



IMPORTANT

Everything in Mule is event-driven. Sources create events, processors transform events, and sinks consume events.



1.6

MULE MESSAGE

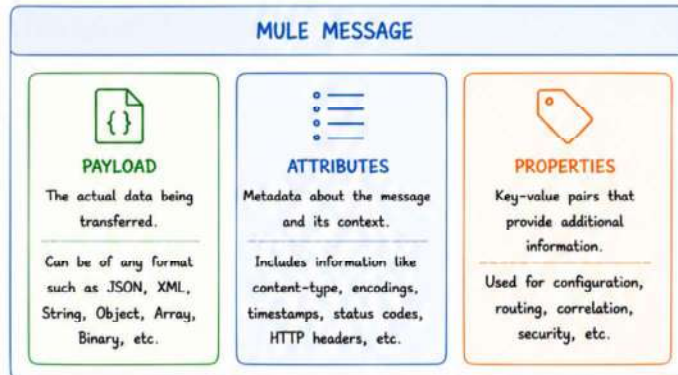


A Mule Message is the **core part** of a Mule Event. It carries the **data** that flows through the application along with its **metadata**.

WHAT IS A MULE MESSAGE?

- It is an **immutable** data container that holds the data and metadata in a Mule Event.
- It travels through the flow and may be **modified** by processors.
- It consists of **Payload**, **Attributes** and **Properties**.

STRUCTURE OF MULE MESSAGE



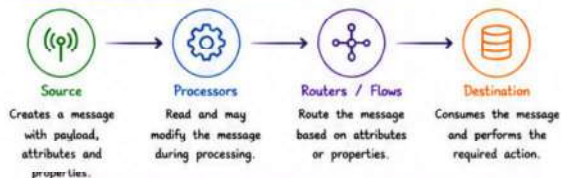
COMPONENTS IN DETAIL

- PAYLOAD**
 - Contains the business data.
 - This is what we actually want to transfer or process.
 - It is the primary data of the message.
 - Example: JSON, XML, CSV, Text, Binary, Object.
- ATTRIBUTES**
 - Contains metadata about the message.
 - Helps in routing, transformation and processing.
 - Common attributes: content-type, content-encoding, httpStatus, headers, mediaType, characterEncoding.
 - Automatically set by connectors or processors.
- PROPERTIES**
 - User-defined key-value pairs.
 - Used to store additional information for processing.
 - Not transmitted as part of the payload.
 - Example: correlationId, userId, config values.

EXAMPLE OF MULE MESSAGE

```
{
  "payload": {                                // Business Data
    "id": 101,
    "name": "John",
    "amount": 2500
  },
  "attributes": {
    "content-type": "application/json",        // Media type
    "content-encoding": "UTF-8",              // Encoding
    "httpStatus": 200,                         // HTTP Status
    "headers": {
      "host": "api.example.com",
      "accept": "application/json"
    }
  },
  "properties": {
    "correlationId": "12345",
    "sourceSystem": "CRM",
    "retryCount": 0                            // Custom property
  }
}
```

MESSAGE FLOW IN A MULE APPLICATION



KEY POINTS

- ✓ Mule Message is part of Mule Event.
- ✓ It contains Payload, Attributes and Properties.
- ✓ It is immutable, but a new message is created when modified.
- ✓ Understanding Mule Message is key to effective data flow and integration.

COMMONLY USED ATTRIBUTES

ATTRIBUTE	DESCRIPTION
content-type	MIME type of the payload
content-encoding	Character encoding (e.g., UTF-8)
httpStatus	HTTP status code
headers	Map of HTTP headers
mediaType	Media type of the payload
characterEncoding	Character encoding of the payload

BEST PRACTICES

- ✓ Use attributes for standard metadata.
- ✓ Use properties for custom information needed across the flow.
- ✓ Avoid storing large data in properties.
- ✓ Maintain message immutability and follow transformation best practices.



A Mule Message carries the **right data**, at the **right time**, in the **right format** to the **right destination**.



1.7 MULE RUNTIME



Mule Runtime is the powerful **engine** that executes Mule applications. It provides the **environment**, **services**, and **infrastructure** required to run, manage and monitor applications.

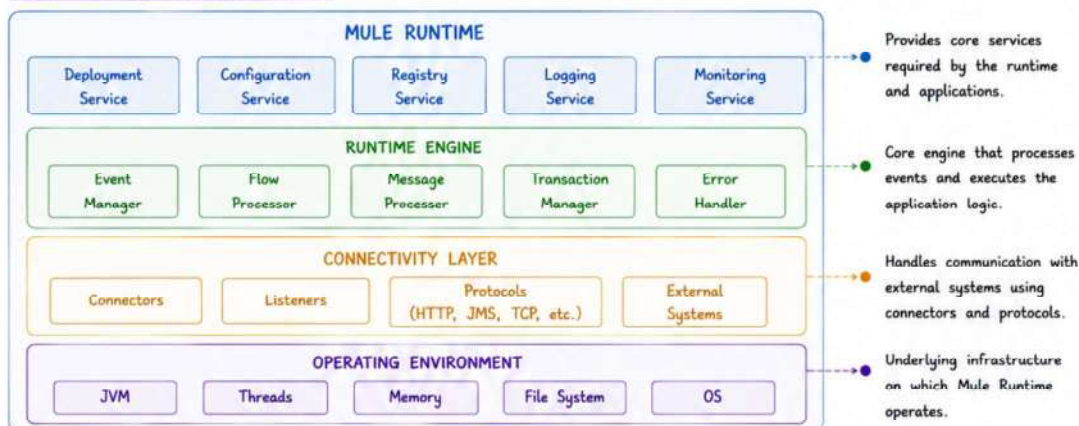
WHAT IS MULE RUNTIME?

- Mule Runtime is the **core engine** of the Mule platform.
- It **executes** Mule applications and manages events as they flow through the application.
- It provides **infrastructure services** such as threading, connectivity, transaction management, security, logging and more.
- Mule applications are **deployed** on Mule Runtime.

KEY RESPONSIBILITIES

 EXECUTION Executes flows, sub-flows and processors.	 EVENT MANAGEMENT Creates, manages and routes events through the flow.	 CONNECTIVITY Manages connectors, listeners and external connections.
 SECURITY Provides authentication, authorization and encryption support.	 MONITORING & LOGGING Collects metrics, logs and enables monitoring.	 RESOURCE MANAGEMENT Manages threads, memory, transactions and other resources.

MULE RUNTIME ARCHITECTURE



HOW IT WORKS (EVENT FLOW)

- EVENT RECEIVED**
A source (e.g., HTTP Listener) receives a request and creates a Mule Event.
- EVENT PROCESSING**
Runtime Engine processes the event through flows and processors.
- ROUTING & TRANSFORMATION**
Event is routed, transformed or enriched as per the flow logic.
- EXTERNAL INTERACTION**
Connectors are used to interact with external systems if required.
- RESPONSE SENT**
The response is returned to the source, completing the flow.

RUNTIME SERVICES (IMPORTANT)

SERVICE	DESCRIPTION
Deployment Service	Deploys and undeploys applications.
Configuration Service	Loads and provides configuration to applications.
Registry Service	Manages connectors and shared resources.
Logging Service	Provides logging framework for applications.
Monitoring Service	Exposes metrics and health of applications.
Object Store Service	Stores and manages persistent data if required.
Scheduler Service	Handles scheduled tasks in applications.

KEY FEATURES

- ✓ Lightweight and high performance.
- ✓ Scalable and supports clustering.
- ✓ Built-in security and message management.
- ✓ Extensible with connectors and modules.
- ✓ Cloud-ready and on-premise capable.

BENEFITS

- ✓ Faster development and deployment.
- ✓ High reliability and availability.
- ✓ Better resource utilization.
- ✓ Centralized management and monitoring.
- ✓ Supports enterprise integration needs.

RUNTIME MODES

- Standalone** : Runs Mule application as a standalone process.
- Clustered** : Multiple Mule runtimes run together for high availability and scaling.
- Managed Cloud** : Runtime is managed by Anypoint Platform (CloudHub).



Mule Runtime is the **heart** of MuleSoft, providing a **robust**, **secure** and **scalable** execution environment for building and running integration solutions.



1.8 MULE DOMAINS



A **Mule Domain** is a logical container that provides a **dedicated runtime environment** for deploying and running Mule applications. It isolates applications, configurations, resources and policies from other domains.

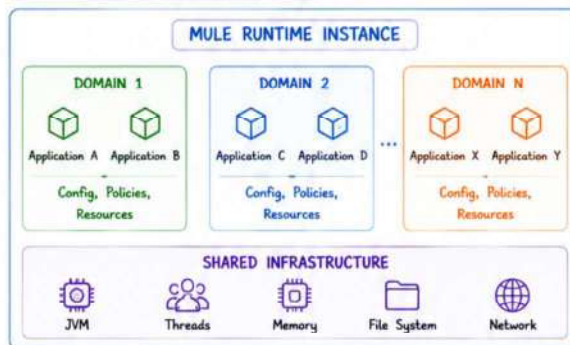
WHAT IS A MULE DOMAIN?

- A Mule Domain is a **runtime environment** that hosts Mule applications.
- It provides **isolation**, **resource management**, and **configuration** for the applications deployed in it.
- Each domain has its own **configuration**, policies, security settings, logging, threading and resources.
- Multiple domains can **coexist** in the same Mule Runtime instance.

WHY USE MULE DOMAINS?



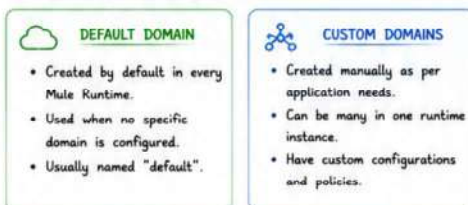
MULE DOMAIN ARCHITECTURE



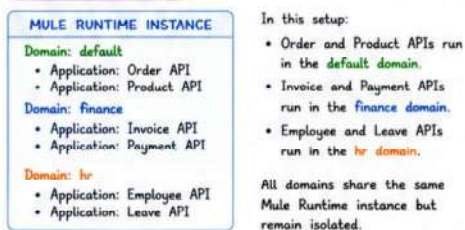
KEY CHARACTERISTICS

FEATURE	DESCRIPTION
Logical Container	A domain is a logical container within Mule Runtime.
Isolation	Applications in one domain are isolated from other domains.
Independent Config	Each domain has its own configuration, properties and policies.
Dedicated Resources	Resources (threads, memory, connection pools, etc.) are managed per domain.
Multiple Applications	Multiple Mule applications can be deployed in the same domain.
Lifecycle Management	Domains can be started, stopped, deployed, undeployed independently.
Monitoring & Logging	Metrics and logs are captured per domain for better visibility.

TYPES OF MULE DOMAINS



EXAMPLE SCENARIO



HOW DOMAINS WORK

- 1 CREATE DOMAIN** - Create a domain with desired configurations, resources and policies.
- 2 DEPLOY APPLICATION** - Deploy Mule applications to the target domain.
- 3 START DOMAIN** - Mule Runtime starts the domain and initializes resources.
- 4 PROCESS EVENTS** - Applications in the domain process events and interact with external systems.
- 5 STOP/UNDEPLOY DOMAIN** - Stop or undeploy the domain as needed.

BEST PRACTICES

- ✓ Create separate domains for different applications or teams.
- ✓ Use meaningful domain names based on business or environment.
- ✓ Allocate resources (threads, memory, connection pools) appropriately per domain.
- ✓ Externalize configurations and use domain-level properties.
- ✓ Apply security policies at the domain level.
- ✓ Monitor domains separately for better visibility and troubleshooting.
- ✓ Regularly review and optimize domain configurations and resources.



Mule Domains provide **isolation**, **control** and **flexibility** to run multiple applications **securely** and **efficiently** within the same Mule Runtime instance.



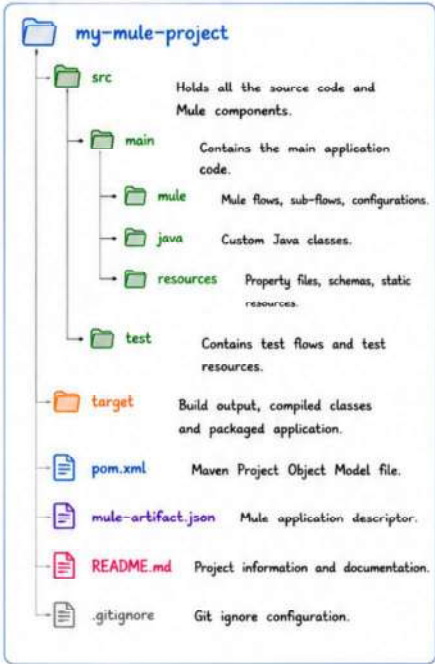
1.9

MULE PROJECT STRUCTURE

MuleSoft
from salesforce

A Mule project is an Anypoint Studio project that contains all the resources, configuration files and assets required to build, run and deploy a Mule application.

TYPICAL MULE PROJECT STRUCTURE



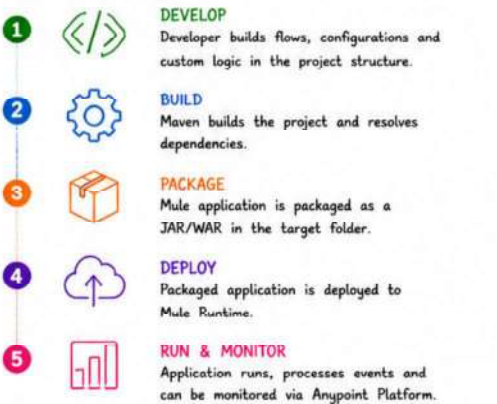
KEY DIRECTORIES AND FILES

ITEM	DESCRIPTION
src/main/mule	- Contains Mule flows, sub-flows and configuration files. - Includes global elements and API implementations.
src/main/java	- Contains custom Java classes, utilities and modules. - Used when custom logic beyond DataWeave is required.
src/main/resources	- Stores property files, schemas, JSON/XML samples, certificates, static content, etc. - Used by the application at runtime.
src/test	- Contains test flows, mock data and test resources. - Used for automated testing with MUnit.
target	- Output directory generated by Maven build. - Contains compiled classes and packaged application (JAR/WAR).
pom.xml	- Defines project dependencies, plugins and build configurations. - Standard Maven configuration file.
mule-artifact.json	- Describes the Mule application. - Contains the app name, version, min Mule version, secure properties, etc.
README.md	Provides project overview, setup and usage instructions.
.gitignore	Specifies files and folders to be ignored by Git.

IMPORTANT FILES EXPLAINED

pom.xml	- Manages dependencies (Mule libraries, connectors, etc.). - Defines build lifecycle and plugins. - Supports profiles for different environments.
mule-artifact.json	- Unique descriptor for every Mule application. - Contains app name, version and secure properties. - Used by Mule Runtime for deployment.
application.properties / application.yml	- Application configuration properties. - Supports externalized configuration.
flows	- XML files defining Mule flows and sub-flows. - Core of the application logic.

HOW IT WORKS



BEST PRACTICES

- ✔ Follow a consistent folder structure across projects.
 - ✔ Keep flows modular; use sub-flows and global elements.
 - ✔ Externalize configurations using property files.
 - ✔ Store sensitive information in secure properties.
- ✔ Write tests in src/test using MUnit.
 - ✔ Keep dependencies updated and avoid unused libraries.
 - ✔ Maintain clean and meaningful project documentation.
 - ✔ Use version control (Git) to manage your projects.
- 



A well-structured Mule project ensures code organization, maintainability, reusability and smooth development to deployment.



1.10 MULE-ARTIFACT.JSON



`mule-artifact.json` is a descriptor file that contains **metadata** about a Mule application. It is used by **Anypoint Platform** and **Mule Runtime** to identify, configure and run the application.

WHAT IS MULE-ARTIFACT.JSON?

- It is a **JSON** descriptor file.
- It contains information about the application such as **name**, **version**, **min Mule version**, **secure properties**, **required plugins**, etc.
- It is **mandatory** in every Mule application.
- It is located in the **root directory** of the Mule project.
- **Mule Runtime** reads this file to deploy and run the application.

TYPICAL `mule-artifact.json` STRUCTURE

```
{
  "name": "my-mule-app",           // Application name
  "minMuleVersion": "4.4.0",      // Minimum Mule version
  "secureProperties": [
    {
      "key": "db.password",
      "value": "123456"
    }
  ],                               // Encrypted secure properties
  "requiredPlugins": [
    {
      "groupId": "org.mule.connectors",
      "artifactId": "mule-http-connector",
      "version": "1.8.1"
    }
  ],                               // Required plugins
  "muleVersion": "4.4.0",         // Target Mule version
  "classLoaderModelLoaderDescriptor": {
    "id": "mule",
    "version": "1.3.0"
  }
}
```

KEY POINTS

- ✓ Always present in the root directory.
- ✓ Automatically generated when you create a new Mule project.
- ✓ Updated when you add plugins or change secure properties.
- ✓ Do not modify it manually unless required.
- ✓ Used during build, deploy and runtime.



mule-artifact.json

PURPOSE

- Identifies the application to Mule Runtime and Anypoint Platform.
- Holds configuration about the application and its dependencies.
- Provides information about secure properties and their encryption.
- Helps in packaging, deploying and managing the application.

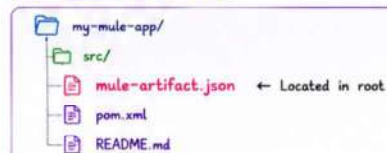
FIELD DESCRIPTION

FIELD	DESCRIPTION
name	The name of the Mule application.
minMuleVersion	The minimum Mule Runtime version required to run this application.
secureProperties	List of secure properties with their encrypted values.
requiredPlugins	List of plugins (connectors, modules) required by the application.
muleVersion	The target Mule version for which the application is built.
classLoaderModelLoaderDescriptor	Defines the classloader model used by the application.

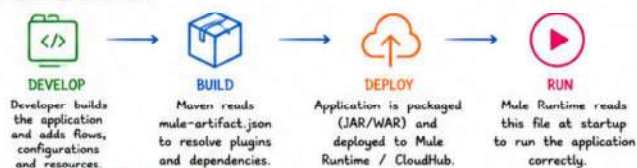
EXAMPLE (EXPLAINED)

- **name** : my-mule-app → The application name.
- **minMuleVersion** : 4.4.0 → Requires at least Mule 4.4.0.
- **secureProperties** : Contains encrypted sensitive data (e.g., database passwords, API keys).
- **requiredPlugins** : Lists connectors used in the application (e.g., HTTP connector).
- **muleVersion** : The Mule version used to build the app.
- **classLoaderModelLoaderDescriptor** : Defines how classes and dependencies are loaded at runtime.

LOCATION IN PROJECT



LIFECYCLE USAGE



BEST PRACTICES

- ✓ Keep **secure** data only in **secureProperties**.
- ✓ Let Anypoint Studio manage this file automatically.
- ✓ Do not store plain text passwords or secrets.
- ✓ Keep plugins list minimal and up to date.
- ✓ Commit this file to version control (Git).



NOTE

- mule-artifact.json** is small but powerful. It tells the platform everything it needs to know about your Mule application.



`mule-artifact.json` is the **identity card** of your Mule application. It ensures your app is built, deployed and run in the **right way**.



2.1 WHAT IS ANYPOINT PLATFORM?



Anypoint Platform is MuleSoft's integration platform that helps organizations connect applications, data, and devices securely. It provides a complete set of tools to **design**, **build**, **test**, **deploy**, **manage** and **monitor** APIs and integrations.

OVERVIEW

Anypoint Platform is a unified platform for API-led connectivity. It enables you to build reusable APIs and integrations and manage them across their entire lifecycle.

- It is a **cloud-based** platform (with on-premises runtime option).
- It provides a **single interface** to all tools and capabilities.
- It supports the entire **API lifecycle**.
- It promotes **reuse**, **standardization**, **security** and **governance**.
- It is built on the principles of **API-led connectivity**.

KEY COMPONENTS

COMPONENT	PURPOSE
Design Center	Design and document APIs using RAML.
Anypoint Studio	Build, test and debug Mule applications.
Exchange	Discover and share reusable assets.
API Manager	Secure, manage and analyze APIs.
Runtime Manager	Deploy, manage and monitor applications.
Access Management	Manage users, roles and permissions.
Anypoint Monitoring	Monitor performance and health.
Visualizer	Visualize API dependencies and flows.
Secrets Manager	Securely store and manage sensitive data.
Object Store	Store and retrieve data persistently.
Anypoint MQ	Reliable messaging for applications.

HIGH-LEVEL ARCHITECTURE



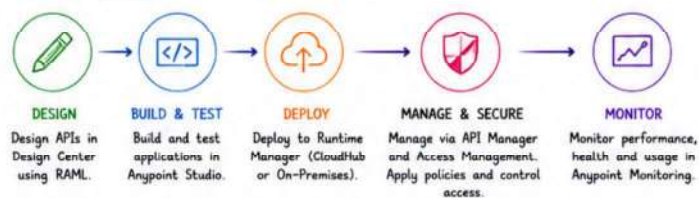
BENEFITS



USE CASES



HOW IT WORKS



ANYPOINT PLATFORM - AT A GLANCE

- ✓ A unified platform for API-led connectivity.
- ✓ Covers the entire API lifecycle - Design, Build, Deploy, Manage, Monitor.
- ✓ Provides powerful tools and services to accelerate integration.
- ✓ Enables secure, scalable and reliable connectivity.
- ✓ Integrates with any system, any data, at any time.



Anypoint Platform is the **foundation** for **building**, **managing** and **scaling** **API-led connectivity** in the enterprise.



2.2 DESIGN CENTER



Anypoint Design Center is a **cloud-based** API design tool that lets you design, document and collaborate on APIs using **RAML**. It enables you to create high-quality **APIs** quickly with built-in validation, mocking and rich documentation.

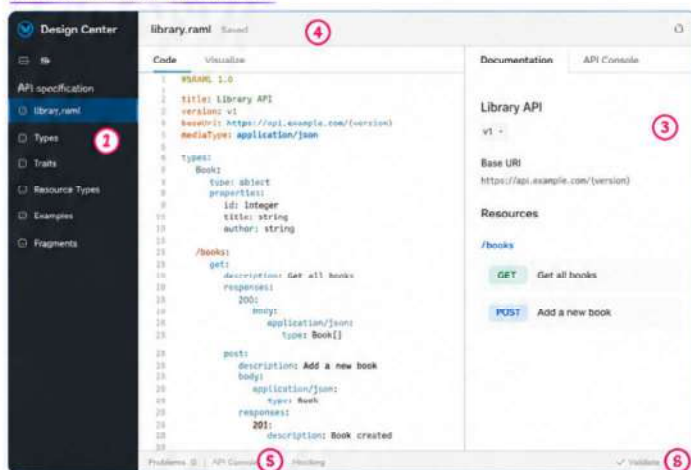
WHAT IS DESIGN CENTER?

- A **web-based** API design platform on Anypoint Platform.
- Used to design and document APIs using **RAML** API specification.
- Provides tools for validation, mocking, documentation and collaboration.
- Eliminates guesswork for developers by providing a **contract-first** approach.
- APIs designed here can be exported, published to **Exchange** or used in implementations.

KEY CAPABILITIES



DESIGN CENTER WORKSPACE



- ① Navigator Organize and manage RAML assets. ② RAML Editor Write and edit RAML with smart assistance. ③ Documentation Auto-generated documentation. ④ View Switch Switch between Code and Visualize views. ⑤ Tools Access problems, mock and API console. ⑥ Validation Validate RAML against best practices.

SUPPORTED RAML VERSIONS

- RAML 1.0
- RAML 0.8 (Legacy)

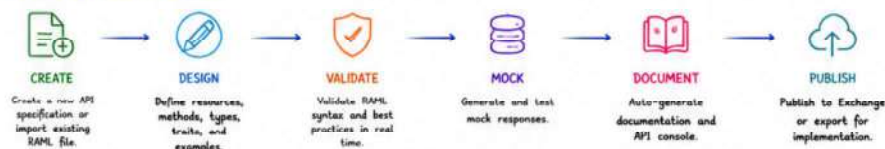
DESIGN CENTER OUTPUTS

- RAML File**
API specification in RAML format.
- API Console**
Interactive documentation for testing.
- Mock Service**
Mock endpoints for API simulation.
- Exchange Asset**
Publish RAML to Exchange for reuse.

BENEFITS

- ✓ Speeds up API design and documentation.
- ✓ Ensures consistency and standardization.
- ✓ Enables early collaboration and feedback.
- ✓ Reduces development time with mocking.
- ✓ Promotes reusability through Exchange.
- ✓ Improves API quality with validation.

DESIGN CENTER WORKFLOW



BEST PRACTICES

- ✓ Design APIs using the contract-first approach.
- ✓ Follow RAML best practices and naming conventions.
- ✓ Use data types, traits and resource types for reusability.
- ✓ Validate frequently to maintain high-quality specifications.
- ✓ Collaborate and review with stakeholders early.
- ✓ Publish reusable assets to Exchange.



USE CASES

- ✓ Designing REST and SOAP APIs.
- ✓ Creating reusable API specifications for teams.
- ✓ Mocking APIs for parallel development.
- ✓ Generating documentation for consumers.
- ✓ Publishing assets for organization-wide reuse.



Design Center helps you **design better APIs, faster and together!**
Contract First. Design Smart. Deliver Quality APIs.



2.3 ANYPOINT STUDIO



Anypoint Studio is the official IDE for **building Mule applications and APIs**. It provides tools to design, develop, test, debug and deploy integrations on Anypoint Platform.

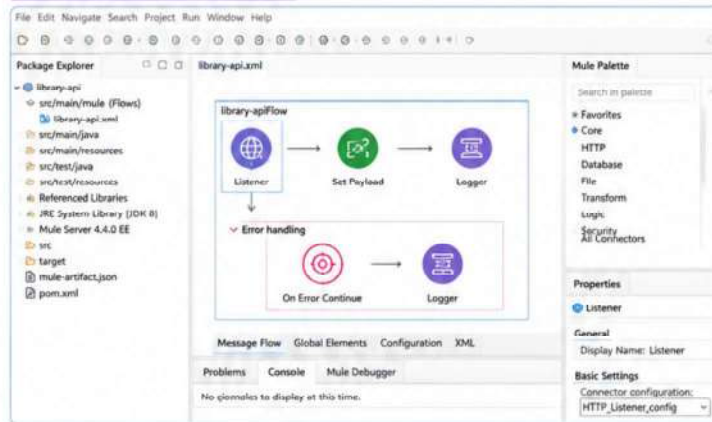
WHAT IS ANYPOINT STUDIO?

- Anypoint Studio is an Eclipse-based integrated development environment (IDE).
- Used to develop Mule applications, APIs and integrations.
- Provides visual flow designer and XML editor.
- Supports debugging, testing and deployment.
- Integrates seamlessly with Anypoint Platform.

KEY CAPABILITIES



STUDIO PERSPECTIVE OVERVIEW



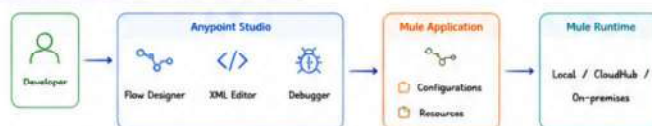
KEY FEATURES

- ✓ Graphical Flow Designer
- ✓ Rich set of connectors and modules
- ✓ XML configuration for advanced use
- ✓ Code auto-completion and validation
- ✓ Error highlighting and quick fixes
- ✓ Built-in debugger and breakpoints
- ✓ MUnit testing framework support
- ✓ Easy dependency management
- ✓ Integration with Git and CI/CD tools

SUPPORTS

- ✓ Mule 4.x
- ✓ Java (8 and above)
- ✓ MUnit for unit testing
- ✓ DataWeave 2.0
- ✓ CloudHub & On-premises deployment

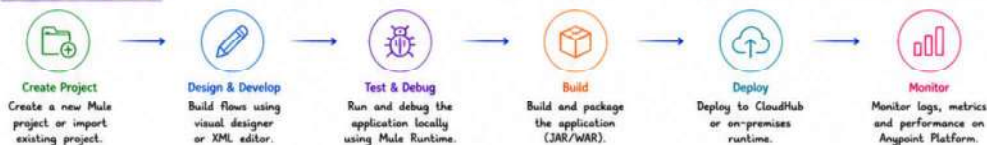
ANYPOINT STUDIO ARCHITECTURE



SYSTEM REQUIREMENTS

Component	Minimum Requirement
OS	Windows 10/11, macOS 10.14+, Linux (64-bit)
RAM	8 GB (16 GB recommended)
Disk Space	5 GB (10 GB recommended)
JDK	Java 8 or 11 (64-bit)
Anypoint Studio	Latest version

STUDIO WORKFLOW



BEST PRACTICES

- ✓ Use global elements for reusability.
- ✓ Externalize configurations using properties.
- ✓ Follow consistent naming conventions.
- ✓ Write unit tests using MUnit.
- ✓ Use error handling to manage exceptions.
- ✓ Keep flows small and focused.
- ✓ Validate and test before deployment.
- ✓ Use connectors instead of custom code.
- ✓ Version control your projects (Git).

USE CASES

- ✓ Build REST and SOAP APIs
- ✓ System integration and data transformation
- ✓ Event-driven and real-time integrations
- ✓ File, DB and legacy system integration
- ✓ Enterprise application connectivity



Anypoint Studio provides everything a developer needs to **build, test, debug and deploy** powerful integrations on Anypoint Platform.



2.4 EXCHANGE

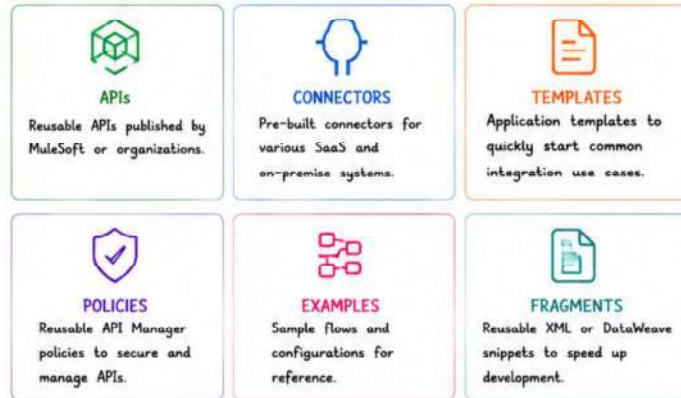


Anypoint Exchange is the central marketplace for APIs, connectors, templates and other assets on the Anypoint Platform. It helps you **discover**, **share** and **reuse** assets across teams and organizations to accelerate integration development.

WHAT IS EXCHANGE?

- Exchange is a **centralized repository** for APIs and integration assets.
- It promotes **reusability** and **collaboration** across teams and organizations.
- You can publish your assets or reuse assets built by MuleSoft or other community members.
- Assets can be **public** or **private** (organization scoped).
- It is accessible from **Anypoint Platform** and **Anypoint Studio**.

ASSETS AVAILABLE IN EXCHANGE



KEY BENEFITS

- ✓ Accelerates development with reusable assets.
- ✓ Ensures consistency and standardization.
- ✓ Encourages collaboration and knowledge sharing.
- ✓ Reduces duplicate efforts and rework.
- ✓ Easy discovery of MuleSoft certified connectors and best practices.
- ✓ Improves governance and security with organization scoped assets.

EXCHANGE ACCESS



SEARCH & FILTER

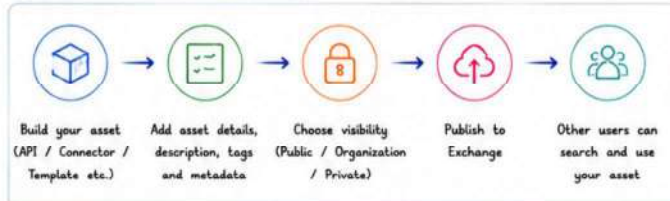
FILTER BY

- Asset Type
- Provider
- Tags
- Mule Version
- Rating
- Organization
- Custom Attributes

SORT BY

- Relevance ✓
- Most Recent
- Most Popular
- Highest Rated

PUBLISH ASSET TO EXCHANGE



EXCHANGE IN ANYPOINT STUDIO



You can search and add connectors, examples and templates directly from Anypoint Studio using the Exchange view.

BEST PRACTICES

- ✓ Use meaningful names and descriptions.
- ✓ Add relevant tags and attributes for easy discovery.
- ✓ Maintain versioning for your assets.
- ✓ Validate assets before publishing.
- ✓ Follow security best practices.
- ✓ Reuse official MuleSoft assets whenever possible.



ASSET VISIBILITY OPTIONS

VISIBILITY	DESCRIPTION
Public	Visible to everyone in the Anypoint Exchange.
Organization	Visible only to users within your organization.
Private	Visible only to specific users or groups.

USE CASES

- ✓ Reuse a connector to integrate with a system quickly.
- ✓ Use a template to bootstrap a common integration pattern.
- ✓ Apply a policy from Exchange to secure an API.
- ✓ Share custom APIs and accelerators with your organization.



Exchange empowers developers to **discover**, **share** and **reuse** high-quality assets to build integrations faster and smarter.



2.5 API MANAGER



API Manager is a powerful gateway and policy management layer on Anypoint Platform. It allows organizations to **secure**, **manage**, **control** and **analyze** API traffic and enforce policies at runtime.

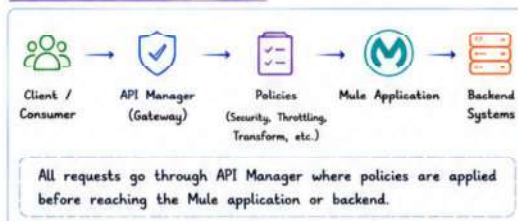
WHAT IS API MANAGER?

- A **runtime service** that sits in front of your APIs as a proxy.
- Enforces policies to **secure** and **manage** API access and traffic.
- Provides **traffic management**, access control, throttling, threat protection and **analytics**.
- Part of Anypoint Platform, deployed on **CloudHub** or on-premises.
- Implements API-led connectivity principles (Autonomy, Loose Coupling, Reusability, Discoverability, Composability).

KEY CAPABILITIES



HOW API MANAGER WORKS



COMMON POLICIES

- **Authentication** - Validate API callers.
- **Authorization** - Check scopes and roles.
- **Rate Limiting** - Limit request rate.
- **Quota** - Limit number of requests over a period.
- **IP Filtering** - Allow or block IPs.
- **Threat Protection** - Protect from common attacks.
- **Message Validation** - Validate request/response.
- **Transform** - Modify headers, body or status.
- **CORS** - Enable cross-origin access.

API MANAGER ARCHITECTURE



SLA TIERS

TIER	DESCRIPTION	EXAMPLE USE CASE
Gold	Highest level. More limits, priority support.	Premium customers, critical APIs
Silver	Medium level. Standard limits and support.	Regular customers, business APIs
Bronze	Basic level. Lower limits, best-effort support.	Internal users, low priority APIs

DEPLOYMENT OPTIONS



API MANAGER COMPONENTS



BENEFITS

- ✓ Centralized API management and security.
- ✓ Fine-grained access control and authentication.
- ✓ Better performance and traffic management.
- ✓ Threat protection against common attacks.
- ✓ Detailed analytics for better decision making.
- ✓ Enforces API governance and compliance.
- ✓ Improves developer experience with portals.

USE CASES

- ✓ Expose APIs to partners and customers securely.
- ✓ Enforce usage limits and manage SLA.
- ✓ Control access to sensitive data and services.
- ✓ Monitor API usage and performance.
- ✓ Protect APIs from malicious attacks.



API Manager acts as the front door for your APIs - **securing**, **managing** and **monitoring** every API call with policies and analytics.



2.6 RUNTIME MANAGER



Runtime Manager is the operations and deployment solution of MuleSoft. It allows you to **deploy**, **manage**, **monitor** and scale Mule applications in the cloud (CloudHub) or **on-premises** environments.

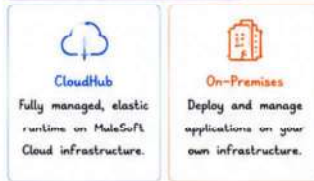
WHAT IS RUNTIME MANAGER?

- A centralized application management and monitoring solution.
- Deploy and manage Mule applications across **multiple environments**.
- Provides **operational insights**, alerts and logs.
- Supports **CloudHub** and **on-premises** deployments.

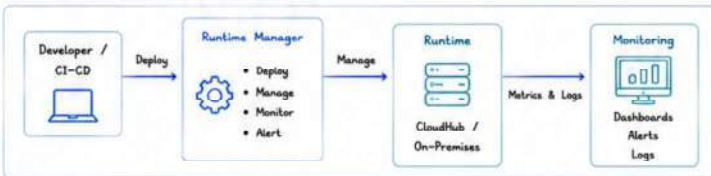
KEY CAPABILITIES



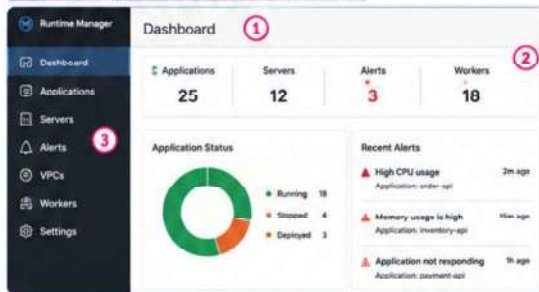
SUPPORTED DEPLOYMENTS



RUNTIME MANAGER ARCHITECTURE



RUNTIME MANAGER CONSOLE OVERVIEW

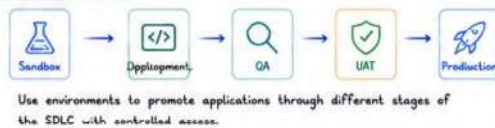


- ① **Dashboard**: Overview of applications, servers, alerts and workers.
- ② **Summary**: Key metrics and health at a glance.
- ③ **Navigation**: Access to all Runtime Manager features.

COMMON OPERATIONS

OPERATION	DESCRIPTION
Deploy Application	Deploy new or updated applications.
Start / Stop / Restart	Control application lifecycle.
Update Application	Update to a new version.
View Logs	Search and download application logs.
View Metrics	Monitor performance and health metrics.
Configure Alerts	Create alerts based on rules and thresholds.
Manage Workers	Scale and manage worker nodes.
Manage Servers	Add, edit or delete on-premises servers.

ENVIRONMENTS



MONITORING & ALERTING



BENEFITS

- Centralized management of all Mule applications.
- Improved operational visibility and faster issue resolution.
- Automated deployments and environment management.
- Scalable and highly available infrastructure.
- Better performance monitoring and proactive alerting.
- Supports DevOps and CI/CD practices.

BEST PRACTICES

- Use environments to promote applications.
- Configure alerts for critical metrics.
- Regularly review logs and metrics.
- Use application properties for configuration.
- Automate deployments using APIs / CLI.
- Follow naming conventions for applications and environments.

USE CASES

- Deploy APIs to CloudHub for high availability.
- Manage on-prem applications in data centers.
- Monitor API performance and SLA.
- Troubleshoot issues using logs and metrics.
- Automate deployments in CI/CD pipelines.



Runtime Manager helps you **deploy**, **manage** and **monitor** your Mule applications with efficiency, visibility and control.



2.7 ACCESS MANAGEMENT



Access Management in Anypoint Platform is used to manage users, roles, permissions and control access to Anypoint Platform resources. It provides secure access for individuals and teams based on their responsibilities.

WHAT IS ACCESS MANAGEMENT?

- Access Management is a security service that controls who can access Anypoint Platform and what they can do.
- It helps organizations manage users, roles, permissions, and resources in a centralized way.
- It ensures the right access to the right people for the right resources.
- It is a foundation for governance, security and compliance.

KEY CAPABILITIES



ACCESS MANAGEMENT HIERARCHY



ACCESS MANAGEMENT COMPONENTS

COMPONENT	DESCRIPTION
Users	Individuals who can log in to Anypoint Platform.
Roles	Collection of permissions. Assigned to users.
Permissions	Specific actions allowed on resources.
Business Groups	Logical container to organize resources and users.
Teams	Sub-groups within business groups.
Client Applications	Applications registered for API access (connected apps).

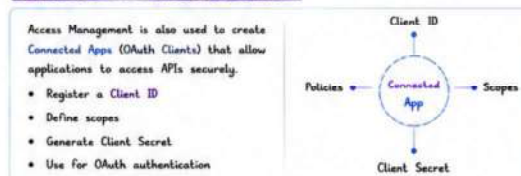
COMMON ROLES (EXAMPLES)

ROLE	DESCRIPTION
Organization Administrator	Full access to the organization. Manages users, roles, business groups, etc.
Business Group Administrator	Full access within a business group.
API Developer	Can design, build and manage APIs.
API Consumer	Can view and consume APIs.
Viewer	Read-only access to resources.
Custom Role	Role with specific permissions.

HOW ACCESS IS GRANTED



CONNECTED APPS (OAUTH CLIENTS)



BEST PRACTICES

- Follow least privilege principle.
- Create custom roles instead of using out-of-the-box roles.
- Regularly review users, roles and permissions.
- Use Business Groups to isolate teams and environments.
- Remove inactive users and unnecessary access.
- Use Connected Apps with appropriate scopes.
- Enable MFA for users.

ACCESS MANAGEMENT BENEFITS



Access Management ensures the right users have the right access to the right resources at the right time.



2.8

ANYPOINT MONITORING



Anypoint Monitoring provides **real-time visibility** into the health, performance and availability of your APIs and applications. It helps you **monitor**, **analyze** and **troubleshoot** issues quickly using dashboards, metrics, alerts and logs.

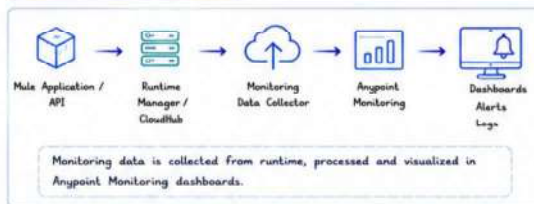
WHAT IS ANYPOINT MONITORING?

- A unified monitoring service for APIs and Mule applications.
- Provides real-time and historical insights on performance, availability and usage.
- Helps in proactive issue detection and faster troubleshooting.
- Part of Anypoint Platform and works with Runtime Manager, API Manager and other services.

KEY CAPABILITIES



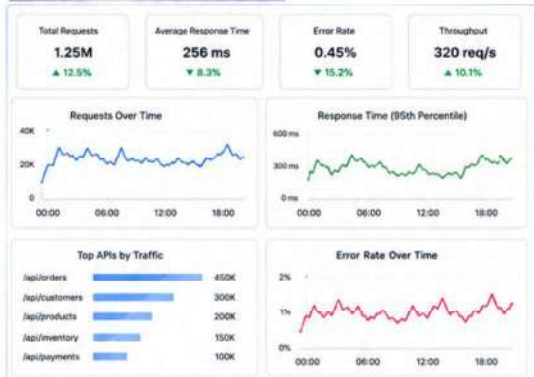
MONITORING ARCHITECTURE



MONITORING DATA

DATA TYPE	DESCRIPTION
Metrics	Numerical values collected over time (e.g., response time, throughput).
Logs	Detailed runtime events and messages for troubleshooting.
Events	State changes and important occurrences (e.g., deployments, alerts).
Traces	End-to-end request flow for distributed tracing.

DASHBOARD OVERVIEW (EXAMPLE)



COMMON METRICS

METRIC CATEGORY	EXAMPLES
Performance	Response Time, Latency, Throughput, CPU, Memory
Reliability	Error Rate, Faults, Timeouts, Availability
Usage	Total Requests, Successful Requests, API Invocations
Resources	CPU Utilisation, Memory Usage, Threads, GC Time
Business	Custom business metrics using custom dashboards

ALERTS

ALERT TYPE	DESCRIPTION
Threshold Alerts	Triggered when a metric crosses a defined threshold.
Anomaly Alerts	Triggered when an unusual pattern is detected.
Log Alerts	Triggered based on log patterns or log levels.
Synthetic Monitor Alerts	Triggered when a synthetic monitor check fails.

ALERT NOTIFICATION CHANNELS



BEST PRACTICES

- ✓ Monitor key business flows and critical APIs.
- ✓ Set meaningful thresholds to avoid alert fatigue.
- ✓ Use custom dashboards for role-based visibility.
- ✓ Review logs and metrics regularly.
- ✓ Enable distributed tracing for complex flows.
- ✓ Clean up old data and optimize retention policies.

LOG EXPLORER

Filters

Log Level

☐ ERROR

☐ WARN

☐ INFO

☐ DEBUG

Application

All

Time Range

Last 1 hour

Search logs...

 LIVE TAIL 

Timestamp	Level	Application	Message
2024-05-28 12:00:01	INFO	order-api	Order created successfully
2024-05-28 12:00:02	WARN	order-api	External service timeout
2024-05-28 12:00:03	ERROR	order-api	Failed to process payment
2024-05-28 12:00:04	INFO	inventory-api	Inventory updated
2024-05-28 12:00:05	INFO	customer-api	Customer retrieved

BENEFITS

- ✓ Real-time visibility into application and API health.
- ✓ Faster issue detection and resolution.
- ✓ Improved performance and reliability.
- ✓ Better user experience and customer satisfaction.
- ✓ Data-driven insights for capacity planning.



USE CASES

- ✓ Monitor API performance and SLA compliance.
- ✓ Detect and troubleshoot integration issues.
- ✓ Track system health and resource usage.
- ✓ Ensure business transactions are running smoothly.
- ✓ Capacity planning and usage trend analysis.



Anypoint Monitoring gives you complete **visibility** and **actionable insights** to keep your APIs and applications **healthy**, **fast** and **reliable**.



2.9.1

SECRETS MANAGER



Secrets Manager is used to securely store and manage sensitive information such as **passwords**, **API keys**, **certificates**, and **tokens** in Anypoint Platform.

WHAT IS SECRETS MANAGER?

- Secure storage service for sensitive information.
- Eliminates hard-coded credentials.
- Improves application security.
- Centralized secret management.
- Supports secure deployments.

WHY USE SECRETS MANAGER?



SECURITY
Protect sensitive information.



CENTRALIZED MANAGEMENT
Store secrets in one place.



ENCRYPTION
Secrets are securely encrypted.



COMPLIANCE
Meets security standards.



EASY ROTATION
Update secrets without code changes.



ACCESS CONTROL
Restrict access using roles.

SUPPORTED SECRETS



Database Passwords



API Keys



OAuth Tokens



Certificates

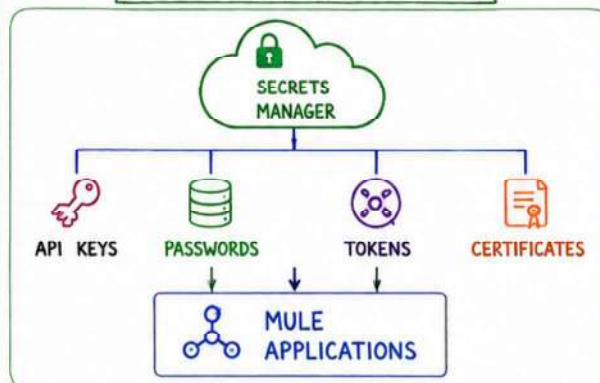


Client Secrets



Secure Properties

HIGH-LEVEL ARCHITECTURE



KEY TAKEAWAYS



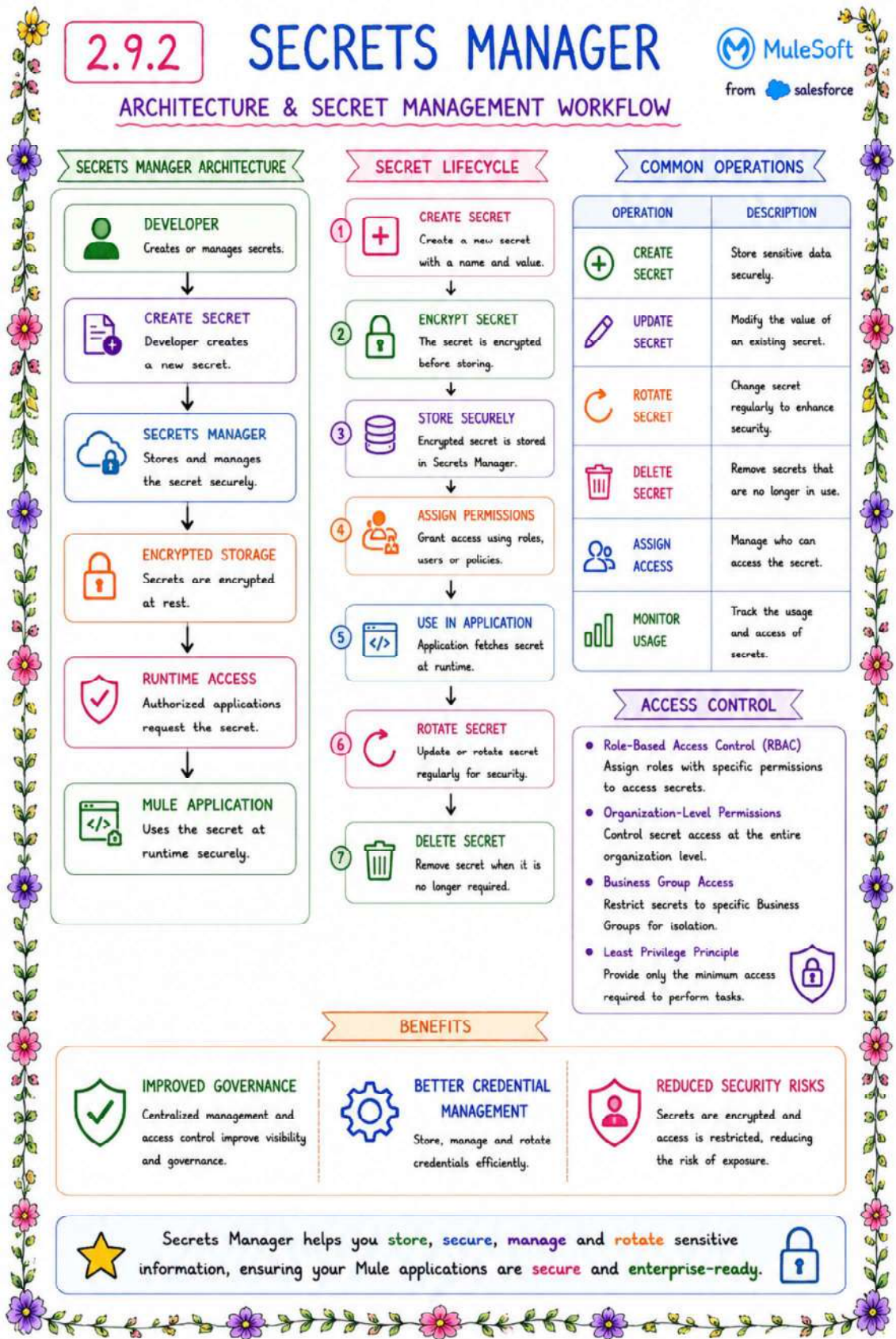
Centralized secret management.



Improved application security.



Secure and scalable deployments.



2.9.3

SECRETS MANAGER

BEST PRACTICES & REAL-TIME USE CASES

MuleSoft

from salesforce

BEST PRACTICES

✓ Never **hard-code** credentials in your application.

✓ Use **separate** secrets for each environment.

✓ **Rotate** secrets regularly to improve security.

✓ Apply **least privilege** access for users and applications.

✓ Use **RBAC** (Role-Based Access Control) for permissions.

✓ Audit secret usage regularly for monitoring and compliance.

REAL-TIME USE CASES

Database Credential Management

Store and manage database usernames and passwords securely.

API Authentication

Store API keys and tokens securely for external integrations.

Salesforce Connected App Secrets

Store client id, client secret and other Salesforce credentials.

OAuth Client ID & Client Secret Management

Securely manage OAuth credentials for authentication flows.

Secure CloudHub Deployments

Inject secrets securely in CloudHub at deployment and runtime.

CI/CD Secure Deployments

Use Secrets Manager in CI/CD pipelines for secure build and deploy.

REAL-TIME EXAMPLE

DATABASE PASSWORD

↓

SECRETS MANAGER

↓

RUNTIME MANAGER

↓

MULE APPLICATION

↓

DATABASE CONNECTION

ENVIRONMENT-BASED SECRET MANAGEMENT

DEV

Database Password - DEV

↓

QA

Database Password - QA

↓

UAT

Database Password - UAT

↓

PROD

Database Password - PROD

BENEFITS

BENEFIT	DESCRIPTION
SECURITY	Protect sensitive information from unauthorized access.
COMPLIANCE	Meets enterprise security and compliance standards.
SCALABILITY	Easily manage secrets across multiple environments.
MAINTAINABILITY	Update secrets without code changes or redeployments.
RELIABILITY	Secure and consistent access to secrets at runtime.
GOVERNANCE	Better visibility, control and auditing of secret usage.

FINAL TAKEAWAY

STORE → SECURE → ENCRYPT → MANAGE → ROTATE

★ Keep your Mule applications **secure** and **enterprise-ready** with Secrets Manager!

2.10 ANYPOINT MQ



Anypoint MQ is a fully managed message broker service that enables **reliable and asynchronous communication** between applications and systems.

It ensures messages are delivered **securely, in order, and at scale**.

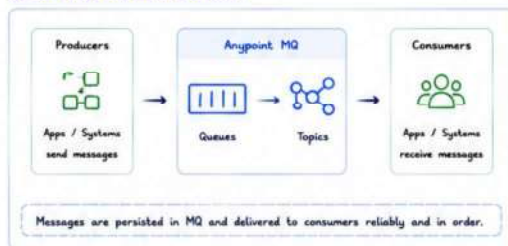
WHAT IS ANYPOINT MQ?

- A fully managed, **cloud-native** message broker service.
- Enables **asynchronous, decoupled** communication between producers and consumers.
- Guarantees message **delivery, durability, and ordering**.
- Supports **request-response, pub-sub, and fire-and-forget** patterns.
- Part of Anypoint Platform and integrates seamlessly with **Mule** applications.

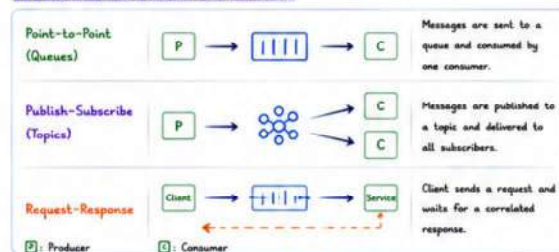
KEY CAPABILITIES



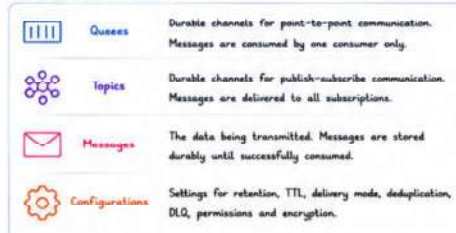
HOW ANYPOINT MQ WORKS



SUPPORTED MESSAGING PATTERNS



CORE COMPONENTS



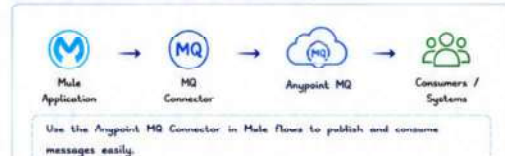
FEATURES HIGHLIGHTS

FEATURE	DESCRIPTION
Message Durability	Messages are persisted to disk for reliable delivery.
Ordering	Maintain strict order of messages within in queue or partition.
Acknowledgements	Manual or auto-ack to confirm successful processing.
Dead Letter Queue (DLQ)	Undeliverable messages are moved to DLQ for review.
Message TTL	Automatically expire messages after a defined time.
Deduplication	Prevent duplicate message processing.
Message Priorities	Control the order in which messages are delivered.
Throughput & Scale	High throughput with automatic scaling.

COMMON USE CASES



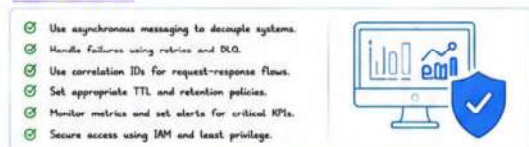
INTEGRATION WITH ANYPOINT PLATFORM



ANYPOINT MQ PLANS

PLAN	BEST FOR	
Standard	General purpose workloads with variable traffic.	Multiple regions
Premium	High throughput with low latency requirements.	High availability
Dedicated	Isolated capacity with dedicated virtual clusters for strict compliance and performance needs.	Encryption by default
		99.99% SLA

BEST PRACTICES



Anypoint MQ delivers **reliable, secure and scalable** messaging so your applications can communicate anytime, anywhere.



2.10.1

ANYPPOINT MQ



Anypoint MQ is a fully managed, cloud-native message broker service that enables **reliable** and **asynchronous** communication between applications and systems. It ensures messages are delivered **securely**, **in order**, and at **scale**.

WHAT IS ANYPOINT MQ?

- Fully managed cloud-native message broker.
- Enables **asynchronous** communication.
- Guarantees message **durability** and **ordering**.
- Supports multiple messaging patterns.
- Seamlessly integrates with Mule applications.

KEY CAPABILITIES



SUPPORTED MESSAGING PATTERNS



Messages are sent to a queue and consumed by one consumer.

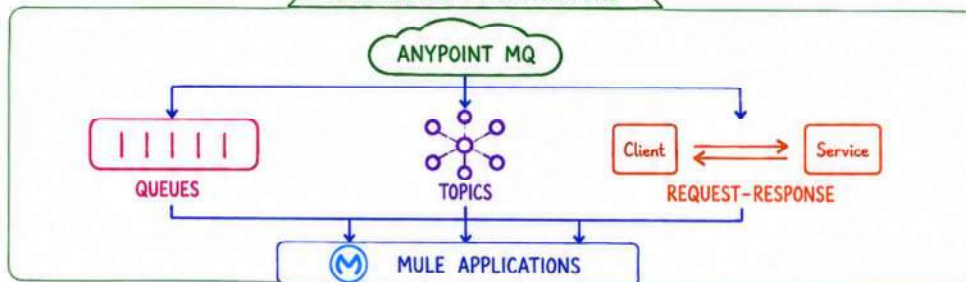


Messages are published to a topic and delivered to all subscribers.

**REQUEST-RESPONSE**

Client sends a request and waits for a correlated response.

HIGH-LEVEL ARCHITECTURE



KEY TAKEAWAYS



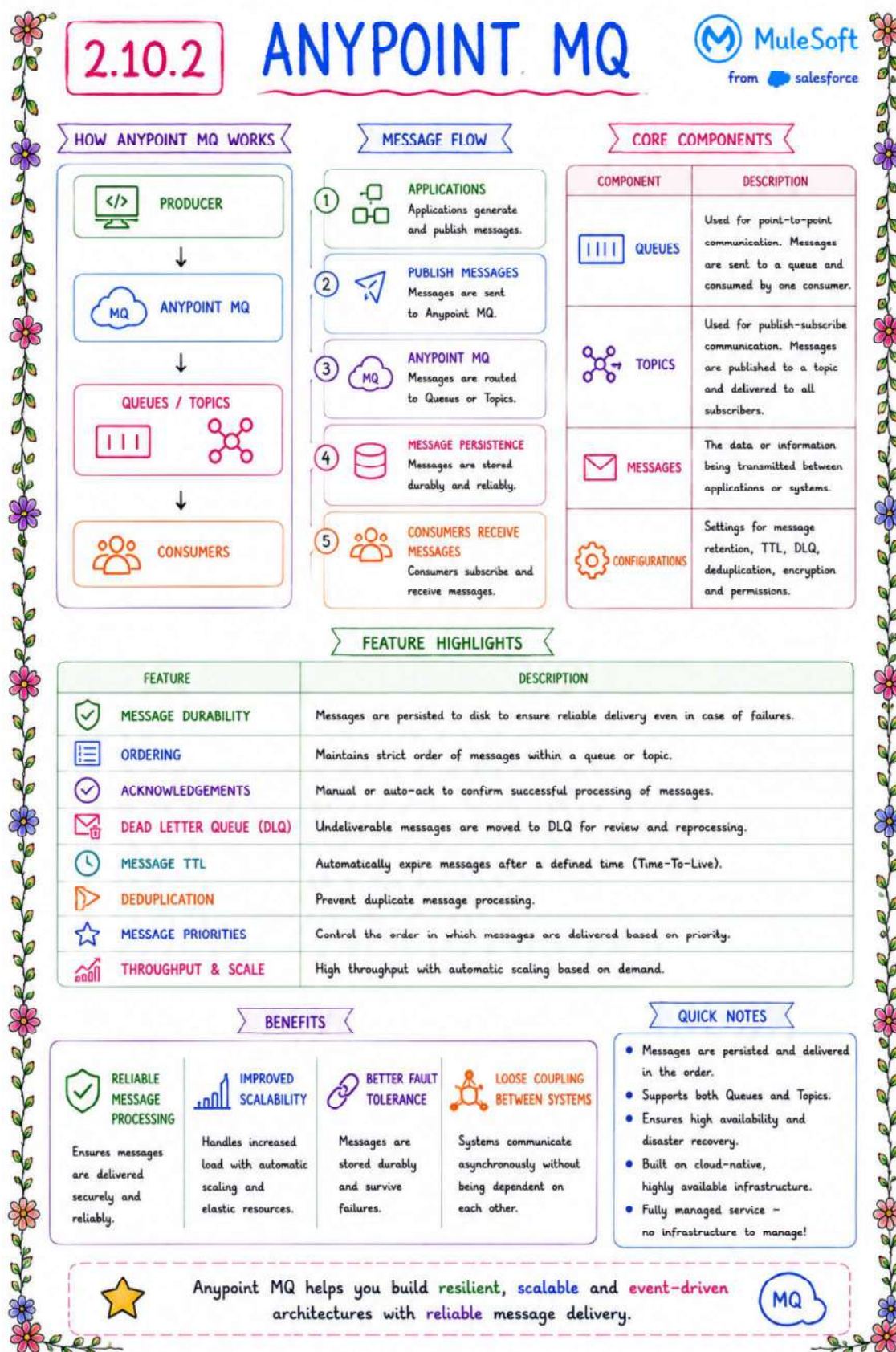
Reliable
messaging.



Asynchronous
communication.



Fully managed
cloud service.



2.10.3

ANYPOINT MQ

INTEGRATION, USE CASES & BEST PRACTICES

MuleSoft
from salesforce

COMMON USE CASES

SYSTEM INTEGRATION
Integrate multiple systems in an asynchronous way.

ORDER PROCESSING
Process orders reliably and handle spikes in traffic.

PAYMENT PROCESSING
Handle payment transactions securely and reliably.

EVENT STREAMING
Stream real-time events between applications.

MICROSERVICES COMMUNICATION
Enable loose coupling and asynchronous communication.

INTEGRATION WITH ANYPOINT PLATFORM

MULE APPLICATION
Build and run your applications.

MQ CONNECTOR
Use Anypoint MQ Connector to publish / consume messages.

ANYPOINT MQ
Fully managed message broker (Queues / Topics).

CONSUMERS / SYSTEMS
Downstream systems or applications consume the messages.

★ Use the Anypoint MQ Connector in Mule flows to publish and consume messages easily.

ANYPOINT MQ PLANS

PLAN	BEST FOR
STANDARD ★	General purpose workloads with variable traffic. Cost-effective and easy to get started.
PREMIUM ◇	High throughput workloads with low latency requirements.
DEDICATED ✓	Isolated capacity with dedicated virtual clusters for compliance and performance needs.

KEY BENEFITS

- Reliable and secure messaging
- Scalable and highly available
- Improved application resilience
- Asynchronous and decoupled communication
- Better performance and fault tolerance

REAL-TIME EXAMPLE - E-COMMERCE ORDER PLACEMENT

CUSTOMER PLACES ORDER
Customer places an order on the E-Commerce site.

EXPERIENCE API
Receives the order and validates the request.

PROCESS API
Processes the order and publishes it to Anypoint MQ.

ANYPOINT MQ QUEUE
Order message is stored in the queue reliably.

INVENTORY SERVICE
Consumes the message and updates inventory in real time.

PAYMENT SERVICE
Processes the payment and confirms the transaction.

NOTIFICATION SERVICE
Sends order confirmation to customer via Email / SMS / Push.

BEST PRACTICES

- ✓ Use asynchronous messaging to decouple systems.
- ✓ Handle failures using **retries** and **Dead Letter Queue (DLQ)**.
- ✓ Use **correlation IDs** for request-response flows.
- ✓ Set appropriate Time to Live (TTL) for messages.
- ✓ Monitor metrics and set alerts for critical KPIs.
- ✓ Secure access using **IAM** and least privilege.
- ✓ Design idempotent consumers to avoid duplicates.
- ✓ Test and validate message flows thoroughly.

ADDITIONAL BENEFITS

DECOUPLED ARCHITECTURE
Systems are loosely coupled and can evolve independently.

IMPROVED SCALABILITY
Scale services independently based on demand.

RELIABLE EVENT PROCESSING
Messages are delivered reliably and in order.

ENHANCED FAULT TOLERANCE
System continues to operate even in case of failures.

FINAL TAKEAWAY

→ → → →

PUBLISH **QUEUE** **DELIVER** **PROCESS** **SCALE**

Build **reliable**, **secure** and **scalable** applications with Anypoint MQ – your fully managed messaging service!

★ Anypoint MQ empowers your applications with **reliable**, **asynchronous** and **scalable** messaging – anytime, anywhere!

2.11.1

BUSINESS GROUPS & ENVIRONMENTS



Business Groups help organize teams, resources, and permissions within an organization, while **Environments** help manage applications across different stages of the SDLC such as **Development**, **QA**, and **Production**.

WHAT ARE BUSINESS GROUPS?



- Logical containers within an organization.
- Organize users, teams, APIs, and resources.
- Provide isolation between business units.
- Simplify governance and access control.

WHAT ARE ENVIRONMENTS?



- Isolated spaces within Business Groups.
- Manage application lifecycle.
- Separate development and production workloads.
- Support secure deployments.

KEY CAPABILITIES



ORGANIZATION

Structure teams and resources.



LIFECYCLE MANAGEMENT

Manage application promotion across environments.



ISOLATION & SECURITY

Separate teams and environments with strong security.



GOVERNANCE & CONTROL

Apply policies, quotas, and SLAs across groups and environments.



DEPENDENCY MANAGEMENT

Manage dependencies and promote safely between environments.

HIGH-LEVEL OVERVIEW



ORGANIZATION



BUSINESS GROUPS



ENVIRONMENTS



MULE APPLICATIONS

EXAMPLES



Business Group: Retail

Contains retail team, APIs, policies, and apps.



Environment: QA

Used for integration and functional testing before staging.



Environment: Production

Live environment handling real customer traffic securely.

KEY TAKEAWAYS



Business Groups organize ownership of resources.



Environments manage application deployments.



Together they provide governance, isolation and security.

2.11.2

BUSINESS GROUPS & ENVIRONMENTS

HIERARCHY & ENVIRONMENT LIFECYCLE

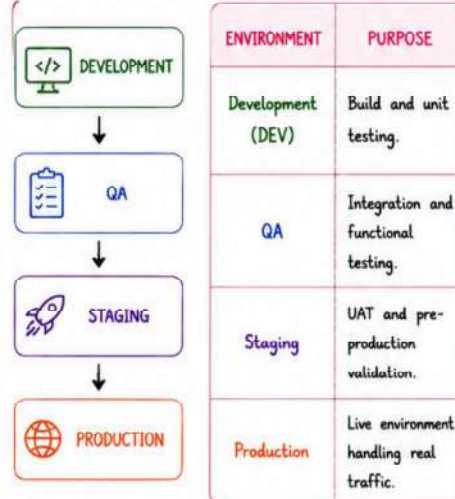


BUSINESS GROUPS HIERARCHY



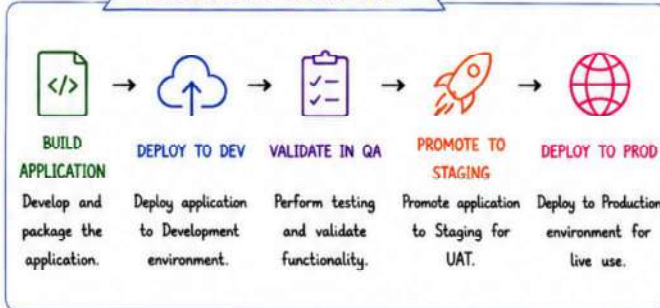
★ Each Business Group is isolated. Users and resources in one Group do not have access to another Group unless explicitly granted.

ENVIRONMENT LIFECYCLE



💡 Environments represent different stages of the application lifecycle.

PROMOTION WORKFLOW



BENEFITS

- ✓ Controlled application promotion.
- ✓ Better SDLC management.
- ✓ Improved deployment reliability.
- ✓ Secure and isolated environments.
- ✓ Reduced risk to production.

KEY TAKEAWAYS

APPLICATIONS MOVE SAFELY
Applications move across environments in a controlled and secure manner.

UNIQUE PURPOSE FOR EACH ENVIRONMENT
Every environment has a specific role in the SDLC lifecycle.

EASY-TO-MANAGE PIPELINES
Deployment pipelines become structured, repeatable and easier to manage.



Business Groups provide organization and governance, Environments help manage the application lifecycle efficiently!





2.12.1

CONNECTED APPS

 MuleSoft
from  salesforce

Connected Apps allow external applications to **securely** connect to Anypoint Platform. They provide standardized authentication and authorization using **OAuth 2.0** so that applications can securely access APIs and platform resources in a **controlled** manner.

WHAT ARE CONNECTED APPS?

- A Connected App represents an external application that needs access to Anypoint Platform resources.
- It enables secure communication between external applications and MuleSoft services.
- Uses **OAuth 2.0** and **OpenID Connect (OIDC)** for authentication and authorization.
- Provides granular access control using **scopes**, **policies**, and **permissions**.
- Helps organizations securely expose APIs and platform capabilities.



KEY CAPABILITIES

 **SECURE AUTHENTICATION**

- Supports **OAuth 2.0** and **OIDC**.
- Provides standards-based authentication.
- Uses access tokens for secure API access.

 **GRANULAR AUTHORIZATION**

- Controls which resources an application can access.
- Uses scopes for fine-grained permissions.
- Prevents unauthorized access to APIs and services.

 **VISIBILITY & CONTROL**

- Monitor usage of Connected Apps.
- Track access patterns and API consumption.
- Manage application lifecycle effectively.

 **LIFECYCLE MANAGEMENT**

- Create and configure Connected Apps easily.
- Update client credentials securely.
- Revoke access whenever required.

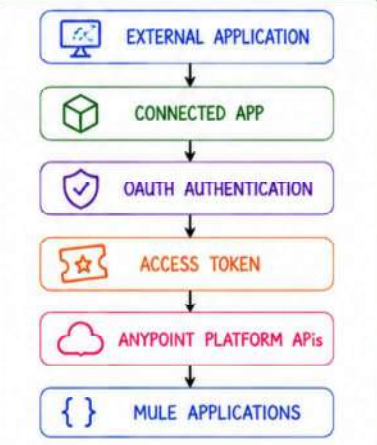
 **INTEGRATION READY**

- Works seamlessly with Mule applications.
- Provides built-in support for Anypoint Platform services.
- Simplifies third-party integrations.

 **POLICY ENFORCEMENT**

- Apply security and access policies.
- Configure rate limiting and access restrictions.
- Improve API governance and compliance.

HIGH-LEVEL ARCHITECTURE



SUPPORTED AUTHENTICATION STANDARDS

AUTHENTICATION STANDARD	PURPOSE
 OAuth 2.0	Secure authentication and authorization
 OpenID Connect (OIDC)	User identity management
 Access Token	Secure API access
 Refresh Token	Renew access tokens
 Client Credentials	Server-to-server authentication

KEY BENEFITS

- Secure API access.
- Centralized access management.
- Granular authorization controls.
- Easy integration with MuleSoft services.
- Improved governance and security.
- Enterprise-ready authentication mechanisms.

KEY TAKEAWAYS

- Connected Apps provide secure and controlled access to Anypoint Platform resources.
- OAuth 2.0** is the primary authentication mechanism used.
- Granular authorization improves API security.
- Connected Apps simplify external application integrations with MuleSoft.

FINAL SUMMARY

CONNECTED APPS



SECURE • CONTROLLED • SCALABLE

Enables external applications to securely access MuleSoft APIs and platform resources.

2.12.2

CONNECTED APPS

OAUTH FLOWS & CONFIGURATION

 MuleSoft

from  salesforce



EXTERNAL APPLICATION

An external application requests access.



AUTHENTICATE USER

User is authenticated using OAuth 2.0 / OIDC.



CONNECTED APP

The Connected App is validated and authorized.



GENERATE ACCESS TOKEN

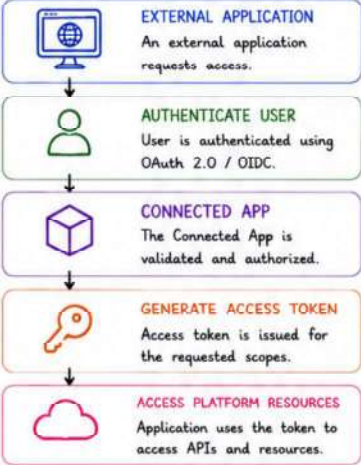
Access token is issued for the requested scopes.



ACCESS PLATFORM RESOURCES

Application uses the token to access APIs and resources.

OAUTH FLOW OVERVIEW



OAUTH 2.0 FLOWS SUPPORTED

FLOW TYPE	DESCRIPTION
Authorization Code (Recommended)	Used by web and mobile applications where a user is present to grant access.
Client Credentials	Used by server-to-server applications without a user context.
Resource Owner Password Credentials	Used when the application has the user's credentials (legacy scenarios).
Refresh Token	Obtains new access tokens using a refresh token without re-authenticating

CONNECTED APP CONFIGURATION

CONFIGURATION	PURPOSE
Basic Information	Name, description, logo, contact email.
Application Type	Web, Native, API, Server-to-Server, or Single Page App.
OAuth Settings	Grant types, redirect URIs, token settings, refresh token validity.
Scopes	Define scopes to control access to APIs and platform resources.
Policies	Apply and enforce policies for access control and security.
Client Credentials	Client ID and Client Secret (or certificate) used to authenticate the app.

CONNECTED APPS OVERVIEW (EXAMPLE)

NAME	CLIENT ID	TYPE	SCOPES	STATUS	LAST MODIFIED
Mobile App	c1a2b3c4...	Native	api.read api.write	Active	May 02, 2024
Partner Portal	d4e5f6g7...	Web	api.read user.read	Active	May 01, 2024
Inventory Service	h7i8j9k0...	API (Server-to-Server)	api.read api.write	Active	Apr 30, 2024
Legacy App	i0m1n2o3...	Web	api.read	Inactive	Apr 28, 2024
Test App	p4q5r6s7...	Native	api.read	Revoked	Apr 10, 2024

Total Connected Apps

24

Active Apps

18

Inactive Apps

3

Revoked Apps

3

BENEFITS

- Standardized authentication using OAuth 2.0 and OIDC.
- Flexible support for multiple application types.
- Granular scopes for fine-grained authorization.
- Centralized management of external applications.
- Better visibility and control over application access.

KEY POINTS

- Connected Apps act as a secure bridge between external applications and Anypoint Platform.
- OAuth 2.0 flows ensure standardized and secure access.
- Scopes and policies provide granular authorization.
- Proper configuration of tokens and credentials is essential for secure integrations.
- Monitor the status and usage of Connected Apps regularly.



QUICK SUMMARY



EXTERNAL APP

External application requests access.

→



AUTHENTICATE

User authenticates using OAuth / OIDC.

→



CONNECTED APP

Connected App is validated.

→



ACCESS TOKEN

Access token is generated.

→



PLATFORM ACCESS

Application accesses APIs and resources.

★

Connected App configuration and OAuth flows ensure secure, controlled, and reliable access to Anypoint Platform resources.



2.12.3

CONNECTED APPS

ACCESS CONTROL, BEST PRACTICES & BENEFITS



MuleSoft
from salesforce

ACCESS CONTROL USING SCOPES

SCOPE	ACCESS
 api.read	Read access to APIs
 api.write	Write access to APIs
 user.read	Read user profile information
 mq.read	Read access to Anypoint MQ resources
 mq.write	Write access to Anypoint MQ resources

Scopes help you define and control what a Connected App can access in the platform.

BEST PRACTICES

 Create separate Connected Apps for each application or integration.

 Use the least privilege principle with appropriate scopes.

 Rotate Client Secrets regularly and revoke unused credentials.

 Use short token lifetimes and refresh tokens when appropriate.

 Monitor usage, token usage, and API access patterns regularly.

 Apply policies for rate limiting, IP restrictions, and anomaly detection.

 Remove or revoke unused or inactive Connected Apps.

REAL-TIME EXAMPLE

 Salesforce Application (External App)

 Connected App in Anypoint Platform

 OAuth 2.0 Token (Access Token)

 API Manager (Policy Enforcement)

 Mule API (Protected Resource)

 Secure Access Granted (API Response)

This flow ensures secure and controlled access to APIs.

BENEFITS



ENHANCED SECURITY

- OAuth-based authentication.
- Secure token-based access to APIs.



BETTER GOVERNANCE

- Centralized access management.
- Enforce policies and access controls.



FASTER INTEGRATION

- Easy onboarding of external applications.
- Reduces integration time and effort.



SCALABILITY

- Supports multiple applications.
- Scales with enterprise requirements.



COMPLIANCE

- Security and audit readiness.
- Helps meet regulatory requirements.



OPERATIONAL EFFICIENCY

- Easy management and monitoring.
- Better visibility of application usage.

CONNECTED APP OVERVIEW (KEY METRICS)

TOTAL APPS24

ACTIVE APPS18

INACTIVE APPS3

REVOKED APPS3

NAME	CLIENT ID	TYPE	SCOPES	STATUS	LAST MODIFIED
 Mobile App	c1a2b3c4...	Native	api.read api.write	Active	May 02, 2024
 Partner Portal	d4e5f6g7...	Web	api.read user.read	Active	May 01, 2024
 Inventory Service	h7i8j9k0...	API (Server-to-Server)	api.read api.write	Active	Apr 30, 2024
 Legacy App	l0m1n2o3...	Web	api.read	Inactive	Apr 28, 2024
 Test App	p4q5r6s7...	Native	api.read	Revoked	Apr 10, 2024

Regularly review and manage Connected Apps to keep your platform secure.

KEY POINTS TO REMEMBER

- ★ Scopes define what resources the application can access.
- ★ Least privilege access improves security.
- ★ Short-lived tokens and refresh tokens enhance security.
- ★ Policies and rate limits protect your APIs.
- ★ Monitor and review Connected Apps regularly.
- ★ Revoke or remove unused applications.

FINAL TAKEAWAY

CONNECTED APPS

 →  →  →  → 

AUTHENTICATE → AUTHORIZE → GENERATE TOKEN → ACCESS RESOURCES → MONITOR & CONTROL

Secure • Controlled • Scalable

Connected Apps enable secure, controlled, and scalable access for external applications to MuleSoft APIs and Anypoint Platform resources.

Connected Apps provide secure, controlled, and scalable access for external applications to interact with Anypoint Platform and its APIs.

3.1.1

WHAT IS RAML?



RAML (RESTful API Modeling Language) is an open API specification language used to **design**, **document** and **publish** RESTful APIs in a standard, machine-readable format. It follows a **design-first** approach for building APIs on **Anypoint Platform**.

INTRODUCTION TO RAML

- RAML stands for **RESTful API Modeling Language**.
- It is a **YAML-based** language for describing RESTful APIs.
- It provides a standardized way to define API structure, resources, methods, requests, responses, parameters, security and more.
- RAML files have a **.raml** extension.
- RAML is the foundation for all MuleSoft APIs and is supported across **Anypoint tools**.



WHY RAML?



DESIGN-FIRST APPROACH

Design APIs before implementation.



STANDARDIZED

Uses a standard format for all APIs.



AUTOMATION READY

Machine-readable format enables automation.



BETTER COLLABORATION

Easy for teams to understand, review and collaborate.



REUSABILITY

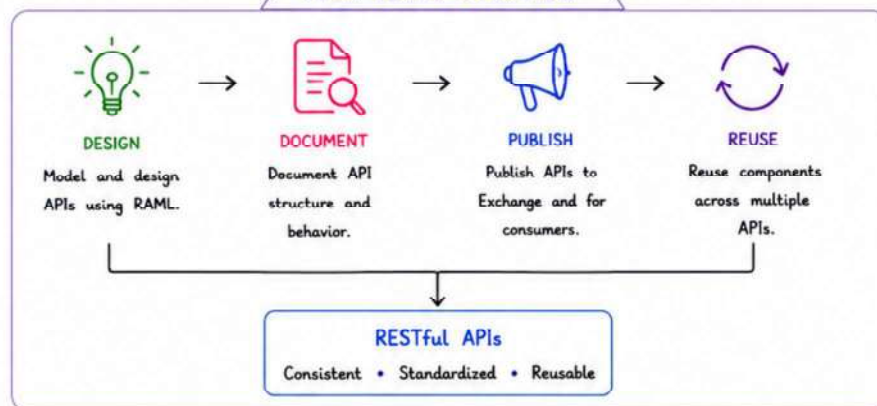
Use libraries, traits and resource types to reuse.



RICH DOCUMENTATION

Auto-generates interactive API documentation.

HIGH-LEVEL OVERVIEW



KEY TAKEAWAYS



RAML promotes a **design-first** approach for building APIs.



It is a **YAML-based** specification language for RESTful APIs.



It helps create **standardized, reusable** and well-documented API definitions.



RAML helps you **design**, **document** and **publish** high-quality RESTful APIs in a **consistent**, **secure** and **scalable** way on **Anypoint Platform**.





3.1.3

WHAT IS RAML?

BENEFITS, EXAMPLE & BEST PRACTICES



BENEFITS OF USING RAML

BENEFIT	DESCRIPTION
Consistency	Ensures consistent and standardized API definitions.
Faster Development	Auto-generates documentation and code scaffolds.
Better Governance	Enforces policies, standards and best practices.
Greater Reusability	Reuse traits, resource types, libraries and fragments.
Improved Collaboration	Clear API contracts for developers, testers and business stakeholders.
Future Ready	Industry standard (OAS compatible via conversion).

EXAMPLE: SIMPLE RESOURCE IN RAML

```
#%RAML 1.0
title: Sample API
version: v1
baseUri: https://api.example.com/{version}

/users:
  get:
    description: Get list of users
    responses:
      200:
        body:
          application/json:
            example:
              {
                "id": 1, "name": "John"
              }

```

WHAT THIS RAML DEFINES:

- ✓ API title, version and base URI
- ✓ A resource: /users
- ✓ A GET method to retrieve users
- ✓ A 200 OK response with JSON example
- ✓ A standard and machine-readable API contract

NOTES

- RAML is YAML-based.
- Indentation is important.
- Use 2 spaces per level.
- RAML 1.0 is the current stable version.

RAML BEST PRACTICES

FOLLOW DESIGN-FIRST APPROACH Design and validate the API before writing the implementation.	USE MEANINGFUL RESOURCE NAMES Use clear, consistent and business-friendly resource names.	REUSE TRAITS AND RESOURCE TYPES Promote reusability and reduce duplication using traits and types.	MAINTAIN PROPER INDENTATION Use 2 spaces per level and maintain correct YAML indentation.	USE RAML 1.0 SPECIFICATIONS Use RAML 1.0 for better compatibility and future support.	DOCUMENT APIS THOROUGHLY Add descriptions, examples, responses and error codes for clarity.
---	---	--	---	---	---

FINAL TAKEAWAY

RAML



CONSISTENT • SECURE • SCALABLE

Build better RESTful APIs using the Design-First approach with RAML on Anypoint Platform.



QUICK REFERENCE NOTES

- ✓ RAML is an open API specification language.
- ✓ It is YAML-based and easy to read.
- ✓ Supports design-first API development.
- ✓ Generates documentation and mocks automatically.
- ✓ Ensures consistent, reusable and well-documented APIs.
- ✓ Fully integrated with Anypoint Platform tools.



RAML helps you **design**, **document** and **publish** high-quality RESTful APIs in a **consistent**, **secure** and **scalable** way.



RAML helps you **design**, **document** and **publish** high-quality RESTful APIs in a **consistent**, **secure** and **scalable** way on Anypoint Platform.



3.2.1

RAML STRUCTURE OVERVIEW

from salesforce

INTRODUCTION

- ☆ A RAML file follows a standard structure to describe a **RESTful API** in a machine-readable format.
- ☆ It defines important API metadata such as the **title**, **version**, **base URI**, **resources**, **methods**, and **reusable components**.
- ☆ A well-structured RAML file improves **readability**, **consistency**, and API **maintainability**.

WHAT IS RAML STRUCTURE?

- RAML provides a standardized way to define the structure of REST APIs.
- It organizes API information in a clear and modular format.
- It supports API documentation, reusability, security definitions, and data modeling.
- RAML follows a **Design-First** approach, where the API contract is designed before implementation.
- It is widely used across MuleSoft and Anypoint Platform tools.

KEY ELEMENTS OF RAML STRUCTURE

ELEMENT	REQUIRED	PURPOSE
##RAML 1.0	Yes	Specifies the RAML version
title	Yes	Defines the API name
version	Yes	Specifies the API version
baseUri	Yes	Defines the API base URL
mediaType	No	Defines the default payload type
documentation	No	Provides API documentation
resources	Yes	Defines API endpoints
securitySchemes	No	Defines authentication mechanisms
traits & resourceTypes	No	Supports reusable API components
libraries & includes	No	Imports external RAML modules

MANDATORY ELEMENTS

- ✓ **##RAML 1.0**
- ✓ **title**
- ✓ **version**
- ✓ **baseUri**

OPTIONAL ELEMENTS

- **mediaType**
- **documentation**
- **resources** (practically required)
- **securitySchemes**
- **traits**
- **resourceTypes**
- **libraries**
- **includes**

WHY IS RAML STRUCTURE IMPORTANT?

- Standardization**
Provides a common structure for all APIs.
- Better Maintainability**
Makes APIs easier to understand and manage.
- Improved Collaboration**
Developers and architects can work using the same API contract.
- Reusability**
Promotes the use of reusable components such as traits and libraries.
- Better Documentation**
Enables automatic generation of API documentation.
- API Governance**
Supports organizational API standards and policies.

KEY BENEFITS

- ✓ Consistent API design
- ✓ Easy API maintenance
- ✓ Improved collaboration among teams
- ✓ Supports modular API development
- ✓ Simplifies API documentation
- ✓ Provides better governance and standardization

QUICK NOTES

- ☆ RAML files are **YAML-based**.
- ☆ Use proper indentation throughout the file.
- ☆ Follow the **Design-First** approach whenever possible.
- ☆ Organize RAML files into reusable and modular components.
- ☆ Keep the structure simple and readable.

KEY TAKEAWAYS

- ☆ RAML follows a standardized API structure.
- ☆ Mandatory and optional elements provide flexibility.
- ☆ A well-designed RAML file improves API quality and maintainability.
- ☆ Proper organization makes APIs easier to build, document, and manage.

FINAL SUMMARY

Define

↓

Organize

↓

Reuse

Clear • Modular • Maintainable APIs

HIGH-LEVEL RAML STRUCTURE

★

A well-structured RAML file is the foundation of building consistent, reusable, and enterprise-grade RESTful APIs on Anypoint Platform.

3.2.2

RAML STRUCTURE EXPLAINED



MuleSoft
from salesforce

COMPLETE RAML STRUCTURE (EXAMPLE)

```

1  ##RAML 1.0
2  title: Sample API
3  version: v1
4  baseUrl: https://api.example.com/{version}
5  mediaType: application/json
6  documentation:
7    title: Overview
8    content: |
9      This API allows you to manage users.
10 /users:
11   get:
12     description: Get all users
13     responses:
14       200:
15         body:
16           application/json:
17             type: User[]
18   post:
19     description: Create a user
20     body:
21       application/json:
22         type: User
23     responses:
24       201:
25         body:
26           application/json:
27             type: User

```

STRUCTURE EXPLAINED

#	ELEMENT	DESCRIPTION
1	##RAML 1.0	Specifies the RAML version (1.0).
2	title	Defines the name of the API.
3	version	Specifies the version of the API (e.g., v1, v2).
4	baseUrl	The base URL for the API endpoints. {version} is a URI parameter.
5	mediaType	Default media type for requests and responses.
6	documentation	Provides additional information about the API.
7	/users	Resource (collection) that groups related endpoints.
8	methods (get, post)	HTTP methods available for the resource.
9	responses	Possible response status codes with response body.
10	body	Defines the request or response payload structure.

RAML STRUCTURE FLOW



API IDENTIFICATION

- ##RAML 1.0
- title
- version



API CONFIGURATION

- baseUrl
- mediaType
- documentation



API INFORMATION

- resources
- methods
- responses



DATA DEFINITIONS

- types
- traits
- resourceTypes



API IMPLEMENTATION

- endpoints
- request / response
- examples



SECURITY & REUSABILITY

- securitySchemes
- traits / resourceTypes
- libraries / includes

★ The above flow shows the logical order of a RAML file. In actual files, sections can appear in different order.

BEST PRACTICES

- ✓ Keep the RAML file modular using libraries and includes.
- ✓ Use meaningful titles and descriptions.
- ✓ Define reusable data types, traits and resource types.
- ✓ Follow consistent naming conventions.
- ✓ Validate RAML using API Designer or CLI.
- ✓ Maintain the logical structure for readability.
- ✓ Document all sections with examples.

KEY TAKEAWAYS

- ☆ RAML has a well-defined structure.
- ☆ It starts with API identification and configuration.
- ☆ Resources and methods define the API behavior.
- ☆ Responses and data types provide the API contract.
- ☆ Reusable components improve maintainability.

★ A well-structured RAML file leads to **clear API design**, **better collaboration** and **easier maintenance**.

3.2.3

RAML STRUCTURE

EXAMPLES, BEST PRACTICES & RULES


MuleSoft
 from  **salesforce**

MINIMAL RAML EXAMPLE

```

#%RAML 1.0
title: Minimal API
version: v1

/ping:
  get:
    responses:
      200:
        body:
          application/json:
            example:
              {
                "message": "pong"
              }
          
```

★ This is the smallest valid RAML API.

COMMON FILE EXTENSION



.raml

RAML files
use the
.raml extension.

WHERE RAML STRUCTURE IS USED



Anypoint Design Center

Design, validate and visualize APIs collaboratively.



API Manager

Apply policies and manage the lifecycle of APIs.



Runtime Manager

Deploy and manage APIs in runtime environments.



Exchange

Publish and share APIs with developers and consumers.



API Console

Generate interactive API documentation.

INDENTATION RULES

CORRECT INDENTATION ✓

```

/users:
  get:
    responses:
      200:
        body:
          application/json:
            type: User[]
          
```

INCORRECT INDENTATION ✗

```

/users:
  get:
    responses:
      200:
        body:
          application/json:
            type: User[]
          
```

💡 • Use 2 spaces for indentation (not tabs).

• Wrong indentation can cause parsing errors.

RAML BEST PRACTICES

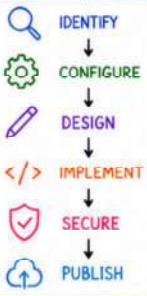
- ✓ Use RAML 1.0 specifications.
- ✓ Maintain proper indentation.
- ✓ Keep APIs modular and reusable.
- ✓ Reuse traits and resource types.
- ✓ Define meaningful titles and descriptions.
- ✓ Document APIs thoroughly with examples.
- ✓ Follow standard naming conventions.
- ✓ Validate RAML using API Designer or CLI.
- ✓ Follow consistent data types and formats.
- ✓ Organize RAML files using libraries and includes.

★ Follow a consistent structure, proper indentation and best practices to build **clear**, **reusable** and **maintainable** APIs.

KEY TAKEAWAYS

- ☆ RAML structure starts with API identification and configuration.
- ☆ Resources and methods define API behavior and endpoints.
- ☆ Responses and data types define the API contract.
- ☆ Reusable components (traits, resource types, libraries) improve modularity.
- ☆ Proper indentation and structure ensure better readability.
- ☆ A well-structured RAML file improves collaboration and maintenance.

FINAL TAKEAWAY



```

graph TD
  IDENTIFY --> CONFIGURE
  CONFIGURE --> DESIGN
  DESIGN --> IMPLEMENT
  IMPLEMENT --> SECURE
  SECURE --> PUBLISH
          
```

★ **RAML STRUCTURE**

Clear • Consistent • Reusable APIs

A well-structured RAML file makes APIs easier to design, build, document and manage. 💡

★ A well-structured RAML file leads to **better API design**, **easier collaboration** and **improved maintainability**. 

3.3 RESOURCES & METHODS

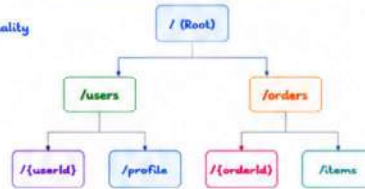


In RAML, a **Resource** represents an endpoint (URI) and **Methods** define the actions that can be performed on that resource using standard **HTTP verbs**.

WHAT ARE RESOURCES?

- Resources represent the data or functionality exposed by your API.
- Each resource is identified by a **URI**.
- Resources can be nested to represent hierarchical relationships.

EXAMPLE: RESOURCE HIERARCHY



COMMON HTTP METHODS IN RAML

GET	Retrieve data from a resource (Read operation)	
POST	Create a new resource (Create operation)	
PUT	Update an entire resource (Replace operation)	
PATCH	Partially update a resource (Partial update)	
DELETE	Delete a resource (Delete operation)	
HEAD	Retrieve headers only (No response body)	
OPTIONS	Get supported methods (Discover capabilities)	

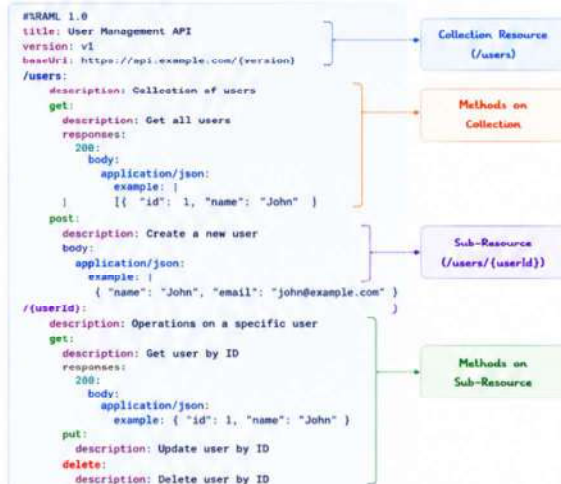
EXAMPLE: RAML RESOURCE STRUCTURE



RESOURCE & METHOD EXAMPLES

RESOURCE	METHOD	DESCRIPTION	USE CASE
/users	GET	Get all users	List all users
	POST	Create a new user	Add a new user
/users/{userId}	GET	Get user by ID	Retrieve specific user
	PUT	Update user by ID	Replace user details
	PATCH	Partially update user	Update few fields
	DELETE	Delete user by ID	Remove user
/users/{userId}/orders	GET	Get orders of a user	List user's orders

COMPLETE RAML EXAMPLE WITH RESOURCES & METHODS



BEST PRACTICES

- Use nouns for resources (e.g., /users, /orders).
- Follow consistent naming conventions.
- Use plural for collections (e.g., /users, /products).
- Use appropriate HTTP methods for operations.
- Keep URIs clean, simple and meaningful.
- Avoid action words in URIs (e.g., avoid /getUsers).

URI PARAMETERS IN RESOURCES

- Use URI parameters to identify specific resources.
- Defined using curly braces {param}.

Example URI	
/users/{userId}/orders/{orderId}	
Meaning	Order of a specific user by userId and orderId

QUICK REFERENCE: HTTP METHODS

METHOD	PURPOSE	IDEMPOTENT
GET	Read/Retrieve data	Yes
POST	Create new resource	No
PUT	Replace/Update resource	Yes
PATCH	Partially update resource	Yes
DELETE	Remove resource	Yes

COMMON USE CASES

/users	→ Get all users, Create user
/users/{userId}	→ Manage a specific user
/orders	→ Manage orders
/products/{productId}	→ Product details

SUMMARY

- Resources define the API endpoints (URIs).
- Methods define the actions on resources.
- Use standard HTTP verbs for consistency.
- Organize resources hierarchically.
- Follow best practices for clean APIs.



Well-structured **Resources** and correct use of **Methods** make your APIs clear, consistent, easy to consume and maintain on **Anypoint Platform**.



3.3.1

WHAT ARE RESOURCES & METHODS?



In RAML, a **Resource** represents an API endpoint (URI), and **Methods** define the operations that can be performed on that resource using standard **HTTP** verbs.

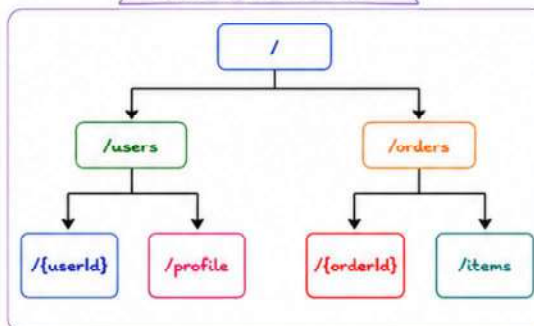
WHAT ARE RESOURCES?

- ★ Resources represent API endpoints.
- ★ Every resource is identified by a **URI**.
- ★ Resources expose **business functionality**.
- ★ Resources can contain **sub-resources**.

EXAMPLES OF RESOURCES

/users **/orders** **/products**

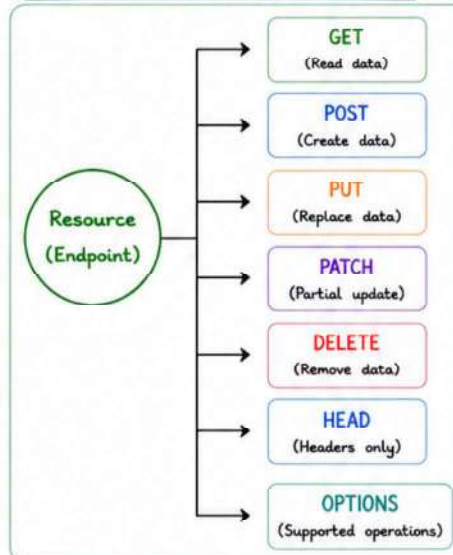
RESOURCE HIERARCHY



COMMON HTTP METHODS

METHOD		PURPOSE
GET		Retrieve data from a resource (Read operation).
POST		Create a new resource (Create operation).
PUT		Replace an entire resource (Replace operation).
PATCH		Update part of a resource (Partial update).
DELETE		Delete a resource (Delete operation).
HEAD		Retrieve headers only (No response body).
OPTIONS		Get supported methods (Discover capabilities).

RESOURCE & METHOD RELATIONSHIP



WHY RESOURCES & METHODS ARE IMPORTANT?

- ★ They form the foundation of any **RESTful** API.
- ★ Resources define **what data** is available.
- ★ Methods define the **actions** that can be performed.
- ★ Standard HTTP methods ensure **consistency**.
- ★ They promote a **clean and predictable** API design.

KEY TAKEAWAYS

- ✓ Resources define API **endpoints** (URIs).
- ✓ Methods define the **operations** on resources.
- ✓ Use **standard HTTP** verbs for consistency.
- ✓ Follow **RESTful** principles for better APIs.
- ✓ A well-designed structure improves **readability**, **maintainability** and **scalability**.



Resources define API endpoints and **Methods** define actions.
Together they build **powerful**, **consistent** and **easy-to-use** APIs!



3.3.2

RAML RESOURCE STRUCTURE & EXAMPLES

 MuleSoft
from  salesforce

RAML RESOURCE EXAMPLE

```
/users:
  get:
    description: Get all users

  post:
    description: Create user
```

★ This defines a collection resource `"/users"` with `GET` and `POST` methods.

SUB-RESOURCE EXAMPLE

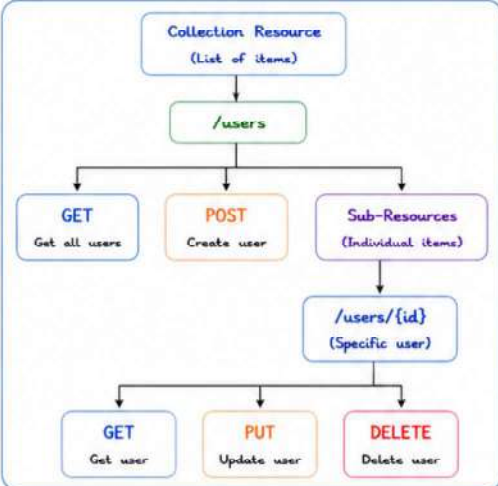
```
/users/{userId}:
  get:
    description: Get user details

  put:
    description: Update user

  delete:
    description: Delete user
```

★ This defines a sub-resource `"/users/{userId}"` with `GET`, `PUT` and `DELETE` methods.

COMPLETE RESOURCE STRUCTURE



RESOURCE & METHOD EXAMPLES

	RESOURCE	METHOD	DESCRIPTION
	<code>/users</code>	<code>GET</code>	Get all users
	<code>/users</code>	<code>POST</code>	Create a new user
	<code>/users/{id}</code>	<code>GET</code>	Get user by ID
	<code>/users/{id}</code>	<code>PUT</code>	Update user details
	<code>/users/{id}</code>	<code>DELETE</code>	Delete user by ID
	<code>/users/{id}/orders</code>	<code>GET</code>	Get orders of a user

BENEFITS OF WELL-STRUCTURED RESOURCES

- ✔ Provides a clear and organized API structure.
- ✔ Improves readability and understandability.
- ✔ Makes APIs easier to extend and maintain.
- ✔ Promotes reusability through sub-resources.
- ✔ Supports RESTful design principles.

KEY TAKEAWAYS

- ★ Resources represent endpoints (URIs) in an API.
- ★ Methods define operations on those resources.
- ★ Sub-resources help in representing hierarchical data.
- ★ Use proper HTTP methods for each operation.
- ★ A good structure leads to better API design.

★ A well-structured resource hierarchy and correct method usage result in clean, consistent and maintainable APIs. 

3.3.3

RESOURCES & METHODS

QUICK REFERENCE, COMMON USE CASES & SUMMARY

from salesforce

QUICK REFERENCE - HTTP METHODS

HTTP METHOD		PURPOSE	IDEMPOTENT
GET		Read / Retrieve data	Yes
POST		Create a new resource	No
PUT		Replace / Update a resource	Yes
PATCH		Partially update a resource	Yes
DELETE		Remove a resource	Yes

GET - Retrieves information from the API.

POST - Creates a new resource.

PUT - Replaces an existing resource.

PATCH - Updates selected fields of a resource.

DELETE - Removes an existing resource.

COMMON USE CASES

1. USER MANAGEMENT API

`/users`

`/users/{userId}`

- Create users
- Retrieve user info
- Update user profiles
- Delete users

2. ORDER MANAGEMENT API

`/orders`

`/orders/{orderId}`

- Manage customer orders
- Track order status
- Update order details
- View order history

3. PRODUCT MANAGEMENT API

`/products`

`/products/{productId}`

- Manage product catalogs
- Retrieve product info
- Update inventory details
- Manage product data

4. PAYMENT MANAGEMENT API

`/payments`

`/payments/{paymentId}`

- Process payments
- Track transactions
- Retrieve payment status

RESTFUL API DESIGN BEST PRACTICES

1. RESOURCE NAMING

Use meaningful and plural resource names.

RECOMMENDED

`/users`

`/orders`

`/products`

`/customers`

`/payments`

AVOID

`/getUsers`

`/createUser`

`/deleteOrder`

`/updateProduct`

2. URI DESIGN GUIDELINES

- ✓ Keep URIs short and meaningful.
- ✓ Use nouns instead of verbs.
- ✓ Follow a consistent naming convention.
- ✓ Use URI parameters for dynamic resources.

EXAMPLES

`/users/{userId}`

`/orders/{orderId}`

`/products/{productId}`

`/users/{userId}/orders/{orderId}`

3. HTTP METHOD GUIDELINES

Always use the appropriate HTTP method for the operation.

OPERATION	METHOD
Retrieve Data	GET
Create Resource	POST
Replace Resource	PUT
Partial Update	PATCH
Delete Resource	DELETE

4. GENERAL GUIDELINES

- ✓ Follow RESTful conventions.
- ✓ Use meaningful resource names.
- ✓ Keep APIs simple and reusable.
- ✓ Maintain consistency across APIs.
- ✓ Properly document all resources and methods.
- ✓ Test and validate APIs regularly.

BENEFITS OF FOLLOWING BEST PRACTICES

- ☆ Improves API readability.
- ☆ Enhances maintainability.
- ☆ Simplifies API consumption.
- ☆ Promotes RESTful standards.
- ☆ Improves API consistency.
- ☆ Supports scalable API design.
- ☆ Provides a better developer experience.

SUMMARY

RESOURCES	Define API endpoints. Represent business entities.
METHODS	Define operations on resources.
URI PARAMETERS	Identify specific resources dynamically.
BEST PRACTICES	Follow RESTful conventions. Use meaningful names. Keep APIs simple and reusable.

FINAL TAKEAWAY

Use Proper Resources
 ↓
 Follow REST Standards
 ↓
 Use Correct HTTP Methods
 ↓
 Keep URIs Meaningful
 ↓
 Build Clean & Scalable APIs

Clear • Consistent • Scalable APIs

Well-structured resources and the proper use of HTTP methods make APIs easier to design, consume, maintain and scale on Anypoint Platform.

3.4.1

URI PARAMETERS



URI Parameters (also called **Path Parameters**) are dynamic values that are included directly in the URI path. They are used to uniquely identify specific resources in **RESTful APIs** and help establish hierarchical relationships between resources.

WHAT ARE URI PARAMETERS?

- ★ URI Parameters are **dynamic values** present in the URI path.
- ★ They are enclosed within curly braces **{ }** in RAML.
- ★ They are used to **uniquely identify** a specific resource.
- ★ URI Parameters are **mandatory** when accessing resource-specific data.
- ★ They play a key role in designing **RESTful** and **resource-centric** APIs.

URI PARAMETER SYNTAX IN RAML

```
/users/{userId}:
  uriParameters:
    userId:
      type: integer
      description: Unique ID of the user
      example: 12345
```

SYNTAX EXPLAINED

ELEMENT	DESCRIPTION
{userId}	URI parameter enclosed within curly braces
type	Specifies the parameter data type
description	Explains the purpose of the parameter
example	Provides a sample value
uriParameters	Defines all URI parameters for a resource

EXAMPLE URI TEMPLATES

URI TEMPLATE	EXAMPLE URI
/users/{userId}	→ /users/12345
/orders/{orderId}	→ /orders/987
/products/{productId}	→ /products/1001
/customers/{customerId}	→ /customers/501

WHY USE URI PARAMETERS?

- Resource Identification** - Helps identify a specific resource uniquely.
- Dynamic API Requests** - Allows APIs to accept variable values in the URI.
- RESTful API Design** - Supports resource-based API architecture.
- Hierarchical Relationships** - Represents parent-child relationships between resources.
- Improved Readability** - Makes API endpoints easy to understand and maintain.

HOW URI PARAMETERS WORK



KEY BENEFITS

- ✓ Makes APIs resource-centric.
- ✓ Supports RESTful API principles.
- ✓ Improves URI readability.
- ✓ Enables dynamic resource identification.
- ✓ Simplifies API design and maintenance.
- ✓ Improves API scalability.

KEY TAKEAWAYS

- ✓ URI Parameters uniquely identify resources.
- ✓ They are essential for RESTful API design.
- ✓ RAML provides built-in support for defining URI Parameters.
- ✓ Proper use of URI Parameters results in **clean** and **scalable** APIs.

COMMON USE CASES

- /users/{userId}** → Retrieves information about a specific user.
- /orders/{orderId}** → Retrieves information about a specific order.
- /products/{productId}** → Retrieves details of a particular product.

QUICK NOTES

- ★ URI Parameters are part of the URI path.
- ★ Always use meaningful parameter names.
- ★ They are enclosed within **{ }**.
- ★ Prefer resource names that are plural.
- ★ Use URI Parameters only for **identifying** resources.

FINAL SUMMARY



URI Parameters make your APIs **RESTful**, **resource-centric** and **easy to maintain**. They are one of the fundamental building blocks of API **design** in MuleSoft and RAML.



3.4.2

URI PARAMETERS

(URI vs QUERY PARAMETERS & MULTIPLE URI PARAMETERS)



URI PARAMETERS vs QUERY PARAMETERS

ASPECT	URI PARAMETERS (Path Parameters)	QUERY PARAMETERS
Location	Included in the URI path	Included after '?' in the URI
Purpose	Identify a specific resource	Filter, search, sort, or paginate data
Mandatory	Usually required	Usually optional
Syntax	<code>/users/{userId}</code>	<code>/users?name=John</code>
RESTful Design	Resource-centric	Query-centric
Caching	Better support for caching	Less efficient for caching

WHEN TO USE URI PARAMETERS?

- ✓ Retrieve a specific resource.
- ✓ Update a specific record.
- ✓ Delete a particular resource.
- ✓ Design hierarchical resource structures.

Examples:

```

/users/{userId}
/orders/{orderId}
/products/{productId}

```

WHEN TO USE QUERY PARAMETERS?

- ✓ Filtering records.
- ✓ Searching data.
- ✓ Sorting results.
- ✓ Implementing pagination.

Examples:

```

/users?name=John
/orders?status=Shipped
/products?category=Electronics
/users?page=1&size=10

```

MULTIPLE URI PARAMETERS

`/orders/{orderId}/items/{itemId}:`

uriParameters:

orderId:

type: integer

description: Unique Order ID

example: 987

itemId:

type: integer

description: Unique Item ID

example: 10

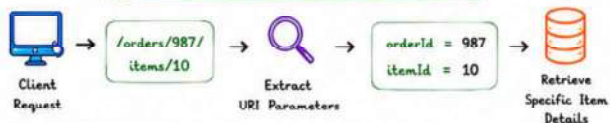
EXAMPLE URI

`/orders/987/items/10`

PARAMETER MAPPING

PARAMETER	VALUE
orderId	987
itemId	10

HOW MULTIPLE URI PARAMETERS WORK



BENEFITS OF USING URI PARAMETERS

- ★ Supports RESTful API design.
- ★ Makes resource identification easy.
- ★ Improves API readability.
- ★ Supports hierarchical resource relationships.
- ★ Enables scalable API structures.

QUICK TIPS

USE URI PARAMETERS

```

/users/{userId}
/orders/{orderId}

```

When:

- Accessing a specific resource.
- Designing resource hierarchies.

USE QUERY PARAMETERS

```

/users?country=India
/orders?status=Delivered

```

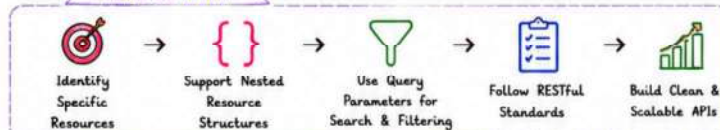
When:

- Filtering data.
- Searching resources.
- Implementing pagination.

KEY TAKEAWAYS

- ✓ URI Parameters identify resources.
- ✓ Query Parameters filter or search resources.
- ✓ Multiple URI Parameters support nested APIs.
- ✓ Follow RESTful resource hierarchy for better API design.
- ✓ Use URI Parameters only when identifying resources.

FINAL SUMMARY



URI Parameters and Query Parameters complement each other in REST API design. Using them appropriately results in clean, intuitive and maintainable MuleSoft APIs.



3.4.3

URI PARAMETERS

(VALIDATION, BEST PRACTICES & COMMON USE CASES)



VALIDATION & CONSTRAINTS

CONSTRAINT	PURPOSE	EXAMPLE
123 type	Defines the data type	integer, string
↓ minimum	Minimum numeric value	1
↑ maximum	Maximum numeric value	999999
A minlength	Minimum length of a string	3
A= maxlength	Maximum length of a string	10
..* pattern	Regular expression validation	[A-Z]{3}[0-9]{3}
≡ enum	Restricts allowed values	Active, Inactive

URI PARAMETER VALIDATION EXAMPLE

```

/users/{userId}:
uriParameters:
  userId:
    type: integer
    description: Unique ID of the user
    example: 123
    minimum: 1
    maximum: 999999
  
```



VALIDATION RULES

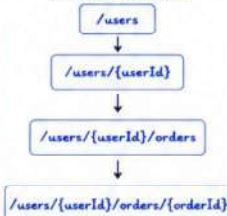
- Must be an integer.
- Must be greater than or equal to 1.
- Must be less than or equal to 999999.
- Must contain a valid user ID.

COMMON VALIDATION EXAMPLES

123	1. NUMERIC VALIDATION	minimum: 1 maximum: 1000
A	2. STRING LENGTH VALIDATION	minLength: 3 maxLength: 20
..*	3. REGULAR EXPRESSION VALIDATION	pattern: "[A-Z]{3}[0-9]{3}" Example: ABC123
≡	4. ENUM VALIDATION	enum: - Active - Inactive - Pending Only the specified values are accepted.

NESTED RESOURCES WITH URI PARAMETERS

RESOURCE HIERARCHY



EXAMPLE REQUEST PATH

/users/123/orders/987

PARAMETER VALUES

PARAMETER	VALUE
userId	123
orderId	987

URI Parameters help represent hierarchical relationships between resources.

BEST PRACTICES

1. USE MEANINGFUL NAMES Good Examples • userId • orderId • productId • customerId • invoiceId Avoid • id • x • param1 • value1	2. USE APPROPRIATE DATA TYPES Examples type: integer type: string type: number Always choose the most appropriate type.	3. APPLY VALIDATION CONSTRAINTS Examples • minimum • maximum • minLength • maxLength • pattern • enum Validation improves API reliability and reduces errors.	4. FOLLOW RESTFUL RESOURCE HIERARCHY Good Examples /Users/{userId} /orders/{orderId} /products/{productId} Avoid unnecessary nesting or overly complex URI structures.	5. DOCUMENT URI PARAMETERS Always provide: • Description • Data type • Example values • Validation rules Well-documented APIs are easier to consume and maintain.
--	---	--	--	--

COMMON USE CASES

/users/{userId} Use Cases: • Retrieve user details • Update user information • Delete a user	/orders/{orderId} Use Cases: • Retrieve order details • Track order status	/products/{productId} Use Cases: • Retrieve product information • Manage inventory	/invoices/{invoiceId} Use Cases: • Retrieve invoice details • Download invoice information
--	---	---	---

BENEFITS OF USING VALIDATED URI PARAMETERS

- Improves API reliability.
- Enhances security.
- Supports RESTful API design.
- Reduces invalid API requests.
- Improves maintainability.
- Provides better API documentation.
- Enables scalable API structures.

QUICK TIPS



- Use URI Parameters only for resource identification.
- Apply validation wherever possible.
 - Keep resource hierarchies simple.
 - Use meaningful parameter names.
 - Provide examples and descriptions for every parameter.

KEY TAKEAWAYS

- URI Parameters uniquely identify resources.
- Validation constraints improve API quality.
- Proper documentation improves developer experience.
- Follow RESTful design principles consistently.
- Keep APIs simple, scalable, and maintainable.

FINAL SUMMARY



- Identify Resources
- Apply Validation
- Follow REST Principles
- Use Meaningful Names
- Build Secure & Scalable APIs

URI Parameters make your APIs **RESTful**, clear, and **resource-centric**. Proper **validation** and **best practices** help build **secure**, **intuitive**, and **enterprise-grade MuleSoft APIs**.

3.5.1

QUERY PARAMETERS



WHAT ARE QUERY PARAMETERS?

Query Parameters are optional values that are added to the end of a URI to **customize** the **API** response. They are commonly used for **filtering**, **searching**, **sorting**, and **pagination** without changing the actual resource path.

Unlike URI Parameters, Query Parameters **do not identify** a specific resource. Instead, they control the data that should be returned by the API.



KEY CHARACTERISTICS

- ☆ Passed in the query string of the URI.
- ☆ Optional by default.
- ☆ Used to customize API responses.
- ☆ Do not identify specific resources.
- ☆ Multiple Query Parameters are separated using the & symbol.
- ☆ Extremely useful for filtering and pagination.

EXAMPLE OF QUERY PARAMETERS

/users?name=John&status=active&page=1&limit=10

Parameter Breakdown

PARAMETER	VALUE	PURPOSE
name	John	Filter users by name
status	active	Retrieve active users
page	1	Return page number 1
limit	10	Return only 10 records

RAML SYNTAX

```

/users:
  get:
    queryParameters:
      name:
        type: string
        required: false
        description: Filter users by name
        example: John
      status:
        type: string
        required: false
        description: Filter users by status
        example: active
      page:
        type: integer
        required: false
        description: Page number
        minimum: 1
        example: 1
      limit:
        type: integer
        required: false
        description: Number of records per page
        default: 10
        maximum: 100
        example: 10
  
```

REQUIRED vs OPTIONAL PARAMETERS

OPTIONAL PARAMETERS
(Default)

required: false

- Can be omitted from the API request.
- Most commonly used in REST APIs.
- Default values can be provided.

REQUIRED PARAMETERS

required: true

- Must be provided in the request.
- Missing parameters may result in 400 Bad Request error.
- Used only when absolutely necessary.

COMMON DATA TYPES

DATA TYPE	DESCRIPTION
string	Text values
integer	Whole numbers
number	Decimal values
boolean	True or False
array	List of values

BEST PRACTICES

- ✓ Use meaningful parameter names.
- ✓ Use appropriate data types.
- ✓ Validate user inputs.
- ✓ Provide default values whenever possible.
- ✓ Clearly document all Query Parameters.
- ✓ Keep parameter names short and descriptive.

WHEN SHOULD WE USE QUERY PARAMETERS?

- Filtering records.
- Searching data.
- Sorting results.
- Implementing pagination.
- Limiting API responses.
- Customizing returned data.

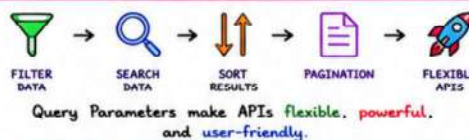
EXAMPLES

- /users?status=active
- /users?name=John
- /users?page=2&limit=20
- /users?sort=createdDate
- /users?search=MuleSoft

KEY TAKEAWAYS

- ☆ Query Parameters customize API responses.
- ☆ They are optional by default.
- ☆ They are ideal for filtering, searching, sorting and pagination.
- ☆ They improve API flexibility and usability.
- ☆ Proper documentation and validation are recommended.

FINAL SUMMARY



3.5.2

QUERY PARAMETERS



TYPES OF QUERY PARAMETERS & EXAMPLES



Query Parameters are used to **filter**, **search**, **sort**, **paginate**, and **customize** API responses. They help APIs return only the required data without changing the resource path.

COMMON DATA TYPES

DATA TYPE	DESCRIPTION
string	Text values
integer	Whole numbers
number	Decimal numbers
boolean	True or False values
array	List of values (comma separated)

TYPES OF QUERY PARAMETERS

TYPE	PURPOSE	DESCRIPTION
Filtering	Filter API responses	Return records that match specific conditions.
Searching	Search using keywords	Find records using keywords or text.
Sorting	Sort results	Sort data in ascending or descending order.
Pagination	Retrieve paged data	Return a specific page of results with a size limit.
Field Selection	Return selected fields	Return only required fields in the response.

EXAMPLES

- Filtering → `/users?status=active`
- Searching → `/users?search=John`
- Sorting → `/users?sort=createdAt&order=desc`
- Pagination → `/users?page=2&limit=10`
- Field Selection → `/users?fields=id,name,email`
- Multiple Filters → `/users?role=admin&status=active`
- Multiple Values → `/users?status=active,inactive,pending`

MULTIPLE VALUES EXAMPLE

You can pass multiple values for the same parameter using comma separated values.

`/users?status=active,inactive,pending`

RAML EXAMPLE

```
status:
  type: string[]
  description: Filter by multiple status values
  example:
    - active
    - inactive
    - pending
```

COMMON USE CASES

USE CASE	EXAMPLE URL
• Filter by Name	<code>/users?name=John</code>
• Filter by Status	<code>/users?status=active</code>
• Pagination	<code>/users?page=3&limit=20</code>
• Sorting	<code>/users?sort=name&order=asc</code>
• Search Users	<code>/users?search=API</code>
• Multiple Filters	<code>/users?role=admin&status=active</code>
• Multiple Values	<code>/users?status=active,inactive</code>
• Field Selection	<code>/users?fields=id,name,email</code>

KEY TAKEAWAYS

- ★ Different types of Query Parameters help customize API responses.
- ★ They make APIs flexible and powerful.
- ★ You can combine multiple parameters to get precise results.
- ★ Multiple values can be passed using comma separated values.
- ★ Field selection helps in reducing the response size and improving performance.



FINAL SUMMARY



Query Parameters help you **FILTER**, **SEARCH**, **SORT**, **PAGINATE**, and **CUSTOMIZE** API responses. Use the right parameter type and follow best practices to build efficient, user-friendly, and scalable APIs.



3.5.3

QUERY PARAMETERS



VALIDATION, BEST PRACTICES & COMMON MISTAKES



Query Parameters make APIs flexible by allowing consumers to **filter**, **search**, **sort**, **paginate**, and **customize** responses. To build robust APIs, it's important to validate inputs, follow best practices, and avoid common mistakes.

1. VALIDATION & CONSTRAINTS

CONSTRAINT	DESCRIPTION	RAML EXAMPLE
required	Parameter is mandatory.	<code>required: true</code>
default	Default value used if not provided.	<code>default: 10</code>
enum	Restricts values to a set list.	enum: - active - inactive - pending
minimum	Minimum allowed value.	<code>minimum: 1</code>
maximum	Maximum allowed value.	<code>maximum: 100</code>
pattern	Validates value using regex.	pattern: " <code>^[a-zA-Z0-9]+\$</code> "
type	Defines the data type.	<code>type: integer</code>

2. QUERY PARAMETERS vs URI PARAMETERS

	QUERY PARAMETERS	URI PARAMETERS
	Appear after ? in the URI.	Part of the URI path.
	Used to filter, search, sort, paginate, and customize data.	Used to identify a specific resource.
	Do not change the resource path.	Change the resource path.
	Optional by default.	Usually required.
	Multiple values can be passed for same parameter.	Usually single value for each URI parameter.

EXAMPLES

Query Parameters
`/users?name=John`
`/users?page=2&limit=20`
`/users?status=active`
`/users?sort=name`
`/users?search=API`

URI Parameters
`/users/123`
`/users/John/orders`
`/products/789/details`

3. COMMON MISTAKES

- Using Query Parameters to identify resources.
- Not validating input values.
- Not setting default values for optional parameters.
- Allowing too many optional parameters.
- Not documenting Query Parameters.
- Using unclear or inconsistent parameter names.
- Accepting invalid data types.
- Ignoring edge cases (null, empty, large values).

4. BEST PRACTICES

- Use meaningful, short, and consistent parameter names.
- Validate all input values and return proper errors.
- Use correct data types for each parameter.
- Set default values where applicable.
- Use enums for parameters with fixed choices.
- Support multiple values using comma separated values.
- Keep the number of optional parameters minimal.
- Document every Query Parameter with examples.
- Maintain consistency across all APIs.

5. BENEFITS OF QUERY PARAMETERS

- Provides flexibility to API consumers.
- Improves API usability and experience.
- Supports powerful filtering and searching.
- Enables efficient pagination.
- Helps retrieve only required data.
- Improves performance and scalability.
- Standard practice in RESTful APIs.



7. KEY TAKEAWAYS

- Query Parameters customize API responses.
- They are optional by default.
- They are ideal for filtering, searching, sorting, and pagination.
- They do not change the resource path.
- Validation and best practices make APIs reliable, secure, and user-friendly.
- Always document and test your Query Parameters.



6. SAMPLE RAML WITH CONSTRAINTS

```

/users:
  get:
    queryParameters:
      status:
        type: string
        required: false
        enum: [active, inactive, pending]
        description: Filter users by status
      page:
        type: integer
        required: false
        minimum: 1
        default: 1
        description: Page number
      limit:
        type: integer
        required: false
        minimum: 1
        maximum: 100
        default: 10
        description: Number of records per page

```

EXAMPLE URI
`/users?`
`status=active,`
`inactive,pending`
`&page=2&limit=20`

RESPONSE EXAMPLE

```

{
  "status": "active",
  "page": 2,
  "limit": 20,
  "data": [
    { ... },
    { ... }
  ]
}

```



Well designed Query Parameters make APIs **FLEXIBLE**, **POWERFUL**, and **USER-FRIENDLY**.

Follow best practices, validate inputs, and avoid common mistakes to build **SCALABLE** and **EFFICIENT** MuleSoft APIs.



3.6.1

REQUEST & RESPONSE


from salesforce



In RAML, every API method can have a **REQUEST** (what the client sends) and one or more **RESPONSES** (what the server returns). Properly defining request and response structures is essential for building reliable and consumer-friendly APIs.

1

REQUEST BODY

- ☆ The body contains the data sent by the client.
- ☆ Used in POST, PUT, and PATCH methods.
- ☆ Specifies media type (e.g., application/json).
- ☆ Supports inline schemas and data types.

RAML SYNTAX EXAMPLE

```
post:
  body:
    application/json:
      type: object
      example:
        id: 101
        name: John
        email: john@example.com
```

 KEY POINTS

- ✓ Used for sending data to APIs.
- ✓ Supports JSON and XML payloads.
- ✓ Helps validate incoming requests.
- ✓ Ensures data is in the correct format.

2

RESPONSE BODY

- ☆ The body contains the data returned by the server.
- ☆ Can have multiple status codes.
- ☆ Includes the response payloads.
- ☆ Helps the client process the returned data.

RAML SYNTAX EXAMPLE

```
responses:
  200:
    body:
      application/json:
        type: object
        example:
          id: 101
          name: John
          email: john@example.com
```

 KEY POINTS

- ✓ Every response can have its own body.
- ✓ Responses should be properly documented.
- ✓ Supports reusable data types.
- ✓ Improves consumer experience.

3

COMMON HTTP STATUS CODES

CODE	MEANING	WHEN TO USE
200 OK	Success	Request succeeded.
201 Created	Created	Resource created successfully.
202 Accepted	Accepted	Request accepted for processing.
204 No Content	No Content	Success, but no data returned.
400 Bad Request	Client Error	Invalid request or parameters.
401 Unauthorized	Unauthorized	Authentication required/failed.
403 Forbidden	Forbidden	Client not allowed to access.
404 Not Found	Not Found	Resource not found.
409 Conflict	Conflict	Resource conflict (e.g., duplicate).
500 Internal	Server Error	Unexpected server error.
503 Service	Unavailable	Service temporarily unavailable.

 BEST PRACTICES

- ✓ Use meaningful status codes.
- ✓ Return helpful error messages.
- ✓ Keep response structures consistent.
- ✓ Use appropriate headers.
- ✓ Validate request data and return proper errors.

SUMMARY


CLIENT
Sends Request

→


REQUEST BODY
Data sent by the client

→


API PROCESSING
Server processes the request

→


RESPONSE BODY
Data returned by the server

→


STATUS CODE
Indicates the result of request

KEY TAKEAWAYS

- ☆ Request Body is used to send data to the API.
- ☆ Response Body is used to return data from the API.
- ☆ HTTP Status Codes help in understanding the result of the request.
- ☆ Properly defining request and response structures makes APIs reliable and easy to consume.
- ☆ Always follow best practices for building robust and user-friendly APIs.



3.6.2

REQUEST & RESPONSE



4 HEADERS

Headers provide additional metadata for requests and responses.

REQUEST HEADERS

- Content-Type
- Authorization
- Accept
- User-Agent
- Custom Headers

RESPONSE HEADERS

- Content-Type
- RateLimit-Remaining
- Cache-Control
- Date
- Server

KEY POINTS

- ✓ Headers are used to send and receive metadata.
- ✓ They can be required or optional.
- ✓ Common headers improve API communication and control.

5 EXAMPLE: COMPLETE METHOD (POST /users)

```

/users:
  post:
    description: Create a new user
    headers:
      Content-Type:
        type: string
        required: true
    body:
      application/json:
        type: object
        example:
          id: 101
          name: John
          email: john@example.com
    responses:
      201:
        description: User created successfully
        body:
          application/json:
            type: object
            example:
              id: 101
              name: John
              email: john@example.com
      400:
        description: Invalid input
        body:
          application/json:
            type: object
            example:
              code: INVALID_REQUEST
              message: Email is required
  
```

KEY POINTS

- ★ A complete method includes headers, body and responses.
- ★ Supports multiple status codes.
- ★ Provides clear examples for consumers.
- ★ Improves API documentation and usability.

METHOD COMPONENTS

- 1 URI
- 2 Description
- 3 Headers
- 4 Request Body
- 5 Responses

6 DATA TYPE EXAMPLES

Example: User Data Type

```

User:
  type: object
  properties:
    id:
      type: integer
      example: 101
    name:
      type: string
      example: John
    email:
      type: string
      example: john@example.com
  
```

Data types help in reusability and ensure consistent data structures.

Example: Error Data Type

```

Error:
  type: object
  properties:
    code:
      type: string
      example: INVALID_REQUEST
    message:
      type: string
      example: Invalid input provided
    details:
      type: string
      example: Email is required
    timestamp:
      type: string
      example: 2024-05-18T10:15:30Z
    path:
      type: string
      example: /users
  
```

KEY POINTS

- ✓ Data types define the structure of request and response data.
- ✓ They improve reusability.
- ✓ Ensure schema validation.
- ✓ Reduce duplication of data structures.
- ✓ Make APIs more maintainable and scalable.

TIPS

- ✓ Use proper headers for authentication and content negotiation.
- ✓ Always define Content-Type.
- ✓ Use reusable data types.
- ✓ Keep method definitions clean and clear.

REMEMBER

- ★ Headers carry metadata.
- ★ Bodies carry payload.
- ★ Data types ensure consistency.
- ★ Good documentation = Better APIs.



Well-structured methods with proper headers, request/response bodies and data types lead to reliable and consumer-friendly APIs.

3.6.3

REQUEST & RESPONSE

MuleSoft
from salesforce

7

REQUEST vs RESPONSE COMPARISON

	REQUEST (Client → Server)	RESPONSE (Server → Client)
	Sent by the client to the server.	Sent by the server to the client.
	Contains data or parameters.	Contains results or status.
	Includes headers and body.	Includes headers and body.
	Used to create, update or request data.	Used to return data or outcome.
	Examples: POST, PUT, PATCH, GET	Examples: 200 OK, 404 Not Found, etc.

8

MULTIPLE RESPONSES EXAMPLE (GET /users/{id})

```
/users/{id}:
get:
  description: Get user by ID
  responses:
    200:
      description: Successful response
      body:
        application/json:
          type: object
          example:
            id: 101
            name: John
            email: john@example.com
    404:
      description: User not found
      body:
        application/json:
          type: object
          example:
            message: "User not found"
    500:
      description: Server error
      body:
        application/json:
          type: object
          example:
            message: "Something went wrong"
```

200 OK

Success

404 Not Found

User not found or resource not available.

500 Server Error

Something went wrong on the server.

9

RESPONSE FLOW

10

ERROR RESPONSE BEST PRACTICES

- Use a consistent error structure across all APIs.
- Return meaningful and human-readable error messages.
- Include application specific error codes.
- Include timestamp for better troubleshooting.
- Include the requested resource path.
- Provide additional details if required.
- Avoid exposing internal server information.

EXAMPLE ERROR STRUCTURE

```
type: object
properties:
  code:
    type: string
    description: Application specific error code
  message:
    type: string
    description: Human readable error message
  details:
    type: string
    description: Additional details about the error
  timestamp:
    type: string
    format: date-time
    example: 2024-05-18T10:15:30Z
  path:
    type: string
    description: The requested resource path
```

QUICK TIPS

- Use proper HTTP status codes.
- Keep error messages meaningful and user-friendly.
- Validate request payloads before processing.
- Always document request and response structures.
- Test APIs using tools like Postman or API Console.

NOTES

- Test your API endpoints thoroughly.
- Validate request and response schemas.
- Use data types and examples for better clarity.
- Keep API designs simple, consistent and consumer-friendly.

FINAL SUMMARY

- Client Sends Request
- API Receives and Processes
- Server Returns Response
- Appropriate HTTP Status Code
- Clear & Consistent Communication
- Reliable & User-Friendly APIs

KEY TAKEAWAYS

- Request and Response are the core of API communication.
- Use proper headers, data types and status codes.
- Document every part clearly for better API usability.
- Good design leads to secure, reliable and scalable APIs.

3.7.1

DATA TYPES

 MuleSoft
from  salesforce



RAML supports various built-in and custom data types that help define the structure of request and response bodies. Data types improve API **validation**, **consistency** and **documentation**.

1 BASIC DATA TYPES (BUILT-IN)

Data Type	Description	Example
string	Textual data	"John Doe"
number	Decimal numbers	25.50
integer	Whole numbers	100
boolean	True or False	true
date-only	Date	2024-05-20
time-only	Time	14:30:00
datetime	Date and Time	2024-05-20T14:30:00Z
datetime-only	Date and Time (No Time Zone)	2024-05-20T14:30:00
file	Binary File	File Content
nil	Null Value	null

2 RAML SYNTAX (TYPE DECLARATION)

```
types:
  User:
    type: object
    properties:
      id: integer
      name: string
      email: string
      isActive: boolean
      createdAt: datetime
```

 KEY POINTS

- Define custom types under the 'types' section.
- Reuse data types across APIs.
- Improves API readability.
- Supports validation and auto-documentation.

3 USING DATA TYPES IN RAML

```
/users:
  post:
    body:
      application/json:
        type: User
        example:
          id: 101
          name: John
          email: john@example.com
          isActive: true
```

 BENEFITS

- Promotes consistency in API design.
- Makes APIs reusable and modular.
- Improves maintainability.
- Provides better documentation.

— TAKE AWAY —



Using the right data types in RAML ensures well-defined, **validated** and **well-documented** APIs that are easy to understand and maintain.





 MuleSoft
 from salesforce

3.7.2 DATA TYPES

Advanced data types in RAML help to model complex structures and flexible data.

4 OBJECT TYPE (EXAMPLE)

RAML Syntax:

```
types:
  Address:
    type: object
    properties:
      street: string
      city: string
      state: string
      zipCode: string
      country: string
```

Example (JSON):

```
{
  "street": "221B Baker Street",
  "city": "London",
  "state": "London",
  "zipCode": "NW1 6XE",
  "country": "UK"
}
```

Object is used to group related properties.

5 ARRAY TYPE (EXAMPLE)

RAML Syntax:

```
types:
  UserList:
    type: array
    items: User
```

Example (JSON):

```
[
  {
    "id": 101, "name": "John"
  },
  {
    "id": 102, "name": "Jane"
  }
]
```

Array holds multiple values of the same type.

6 NESTED OBJECTS & ARRAYS

RAML Syntax:

```
types:
  Order:
    type: object
    properties:
      orderId: integer
      user: User
      items:
        type: array
        items: OrderItem
```

You can nest objects inside objects and arrays.

7 SCALAR TYPE CONSTRAINTS

Type	Constraints You Can Use	Example
string	minLength, maxLength, pattern, enum	minLength: 3 maxLength: 50 pattern: "[A-Z].*"
number	minimum, maximum, multipleOf	minimum: 1 maximum: 100
integer	minimum, maximum, multipleOf	minimum: 1 maximum: 100
boolean	true or false	true / false
datetime	ISO format	2024-05-20T14:30:00Z

Constraints help to validate data.

8 ENUM TYPE

RAML Syntax:

```
types:
  OrderStatus:
    type: string
    enum:
      - PENDING
      - PROCESSING
      - SHIPPED
      - DELIVERED
      - CANCELLED
```

Example Values:
PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED

Enum restricts values to a predefined list.

9 UNION TYPE

RAML Syntax:

```
types:
  ResponseMessage:
    type: string | nil
```

Examples:

- "Operation Successful."
- null

Allows one or more multiple possible types.

BENEFITS OF ADVANCED DATA TYPES



Supports flexible and complex data modeling.



Improves schema validation.



Ensures data consistency.



Makes APIs scalable and maintainable.



Enhances API documentation.

KEY TAKEAWAYS

- ★ Use Object to define structured data with properties.
- ★ Use Array to hold multiple values of the same type.
- ★ Use Enum to limit values to a predefined list.
- ★ Use Union type for fields that can hold multiple possible types.
- ★ Constraints ensure correct data and improve API reliability.





 from 

3.7.3 DATA TYPES

Special data types, reusability and best practices in RAML.

10 FILE TYPE

RAML Syntax:

```
types:
  FileUpload:
    type: object
    properties:
      file:
        type: file
```

Used For:

- File upload APIs
- Image upload APIs
- Document upload APIs



11 NIL TYPE

RAML Syntax:

```
OptionalNote:
  type: string | nil
```

Examples:

- "Delivery between 9 AM - 5 PM"
- OR
- **null**



12 CUSTOM DATA TYPES (REUSABILITY)

RAML Syntax:

```
types:
  Money:
    type: object
    properties:
      currency: string
      amount: number
```

Benefits:

- Encourages reuse
- Ensures consistency
- Easy to maintain



13 REUSE IN RAML

RAML Syntax:

```
/payments:
  post:
    body:
      application/json:
        type: Money
        example:
          currency: "USD"
          amount: 99.99
```

This reuses the "Money" data type defined above.



14 COMPLETE DATA TYPES EXAMPLE

User	Address	OrderStatus
types: User: type: object properties: id: integer name: string email: string address: Address status: OrderStatus createdAt: datetime	types: Address: type: object properties: street: string city: string state: string zipCode: string country: string	types: OrderStatus: type: string enum: - PENDING - PROCESSING - SHIPPED - DELIVERED - CANCELLED

✓ These types can be reused across all APIs for consistency.

15 EXAMPLE RESPONSE

Example JSON Response:

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john@example.com",
  "address": {
    "street": "221B Baker Street",
    "city": "London",
    "state": "London",
    "zipCode": "NW1 6XE",
    "country": "UK"
  },
  "status": "ACTIVE",
  "createdAt": "2024-05-20T14:30:00Z"
}
```

16 BEST PRACTICES

- ✓ Define data types for all request bodies.
- ✓ Define data types for all response bodies.
- ✓ Use meaningful and consistent names.
- ✓ Apply constraints to validate data.
- ✓ Prefer reusable data types and modules.
- ✓ Use enums wherever possible.
- ✓ Always document with examples.



KEY TAKEAWAYS



Data types are the foundation of RAML APIs.



They ensure validation, consistency and reliability.



Reuse data types to build scalable and maintainable APIs.



Good data modeling leads to strong API contracts.



REMEMBER!

- Use the right data type for the right purpose.
- Validate incoming and outgoing data.
- Keep your API models clean, reusable and well-documented.



3.8.1

EXAMPLES

(EXAMPLES IN RAML)



Examples in RAML are used to provide sample request and response payloads. They improve API documentation, help developers understand the expected data format and make APIs easier to test and consume.

1 WHAT ARE EXAMPLES?

- ✓ Help API consumers understand the expected data.
- ✓ Provide sample request and response payloads.
- ✓ Improve API documentation and readability.
- ✓ Demonstrate how APIs should be used.
- ✓ Used only for documentation (not at runtime).

2 RAML SYNTAX OVERVIEW

```
/users:
  get:
    responses:
      200:
        body:
          application/json:
            example:
              {
                "id": 1,
                "name": "John"
              }
```

3 WHERE CAN EXAMPLES BE USED?

- | | |
|------|---------------------|
| REQ | Request Body |
| | • POST, PUT, PATCH |
| RES | Response Body |
| | • Success Responses |
| | • Error Responses |
| HEAD | Headers |
| | • Request Headers |
| | • Response Headers |

4 SINGLE EXAMPLE (INLINE)

Request Example

```
/users:
  post:
    body:
      application/json:
        example:
          {
            "name": "John",
            "email": "john@example.com"
          }
```

Response Example

```
/users:
  post:
    responses:
      200:
        body:
          application/json:
            example:
              {
                "id": 101,
                "name": "John",
                "email": "john@example.com"
              }
```

- ✓ Use when only one sample is needed.

5 MULTIPLE EXAMPLES (INLINE)

```
/users:
  get:
    responses:
      200:
        body:
          application/json:
            examples:
              success:
                value:
                  {
                    "id": 101,
                    "name": "John"
                  }
              notFound:
                value:
                  {
                    "id": 102,
                    "name": "Jane"
                  }
```

- ✓ Supports multiple valid responses for the same API.

6 NAMED EXAMPLES (INLINE)

```
/users:
  get:
    responses:
      200:
        body:
          application/json:
            examples:
              userFound:
                description: User found by ID*
                value:
                  {
                    "id": 101,
                    "name": "John"
                  }
              userNotFound:
                description: User not found
                value:
                  {
                    "message": "User not found"
                  }
```

- ✓ Makes documentation meaningful and improves readability.

BEST PRACTICES

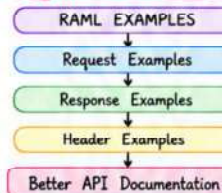
- ✓ Use meaningful example data.
- ✓ Keep examples realistic.
- ✓ Document both requests and responses.
- ✓ Use named examples wherever possible.
- ✓ Keep examples updated.
- ✓ Avoid sensitive information.



KEY TAKEAWAYS

- ✓ Examples improve API documentation.
- ✓ Help consumers understand payload structure.
- ✓ Multiple examples improve usability.
- ✓ Named examples make APIs easier to consume.
- ✓ Good examples enhance developer experience.

FINAL SUMMARY



Well-written examples make your APIs self-explanatory, easier to test and easier for developers to integrate and consume.




MuleSoft
 from salesforce

EXAMPLES

3.8.2

1 EXTERNAL EXAMPLE FILES

```

/users:
  post:
    body:
      application/json:
        example: !include examples/createUser.json
      
```

Example File: createUser.json

```

{
  "name": "John",
  "email": "john@example.com",
  "status": "active"
}
      
```

Benefits:

- Keeps RAML files clean and organized.
- Promotes reuse of example payloads.
- Easier to maintain and update examples.

2 EXAMPLE WITH HEADERS

```

/users:
  get:
    responses:
      200:
        headers:
          Content-Type:
            example: application/json
          X-RateLimit-Limit:
            example: 100
        body:
          application/json:
            example:
              {
                "id": 101,
                "name": "John"
              }
      
```

Use Case:
Document response headers clearly with examples.

3 EXAMPLE WITH QUERY PARAMETERS

```

/users:
  get:
    queryParameters:
      name:
        type: string
        example: John
      limit:
        type: integer
        example: 10
    responses:
      200:
        body:
          application/json:
            example:
              [
                { "id": 101, "name": "John" }
              ]
      
```

Use Case:
Show typical values for query parameters.

4 ADVANCED EXAMPLE (TYPES + EXAMPLES)

```

/users:
  post:
    body:
      application/json:
        type: User
        example:
          {
            "name": "John",
            "email": "john@example.com"
          }
      
```

Use Case:
Reusable types with external examples for clean models.

5 EXAMPLE BEST PRACTICES

- Always use realistic data.
- Include both success and error examples.
- Use named examples wherever possible.
- Use external files for reusable examples.
- Keep examples updated.
- Avoid sensitive information.
- Document edge cases.

6 COMMON SCENARIOS

Scenario	What to Example
Create Resource	Request Body
Get Resource	Response Body
Update Resource	Request + Response
Delete Resource	Response
Error Handling	Error Response
Pagination	Response with Data

7 ERROR RESPONSE EXAMPLE

```

/users/999:
  get:
    responses:
      404:
        body:
          application/json:
            example:
              {
                "error": "User not found",
                "message": "No user exists with ID 999",
                "status": 404
              }
      
```

Benefits:

- Helps consumers handle API errors properly.
- Improves API usability and developer experience.

8 FINAL SUMMARY

```

graph TD
    A[RAML EXAMPLES] --> B[Request Examples]
    B --> C[Response Examples]
    C --> D[Header Examples]
    D --> E[Query Parameter Examples]
    E --> F[Error Examples]
    F --> G[Better API Documentation]
      
```

Good examples make your APIs self-explanatory, easy to test, and easier for consumers to understand and integrate.

TIP

Well-written examples support quick testing and improve developer experience!

REMEMBER

Examples don't run your APIs, they simply help in understanding and testing!

QUICK CHECKLIST

- Are examples realistic?
- Do examples cover all key scenarios?
- Are error examples included?
- Are examples easy to understand?

3.9.1

TRAITS



Traits in RAML are **reusable** pieces of metadata that can be applied to API resources, methods, or responses. They help enforce **consistency**, **reduce duplication**, and improve API **maintainability**.

1 WHAT ARE TRAITS?

Traits are reusable specifications that can contain:

- Headers
- Query Parameters
- Responses
- Security definitions
- Request validations
- Common API behaviors

★ KEY BENEFITS

- Promotes consistency across APIs.
- Reduces code duplication.
- Improves API readability.
- Speeds up API development.
- Encourages standardization.

2 HOW TRAITS WORK

DEFINE TRAIT



APPLY TRAIT



RESOURCE / METHOD



CONSISTENT BEHAVIOR



KEY POINTS

- Traits are defined inside the 'traits' section.
- Each trait has a name and body.
- Applied using the 'is' keyword.
- Multiple traits can be applied together.

3 RAML SYNTAX OVERVIEW

DEFINE A TRAIT

traits:

```
<traitName>:
  <trait body>
```

APPLY A TRAIT

```
/users:
```

```
  get:
```

```
    is: [ client-id-required ]
```

★ You can apply one or more traits using an array.

4 WHERE CAN TRAITS BE APPLIED?

Element	Example
Resource	/users
Method	GET /users
Response	200 Response
API Root (Optional)	Security Trait

5 EXAMPLE: DEFINE TRAITS

```
traits:
  client-id-required:
    headers:
      client-id:
        type: string
        required: true
  rate-limit:
    headers:
      X-Rate-Limit:
        type: integer
        example: 100
```

6 APPLY TRAITS TO RESOURCES & METHODS

```
/users:
  get:
    is:
      [
        client-id-required,
        rate-limit
      ]
```

★ Traits are applied using the 'is' keyword inside methods or resources.

7 BENEFITS OF TRAITS

- Enforces API standards.
- Improves reusability.
- Makes RAML cleaner.
- Easier maintenance.
- Faster API development.

KEY TAKEAWAYS



- Traits are reusable pieces of metadata in RAML.
- They reduce duplication and enforce consistency.
- Traits are applied using the 'is' keyword.
- Multiple traits can be applied together.
- Traits improve maintainability and developer experience.



3.9.2

TRAITS


MuleSoft
 from  **salesforce**

1 TRAITS WITH QUERY PARAMETERS

```
traits:
  paginated:
    queryParams:
      page:
        type: integer
        example: 1
    limit:
      type: integer
      example: 20
```

APPLYING PAGINATION TRAIT

```
/users:
  get:
    is: [
      client-id-required,
      paginated,
      rate-limit
    ]
```

 **Use Case:** Reusable query parameters for pagination.

2 MULTIPLE TRAITS & OVERRIDING

A. MULTIPLE TRAITS

```
/orders:
  get:
    is: [
      client-id-required,
      rate-limit,
      paginated
    ]
```

B. OVERRIDING TRAIT PROPERTIES

```
/orders:
  get:
    is: [ paginated ]
    queryParams:
      limit:
        type: integer
        example: 50
```

 **Element-level definitions override trait properties when the same property exists.**

3 TRAIT WITH SECURITY SCHEME

```
traits:
  oauth2-secured:
    securedBy:
      [ oauth_2_0 ]
```

SECURITY SCHEME EXAMPLE

```
securitySchemes:
  oauth_2_0:
    type: OAuth 2.0
```

APPLYING SECURITY TRAIT

```
/secure-data:
  get:
    is: [
      oauth2-secured,
      standard-error-response
    ]
```

 **Use Case:** Secure APIs using reusable security traits.

4 COMMON TRAITS USED IN APIS

Trait Name	Purpose
 client-id-required	Validate client-id
 rate-limit	API rate limiting
 standard-error-response	Common error responses
 oauth2-secured	Apply OAuth security
 cors-headers	CORS support
 request-id	Request tracing
 audit-info	Audit headers

5 TRAITS BEST PRACTICES

- ✓ Create small and reusable traits.
- ✓ Use meaningful and consistent names.
- ✓ Avoid very large traits.
- ✓ Keep traits generic (avoid business logic).
- ✓ Document all traits clearly.
- ✓ Version traits if reused across multiple APIs.

6 TRAITS VS DATA TYPES

TRAITS	DATA TYPES
 Reusable metadata	 Data structures
 Applied using is keyword	 Used in request/response bodies
 Used for API behavior	 Used for payload validation
 Reduces duplication and enforces consistency	 Defines the shape and structure of data

7 ERROR RESPONSE EXAMPLE

```
/users/999:
  get:
    responses:
      404:
        body:
          application/json:
            example:
              {
                "error": "User not found",
                "message": "No user exists with ID 999",
                "statusCode": 404
              }
```

Use Case: Standard error responses can be reused using traits.

Benefits

- ✓ Improves API consistency.
- ✓ Helps consumers handle errors predictably.

TIP

 Great examples and reusable traits support quick implementation and improve developer experience!

8 FINAL SUMMARY

RAML TRAITS

```

      Define Traits
      ↓
      Reusable Logic
      ↓
      Apply Traits
      ↓
      Resources & Methods
      ↓
      Consistent APIs
      ↓
      Secure & Maintainable APIs
    
```



REMEMBER

- Traits reduce repetition and improve API consistency.
- Use traits to enforce standards and security.
- Keep traits reusable, small and well-documented.
- Good traits make your APIs scalable and maintainable!



3.9.3

TRAITS

(ADVANCED USAGE & PRACTICAL EXAMPLES)



Traits make RAML reusable, consistent and easy to maintain. This page covers **composite traits**, **parameterized traits**, **conditional usage**, **practical examples**, and tips for effective trait design.

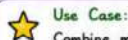
1 COMPOSITE TRAITS

A trait can use other traits inside its definition to build a composite trait.

```
traits:
  secured-trait:
    is: [ client-id-required, oauth2-secured ]

  client-id-required:
    headers:
      client-id:
        type: string
        required: true

  oauth2-secured:
    securedBy: [ oauth_2_0 ]
```



Use Case:
Combine multiple traits into one for easier reuse.

2 PARAMETERIZED TRAITS

Traits can accept parameters to make them flexible and reusable.

```
traits:
  header-required:
    (headerName):
      headers:
        <<headerName>>:
          type: string
          required: true

/orders:
  get:
    is: [ header-required: "X-Client-Id" ]
```



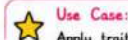
Use Case:
Use the same trait for different headers, values, or inputs.

3 CONDITIONAL TRAITS (WHEN/IF)

Traits can be applied conditionally using **when** with expressions.

```
traits:
  internal-access-only:
    headers:
      internal:
        type: string
        required: true

/admin:
  get:
    is: [ internal-access-only ]
    when: "%environment == 'production'"
```



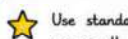
Use Case:
Apply traits only in specific environments or scenarios.

4 TRAITS WITH RESPONSES

Define standard responses in a trait and reuse them.

```
traits:
  standard-error-response:
    responses:
      400:
        body:
          application/json:
            example: { "error": "Bad Request" }
      500:
        body:
          application/json:
            example: { "error": "Internal Server Error" }

/users:
  post:
    is: [ standard-error-response ]
```



Use Case:
Use standardized error responses across all APIs.

5 PRACTICAL EXAMPLE: COMPLETE API USING MULTIPLE TRAITS

```
traits:
  client-id-required:
    headers:
      client-id:
        type: string
        required: true

  rate-limit:
    headers:
      X-Rate-Limit:
        type: integer
        example: 100

  standard-error-response:
    responses:
      400:
        body:
          application/json:
            example: { "error": "Bad Request" }
      500:
        body:
          application/json:
            example: { "error": "Internal Server Error" }

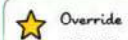
/products:
  get:
    is: [ client-id-required, rate-limit, standard-error-response ]
    responses:
      200:
        body:
          application/json:
            example: { "id": 1, "name": "Laptop" }
```

6 OVERRIDE TRAIT AT RESOURCE / METHOD LEVEL

You can override or extend trait properties when needed.

```
traits:
  paginated:
    queryParameters:
      page:
        type: integer
        default: 1
      limit:
        type: integer
        default: 20

/users:
  get:
    is: [ paginated ]
    queryParameters:
      limit:
        default: 50 # Override default
```



Use Case:
Override values to suit specific resource requirements.

7 TIPS FOR EFFECTIVE TRAIT DESIGN

- ✓ Keep traits small, focused and reusable.
- ✓ Use meaningful and descriptive names.
- ✓ Avoid too many parameters.
- ✓ Make traits generic and independent of specific business logic.
- ✓ Document traits clearly with examples.
- ✓ Test traits independently before use.
- ✓ Version traits when used across multiple APIs.

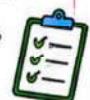


8 WHEN TO USE TRAITS?

- ➔ When the same headers or query parameters are used across multiple APIs.
- ➔ When standard responses are common.
- ➔ When security requirements are repeated.
- ➔ When you want to keep RAML clean and maintainable.
- ➔ When you want to enforce consistency across teams.

9 TRAITS DESIGN CHECKLIST

- ✓ Is the trait reusable?
- ✓ Is the trait name clear?
- ✓ Is the trait generic and not tightly coupled?
- ✓ Are parameters used only when necessary?
- ✓ Is the trait well-documented?
- ✓ Have you tested the trait?



KEY TAKEAWAY

Traits are powerful tools in RAML to build **reusable**, **consistent** and **maintainable** APIs. Use them smartly to improve **productivity**, enforce standards and **reduce duplication**.

3.10.1

RESOURCE TYPES



Resource Types in RAML let you define **reusable** resource templates with common configurations. They help you avoid **repetition** and ensure **consistency** across your API.

1. WHAT ARE RESOURCE TYPES?

- ✓ A template for a resource.
- ✓ Can include URI parameters, methods, query parameters, headers, traits, etc.
- ✓ Can be applied to one or more resources.
- ✓ Promote reuse and standardization.
- ✓ Support consistent documentation and implementation.



Think of resource types as blueprints for similar API resources.

2. HOW RESOURCE TYPES WORK?

1. Define

Create a resource type



2. Apply

Use the type on one or more resources



3. Customize

Override or extend if needed



4. Reuse

Apply again wherever needed



KEY POINTS

- ✓ Defined under the resourceTypes section.
- ✓ Applied with type: resourceName.
- ✓ Element-level properties can be overridden.
- ✓ Encourages DRY (Don't Repeat Yourself) principles.

3. SYNTAX OVERVIEW

Define a Resource Type

```
resourceTypes:
  typeName:
    <trait body or declaration>
```

Use a Resource Type

```
/resourcePath:
  type: typeName
  <additional overrides>
```

Important



You can apply resource types to one or more resources.

4. EXAMPLE: DEFINE A RESOURCE TYPE

```
#%RAML 1.0
title: Sample API
version: v1
resourceTypes:
  collection:
    description: Collection of <resourcePathName>
    get:
      description: Get all <resourcePathName>
      responses:
        200:
          body:
            application/json:
              example: []
  item:
    description: Resource in <resourcePathName>
    get:
      description: Get a <resourcePathName>
      responses:
        200:
          body:
            application/json:
              example: {}
  post:
    description: Create a <resourcePathName>
    responses:
        201:
          body:
            application/json:
              example: {}
```

Explanation

- **resourceTypes:** Defines reusable resource templates.
- **collection:** Template for a collection of resources.
- **item:** Template for a single resource.
- Includes methods (GET, POST), responses, and example payloads.
- Helps maintain consistency in structure across similar resources.

Use Case



Use resource types when multiple resources share the same structure and behavior. It reduces duplication and improves maintainability.

3.10.2

RESOURCE TYPES



1. EXAMPLE: APPLY RESOURCE TYPE RESOURCES

```

/users:
  type: collection
  resourcePathName: Users

/products:
  type: collection
  resourcePathName: Products

/orders:
  type: collection
  resourcePathName: Orders
  
```



Use a resource type on one or more resources for consistent structure.

2. OVERRIDING AND EXTENDING

Override a specific property

```

/users:
  type: collection
  resourcePathName: Users
  get:
    description: Get all users (overridden)
  
```

Add or change query parameter

```

/products:
  type: collection
  resourcePathName: Products
  get:
    queryParameters:
      page:
        type: string
        required: false
        example: 1
  
```

Disable a method (remove POST)

```

/orders:
  type: collection
  resourcePathName: Orders
  post: !include disabled.raml
  
```



Overrides apply at the element level.

3. RESOLVED (EFFECTIVE) VIEW

```

/users:
  get: { ... }
  post: { ... }

/products:
  get: { ... }
  post: { ... }

/orders:
  get: { ... }
  post: { ... }
  
```

Explanation

The resolved view shows the final, expanded version after applying the resource type to the resources.

4. RESOURCE TYPE WITH TRAITS

```

resourceTypes:
  securedCollection:
    get:
      is: [ paginated, secured ]
    post:
      description: Get all <resourcePathName>
      responses:
        200:
          body:
            application/json:
              example: {}
  
```



Traits can be included in resource types for reusability.

5. NESTED RESOURCE TYPES

```

/accounts:
  type: securedCollection
  (/accounts/{id}):
    type: item
    get:
      description: Get a <resourcePathName>

/users:
  type: collection
  resourcePathName: User
  get:
    queryParameters:
      limit:
        type: integer
  
```



Resource types can be used at multiple levels (parent and child resources).

6. COMMON USE CASES



CRUD Operations

Define standard CRUD behavior once and reuse across multiple resources.



Security

Apply common security schemes or traits to multiple resources.



Pagination & Filtering

Standardize query parameters like limit, offset, sort, etc.



Consistent Responses

Ensure consistent response structures and status codes.

7. RESOURCE TYPE BEST PRACTICES

- ✓ Create resource types for commonly used patterns.
- ✓ Keep resource types focused and cohesive.
- ✓ Use meaningful and consistent names.
- ✓ Override only what is necessary.
- ✓ Document resource types clearly.
- ✓ Version your resource types if used across APIs.

8. RESOURCE TYPES VS TRAITS

Aspect	Resource Types	Traits
Purpose	Template for resources	Reusable metadata/behavior
Applies to	Resources (endpoints)	Methods, resources, components, etc.
Structure	Contain resources, declarations (methods, responses, etc.)	Contain properties, headers, queries, responses, etc.
Example	Users, Orders, Products, Items, SecuredItem	Paginated, Cacheable, Secured, RateLimited
Use when	You want to standardize resource structures	You want to add reusable behavior or rules

9. QUICK REFERENCE



Define:

```
resourceTypes: <name>: <declaration>
```



Apply:

```
/path:
  type: <name>
```



Override:

```
/path:
  type: <name>
  <override properties>
```



Add Traits:

```
is: [ <trait1>, <trait2> ]
```



Best For:

Reusability, consistency and maintainability across APIs.



Resource Types help you build **consistent**, **reusable** and **maintainable** APIs by reusing standards across your resources, methods and responses.



3.10.3

RESOURCE TYPES



10. COMMON USE CASES

	CRUD Operations Define standard CRUD behavior once and reuse across multiple resources.
	Security Apply common security schemes or traits to multiple resources.
	Pagination & Filtering Standardize query parameters like limit, offset, sort, etc.
	Consistent Responses Ensure consistent response structures and status codes.

11. RESOURCE TYPE BEST PRACTICES

- ✓ Create resource types for commonly used patterns.
- ✓ Keep resource types focused and cohesive.
- ✓ Use meaningful and consistent names.
- ✓ Override only what is necessary.
- ✓ Document resource types clearly.
- ✓ Version your resource types if used across APIs.



Following best practices ensures reusability, consistency and easier maintenance.

12. RESOURCE TYPES VS TRAITS

Aspect	Resource Types	Traits
Purpose	Template for resources	Reusable metadata/behavior
Applies to	Resources (endpoints)	Methods, resources, components, etc.
Structure	Contain resources, declarations (methods, responses, etc.)	Contain properties, headers, queries, responses, etc.
Example	Users, Orders, Products, Items, SecureItem	Paginated, Cacheable, Secured, RateLimited
Use when	You want to standardize resource structures	You want to add reusable behavior or rules



Key Difference

Resource Types define the structure of resources, while Traits define reusable behavior or metadata.



Remember

Use both together to build consistent, reusable and well-structured RAML APIs.

13. QUICK REFERENCE

	Define	resourceTypes: <name>: <declaration>
	Apply	/path: type: <name>
	Override	/path: type: <name> <override properties>
	Add Traits	is: [<trait1>, <trait2>]
	Best For	Reusability, consistency and maintainability across APIs.

Example

```
resourceTypes:
  collection:
    description: Collection of <resourcePathName>
    get:
      description: Get all <resourcePathName>
      responses:
        200:
          body:
            application/json:
              example: []
/users:
  type: collection
  resourcePathName: Users
```



Resource Types help you build **consistent**, **reusable** and **maintainable** APIs by reusing standards across your resources, methods and responses.



3.11.1

LIBRARIES



Libraries in RAML let you extract and reuse fragments of RAML across multiple APIs.

They promote **reusability**, **consistency** and **modular design**.

1. WHAT ARE LIBRARIES?

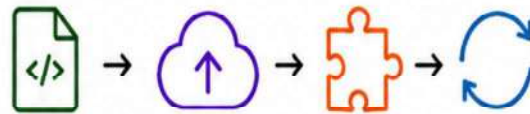
- ✓ A library is an external RAML file that contains reusable fragments like types, traits, resource types, security schemes, examples and more.
- ✓ Imported into an API and referenced using a namespace.
- ✓ Encourages reuse and prevents duplication.
- ✓ Centralizes common definitions and simplifies maintenance.



Think of libraries as building blocks that can be reused across many APIs.

2. HOW LIBRARIES WORK

1. Create Library
2. Publish/Share
3. Use in API
4. Reuse



Define reusable components in a separate RAML file.	Store in a repository or share within your organization.	Import the library in any RAML API using %RAML.	Reference and reuse components with namespaces.
---	--	---	---

2. Key Points

- ✓ Libraries follow DRY (Don't Repeat Yourself).
- ✓ Define once, use many times.
- ✓ Version your libraries for safe evolution.
- ✓ Namespaces prevent naming conflicts.



Libraries are the key to building **consistent**, **scalable** and **maintainable** APIs by reusing common resource patterns in a single place.

3.11.2

LIBRARIES



3. SYNTAX OVERVIEW

Define Library (library.raml)

```

#%RAML 1.0 Library
usage: Provides reusable types, traits,
resource types, security schemes, etc.

```

Use Library in an API (api.raml)

```

#%RAML 1.0
uses:
  lib: libraries/library.raml
# Use components with the "lib" prefix
/users:
  get:
    200: lib.SuccessResponse

```



Use a local or remote RAML file as a library.

4. WHAT CAN BE IN A LIBRARY?

Element	Description
Types	Data type definitions (objects, arrays, enums).
Traits	Reusable request/response fragments.
Resource Types	Templates for resources and methods.
Parameters	Standardized request parameters.
Examples	Reusable example payloads.
Security Schemes	Security definitions (OAuth, API Key, etc.).
Annotations	Custom annotations and metadata.

NAMESPACE AND REFERENCES

```

uses:
  common: libraries/common-types.raml
  paging: libraries/paging.raml

# Referencing
common.User           # Type
common.paginated      # Traits
common.SuccessResponse # Response
paging.PagingRequest  # Another library
paging.Pagination     # Resource type

```



Always use namespaces to reference items from a library.

3.11.3

LIBRARIES



1. EXAMPLE: DEFINE A LIBRARY

File: libraries/common-types.raml

```
#%RAML 1.0 Library
types:
  User:
    type: object
    properties:
      id: integer
      name: string
      email: string
      active: boolean
      age?: integer
    example:
      id: 101
      name: "John"
      email: "john@example.com"
      active: true
  traits:
    paginated:
      queryParameters:
        page:
          type: integer
          required: false
        limit:
          type: integer
          required: false
  responses:
    SuccessResponse:
      description: Request was successful
      body:
        application/json:
          example: { "message": "Success" }
    NotFoundResponse:
      description: Resource not found
      body:
        application/json:
          example: { "message": "Not Found" }
```

2. EXAMPLE: USE LIBRARY IN AN API

File: api.raml

```
#%RAML 1.0
uses:
  common: libraries/common-types.raml
/users:
  get:
    is: [ common.paginated ]
    description: Get all users
    responses:
      200:
        body:
          application/json:
            type: common.User
            example: { "message": "Success" }
      404:
        body:
          application/json:
            type: common.NotFoundResponse
/users/{id}:
  get:
    is: [ common.paginated ]
    description: Get user by ID
    responses:
      200:
        body:
          application/json:
            type: common.User
      404:
        body:
          application/json:
            type: common.NotFoundResponse
```



Use "is" to include resources like traits, responses, headers, resource types, etc.

3. EXAMPLE: LIBRARY WITH RESOURCE TYPE

File: libraries/resource-types.raml

```
#%RAML 1.0 Library
resourceTypes:
  collection:
    get:
      is: [ paginated ]
      responses:
        200:
          body:
            application/json:
              type: array
              items: <<item>>
              example: []
```



Use "<<item>>" to pass the resource type when using this resource type.

4. USING RESOURCE TYPE FROM LIBRARY

File: api.raml

```
#%RAML 1.0
uses:
  common: libraries/resource-types.raml
  products: libraries/common-types.raml
/products:
  type: collection
  get:
    description: Get all products
    responses:
      200:
        body:
          application/json:
            type: common.Product
```



Pass parameters to resource types using "type".



Libraries are the key to building consistent, scalable and maintainable APIs by reusing common resource patterns in a single place.



3.12.1 FRAGMENTS



Fragments in RAML let you extract and reuse any YAML content across your API. They help you follow DRY (Don't Repeat Yourself), improve **consistency**, and **simplify maintenance**.

PART 1 OF 3 1. WHAT ARE FRAGMENTS?

- ✓ A fragment is a reusable piece of YAML content.
- ✓ Can be used for data types, traits, headers, query parameters, responses, examples, etc.
- ✓ Stored in separate YAML files.
- ✓ Reused in multiple APIs to reduce duplication.
- ✓ Promotes reusability, consistency and modularity.



Think of fragments as **reusable puzzle pieces** that can be used wherever needed.

PART 2 OF 3 2. HOW FRAGMENTS WORK

1. Create Fragment



Create reusable content in a separate YAML file.

2. Reference



Reference the fragment using include.

3. Reuse Everywhere



Use the fragment in any RAML API multiple times.

4. Update Once



Update the fragment once, changes reflect everywhere.

★ Key Benefits

- ✓ Eliminates duplication and reduces errors.
- ✓ Enforces consistency across your API.
- ✓ Simplifies updates and maintenance.
- ✓ Encourages modular and scalable API design.

PART 3 OF 3 3. SYNTAX OVERVIEW

Define a Fragment File

```
# File: fragments/common-headers.raml
headers:
  X-Correlation-ID:
    type: string
    description: Correlation ID for tracing
  Accept:
    type: string
    description: Accepted token
```

Use a Fragment in RAML

```
# RAML 1.0
uses:
  h: fragments/common-headers.raml

/users:
  get:
    headers: h
```



Fragment path is relative to the RAML file (including sub-folders).



Fragments help you build **clean**, **consistent** and **maintainable** RAML APIs. Define once, reuse everywhere!



3.12.2 FRAGMENTS



Fragments in RAML let you extract and reuse any YAML content across your API. They help you follow DRY (Don't Repeat Yourself), improve **consistency**, and **simplify maintenance**.

PART 1 OF 3 1. CREATE FRAGMENT



Create a reusable YAML fragment and store it in a separate file.

```
File: fragments/common-headers.raml
headers:
  X-Correlation-ID:
    type: string
    description: Correlation ID for tracing
  Accept:
    type: string
    description: Accepted token
```

i Keep fragments focused, reusable and well-structured.

PART 2 OF 3 2. REFERENCE FRAGMENT

include

Reference the fragment inside your RAML file using the include keyword.

```
File: api.raml
#%RAML 1.0
uses:
  common-headers: fragments/common-headers.raml
/users:
  get:
    description: Get all users
    headers: !include common-headers.headers
    responses:
      200:
        body:
          application/json:
            type: array
```

i Use **!include** to include a fragment (or part of it) at any place in your RAML.

PART 3 OF 3 3. BENEFITS



Eliminates Duplication

Avoid repeating the same content across multiple APIs or endpoints.



Enforces Consistency

Ensure consistent headers, parameters, responses and data types.



Simplifies Updates

Make changes in one place and it reflects everywhere.



Encourages Modular Design

Build small, reusable pieces and compose APIs easily.



Improves Maintainability

Easier to manage, debug and maintain large APIs.



Scalable and Reusable

Reuse fragments across teams, projects and APIs.



Fragments help you build **clean**, **consistent** and **maintainable** RAML APIs. Define once, reuse everywhere!

3.12.3 FRAGMENTS



Fragments in RAML let you extract and reuse any YAML content across your API. They help you follow **DRY (Don't Repeat Yourself)**, improve **consistency**, and **simplify maintenance**.

PART 1 OF 3 1. SYNTAX OVERVIEW

Define a Fragment File

```
# File: fragments/common-headers.raml
headers:
  X-Correlation-ID:
    type: string
    description: Correlation ID for tracing
  Accept:
    type: string
    description: Accepted token
```

Use a Fragment in RAML

```
# File: api.raml
#%RAML 1.0
uses:
  h: fragments/common-headers.raml

/users:
  get:
    headers: h
```



Fragment path is relative to the RAML file (including sub-folders).

PART 2 OF 3 2. EXAMPLE: USING FRAGMENTS IN RAML

Fragment File

```
# File: fragments/common-headers.raml
headers:
  X-Request-ID:
    type: string
    description: Unique request ID
  Authorization:
    type: string
    description: Bearer token
```

Another Fragment File

```
# File: fragments/pagination.raml
queryParams:
  page:
    type: integer
    description: Page number
    default: 1
  limit:
    type: integer
    description: Items per page
    default: 10
```

Main RAML File Using Fragments

```
# File: api.raml
#%RAML 1.0
uses:
  hdr: fragments/common-headers.raml
  pg: fragments/pagination.raml
/users:
  get:
    description: Get all users
    headers: !include hdr.headers
    queryParams: !include pg.queryParameters
    responses:
      200:
        body:
          application/json:
            type: array
            items: !include fragments/user-response.raml
```



Use **!include** to include fragments (or parts of them) at any place in your RAML.

PART 3 OF 3 3. BENEFITS OF USING FRAGMENTS



Eliminates Duplication

Avoid repeating the same content across multiple APIs or endpoints.



Enforces Consistency

Keep headers, parameters, responses and data types consistent across your API.



Simplifies Updates

Change content in one place and it reflects everywhere it is used.



Encourages Modular Design

Break large APIs into small, reusable pieces that are easy to understand.



Scalable and Reusable

Reuse fragments across teams, projects and multiple APIs.



Improves Maintainability

Makes APIs easier to manage, debug and maintain over time.



Fragments help you build **clean**, **consistent** and **maintainable** RAML APIs.
Define once, reuse everywhere!



3.13.1 INCLUDES



Includes in RAML let you split your API definition into multiple files and bring them together. They promote **modularity**, **reusability** and easier **maintenance**.

PART 1 OF 3 1. WHAT ARE INCLUDES?

- ✓ Include brings content from another RAML file into the current file.
- ✓ Helps to split large RAML files into smaller, manageable parts.
- ✓ Improves readability and maintainability.
- ✓ Supports reuse of common definitions across multiple APIs.



Think of includes as **building blocks** that you assemble to build your API.

PART 2 OF 3 2. HOW INCLUDES WORK

1. Create



Create reusable RAML fragments.

2. Include



Use **!include** to bring them in.

3. Merge



RAML processor merges the content.

4. Resolve



Final RAML is ready to be used.

★ Key Points

- ✓ Included content is merged at the location of the **!include**.
- ✓ Relative paths are resolved from the including file.
- ✓ Includes can be nested.
- ✓ Avoid circular includes.

PART 3 OF 3 3. SYNTAX OVERVIEW

```
!include <relative-path-to-file.raml>
```

Example

```
types:  
  User: !include types/user.raml  
traits:  
  paginated: !include traits/paginated.raml  
/users: !include resources/users.raml
```



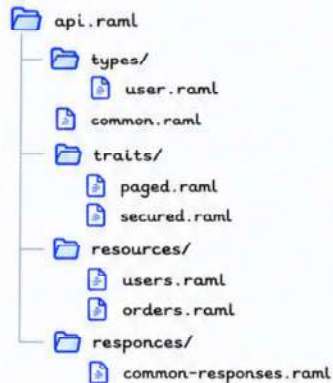
The included file can contain any valid RAML fragment (types, traits, resources, etc.).

3.13.2 INCLUDES



Includes in RAML let you split your API definition into multiple files and bring them together. They promote **modularity**, **reusability** and easier **maintenance**.

PART 1 OF 3 1. EXAMPLE: PROJECT STRUCTURE



★ Key Points

- ✓ Organize your RAML using small, focused files.
- ✓ Group related fragments by domain or purpose.
- ✓ Keeps main RAML file clean and easy to read.
- ✓ Encourages reuse and consistency across APIs.



Think of **includes** as building blocks that you assemble to build your API.

PART 2 OF 3 2. EXAMPLE: MAIN FILE (api.raml)

```

#%RAML 1.0
title: Sample API
version: v1
baseUrl: https://api.example.com/{version}
mediaType: application/json
uses:
  types: types/
  traits: traits/
  responses: responses/common-responses.raml

/users: !include resources/users.raml
/orders: !include resources/orders.raml
  
```



The main file imports types, traits and common responses.

- Resource files are included where the resources are defined.
- Keeps the main file simple and focused.

PART 3 OF 3 3. EXAMPLE: INCLUDED FILES

types/user.raml

```

User:
  type: object
  properties:
    id: integer
    name: string
    email: string
    status: string
  
```

traits/paged.raml

```

paged:
  queryParameters:
    page:
      type: integer
      required: false
      example: 1
    limit:
      type: integer
      required: false
      example: 10
  
```

traits/secured.raml

```

secured:
  headers:
    Authorization:
      type: string
      description: Bearer token
      required: true
  
```

resources/users.raml

```

/users:
  is: [paged, secured]
  get:
    description: Get all users
    responses:
      200: !include ../responses/success.raml
  post:
    description: Create a new user
    responses:
      201: !include ../responses/created.raml
  
```

resources/orders.raml

```

/orders:
  is: [paged, secured]
  get:
    description: Get all orders
    responses:
      200: !include ../responses/success.raml
  post:
    description: Create a new order
    responses:
      201: !include ../responses/created.raml
  
```

responses/common-responses.raml

```

success:
  200:
    description: Successful response
    body:
      application/json:
        type: User
  created:
    201:
      description: Resource created
      body:
        application/json:
          type: User
  
```



Structure your RAML using includes to build **clean**, **modular** and **maintainable** APIs.
Small files, big impact!

3.13.3 INCLUDES



Includes in RAML let you split your API definition into multiple files and bring them together. They promote **modularity**, **reusability** and easier **maintenance**.

PART 1 OF 3 1. SYNTAX OVERVIEW

```
!include <relative-path-to-file.raml>
```

Syntax Rules

- ✓ Use **!include** to bring content from another RAML file.
- ✓ The path is relative to the file containing the include.
- ✓ The included content is merged at the location of the include.
- ✓ Includes can be nested (an included file can include others).
- ✓ Avoid circular includes.

Example

```
api.raml
%%RAML 1.0
title: Sample API
version: v1
baseUrl: https://api.example.com/{version}
mediaType: application/json

uses:
  types: types/user.raml
  traits: traits/pagination.raml

/users:
  get:
    is: [ pagination ]
    responses:
      200:
        body:
          application/json:
            type: User
```

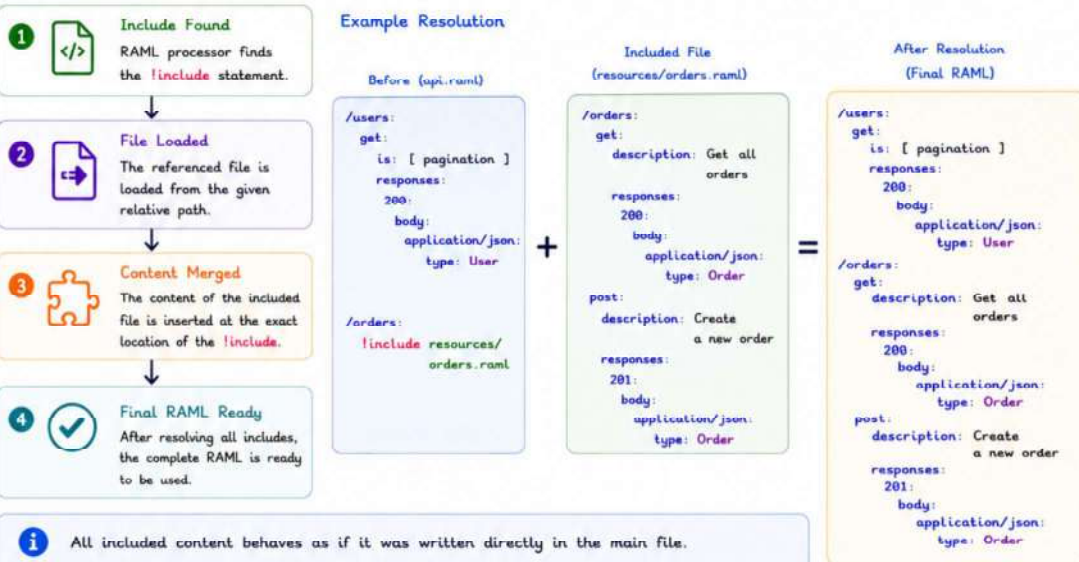


Tip
Think of includes as building blocks that you assemble to build your API.



Note
The included file can contain any valid RAML fragment (types, traits, resources, etc.).

PART 2 OF 3 2. HOW INCLUDES ARE RESOLVED



PART 3 OF 3 3. BEST PRACTICES



Keep It Modular
Split RAML into small, focused files by domain or purpose.



Reuse Common Fragments
Store shared types, traits and responses in common files and include them where needed.



Use Clear Structure
Organize files in a logical folder structure for easy navigation and maintenance.



Avoid Deep Nesting
Too many nested includes can make it hard to trace and maintain.



Avoid Circular Includes
Circular references will cause errors. Plan your include dependencies carefully.



Test After Changes
After modifying included files, test your API to ensure everything works as expected.



Includes help you build **clean**, **modular** and **maintainable** RAML APIs.
Small files. Big impact!



3.14.1 SECURITY SCHEMES



Security schemes in RAML let you define how your API is secured.

They promote **consistency**, **reuse** and clear **documentation** of authentication methods.

PART 1 OF 3 1. WHAT ARE SECURITY SCHEMES?

- ✓ Define the authentication and authorization methods used to access your API.
- ✓ Can be referenced globally or at resource, method or trait level.
- ✓ Encourage reuse and consistency.
- ✓ Improve API documentation and developer experience.
- ✓ Help enforce security best practices.

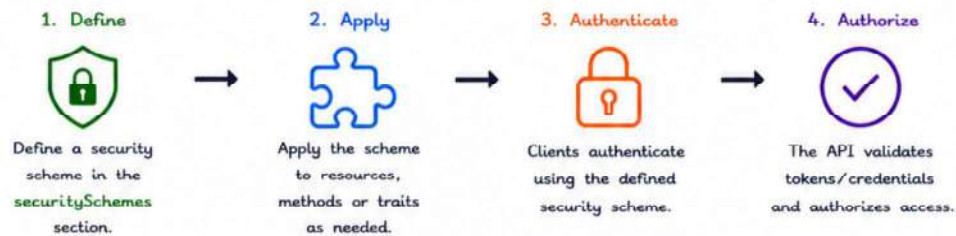


Define once, use everywhere.
Keep your API secure and consistent.

★ Key Points

- ✓ Define once and reuse in multiple places.
- ✓ Choose the right scheme for your API.
- ✓ Multiple schemes can be defined and used.
- ✓ Secure by default, but allow flexibility.
- ✓ Document clearly for better API consumer experience.

PART 2 OF 3 2. HOW SECURITY SCHEMES WORK



Security schemes help enforce authentication and authorization consistently across your API.

PART 3 OF 3 3. SYNTAX OVERVIEW

Define a security scheme

```

securitySchemes:
  <schemeName>:
    description: <description>
    type: <schemeType>
    describedBy:
      headers:
        <headerName>:
          description: <header description>
          type: <string|integer|boolean>
          required: <true|false>
  
```

Example

```

securitySchemes:
  customScheme:
    description: Custom authentication using
      API key in header
    type: x-custom
    describedBy:
      headers:
        x-api-key:
          description: API Key for authentication
          type: string
          required: true
  
```

Common Properties

Property	Description
securitySchemes	Root node to define one or more security schemes.
<schemeName>	Name of the security scheme (e.g., customScheme, basicAuth, oauth_2_0).
type	Type of the scheme (e.g., basic, digest, oauth_2_0, x-custom).
describedBy	Provides additional details (e.g., headers, query parameters, responses).
headers	Used to define HTTP headers required for authentication/authorization.



Note

Different scheme types have different properties. Refer to RAML specification for details on each type.



Security schemes help you define authentication methods clearly, ensuring **consistent**, **reusable** and **well-documented** API security.



3.14.2 SECURITY SCHEMES



Security schemes in RAML let you define how your API is secured.

They promote **consistency**, **reuse** and clear **documentation** of authentication methods.

PART 1 OF 3 1. SECURITY SCHEME TYPES

Type	Description	Common Use	Example
basic	HTTP Basic Authentication	Simple user + password	Authorization: Basic <token>
digest	HTTP Digest Authentication	Enhanced password handling	Authorization: Digest <token>
oauth_1_0	OAuth 1.0	Legacy OAuth 1.0 flows	OAuth 1.0 signed requests
oauth_2_0	OAuth 2.0	Modern OAuth 2.0 flows	Bearer <access_token>
x509	X.509 (mutual TLS)	Secure API to API communication	Client certificate
x509-sni	X.509 Client Certificate (with SNI)	Mutual TLS in enterprise environments	Client certificate + SNI
custom	Custom authentication mechanism	Proprietary or custom auth	Custom headers / logic

★ Key Points

- ✓ Choose the scheme that best matches your security needs.
- ✓ Multiple schemes can be defined and used.
- ✓ Security schemes can be applied at API root, resource or method level.
- ✓ Secure by default, but allow flexibility.



Define once, use everywhere.
Keep your API secure and consistent.

PART 2 OF 3 2. EXAMPLE: DEFINE SECURITY SCHEMES

File: security.raml

```
#%RAML 1.0
securitySchemes:
  basicAuth:
    type: Basic Authentication
    description: Basic auth using username and password

  oauth2_0:
    type: OAuth 2.0
    description: OAuth 2.0 using Authorization Code Grant
    describedBy:
      headers:
        Authorization:
          type: string
          description: "OAuth 2.0 access token"

  apiKeyAuth:
    type: x-api-key
    describedBy:
      headers:
        X-API-Key:
          type: string
          description: "API Key for authentication"

  customAuth:
    type: Custom Authentication
    describedBy: API Key passed in request header
```

i Explanation

This file defines four security schemes:

- basicAuth → HTTP Basic Authentication
- oauth2_0 → OAuth 2.0 (Authorization Code Grant)
- apiKeyAuth → API Key in request header
- customAuth → Custom authentication mechanism

These schemes can now be applied to your API, resources or methods.

★ Best Practices

- ✓ Define schemes in a separate file (e.g., security.raml).
- ✓ Use clear and descriptive names.
- ✓ Document required headers or parameters.
- ✓ Reuse schemes across APIs and resources.
- ✓ Secure by default, override when necessary.

PART 3 OF 3 3. APPLY SECURITY SCHEMES

Example 1: Apply to Entire API

```
#%RAML 1.0
title: Sample API
version: v1
baseUrl: https://api.example.com/{version}
securitySchemes: !include security.raml
securedBy: [ basicAuth ] # Apply to entire API

/users:
  get:
    description: Get all users
```

Example 2: Apply to Resource

```
#%RAML 1.0
title: Sample API
version: v1
baseUrl: https://api.example.com/{version}
securitySchemes: !include security.raml
/users:
  securedBy: [ oauth2_0 ] # Apply to /users
  get:
    description: Get all users
  post:
    description: Create a new user
```

Example 3: Apply to Method

```
#%RAML 1.0
title: Sample API
version: v1
baseUrl: https://api.example.com/{version}
securitySchemes: !include security.raml
/products:
  get:
    description: Get products (public)
  post:
    securedBy: [ apiKeyAuth ] # Only this method
    description: Create a new product
```

⚠ Important Notes

- Multiple schemes can be applied using an array: `securedBy: [ basicAuth, apiKeyAuth ]`
- You can override a scheme at a lower level (resource or method).
- If no scheme is applied, the resource or method is considered public.



Tip

Use security schemes to enforce consistent authentication and authorization across all your APIs.



Use security schemes to protect your APIs with the right authentication methods.

Define once, reuse everywhere, and keep your APIs **secure** and **consistent**!



3.14.3 SECURITY SCHEMES



Security schemes in RAML let you define how your API is secured. They promote **consistency**, **reuse** and clear **documentation** of authentication methods.

PART 1 OF 3 1. EXAMPLE: APPLY SECURITY SCHEMES

File: api.raml

```
#%RAML 1.0
title: User API
version: v1
baseUri: https://api.example.com/{version}
securitySchemes: !include security.raml

/users:
  get:
    description: Get all users
    securedBy: [ basicAuth ] # Apply to entire API
    responses:
      200:
        description: Successful response
        body:
          application/json:
            type: array
            items: User

  post:
    description: Create a new user
    securedBy: [ oauth2_0 ] # Override for this method
    body:
      application/json:
        type: User

/products:
  get:
    description: Get products (public)
    # No security applied -> public

  post:
    description: Create a new product
    securedBy: [ apiKeyAuth ] # Only this method
```

How It Works

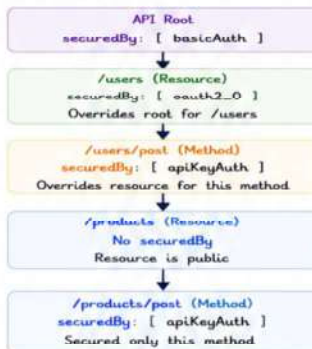
- 1** **Apply at API root**
Use `securedBy` at the root level to apply a scheme to all resources/methods.
- 2** **Override when needed**
Use `securedBy` at resource or method level to override the applied scheme.
- 3** **Method level security**
Apply a scheme only to specific methods that require protection.
- 4** **Public access**
If no scheme is applied, the resource or method is considered public.



Define once, use everywhere.
Keep your API secure and consistent.

PART 2 OF 3 2. INHERITANCE AND OVERRIDE

Inheritance Flow



Effective Security

Level	Example	Secured By	Result
API Root	All resources/methods	[basicAuth]	All inherit basicAuth
Resource Override	/users	[oauth2_0]	/users and its methods inherit oauth2_0
Method Override	/users/post	[apiKeyAuth]	Only this method uses apiKeyAuth
No Security	/products	(none)	Public access
Method Only	/products/post	[apiKeyAuth]	Only this method is secured



Important Notes

- Method level `securedBy` always overrides resource and API root level.
- Resource level `securedBy` overrides API root level.
- If no `securedBy` is defined at any level, the resource or method is public.
- Choose the most restrictive level that meets your security requirements.

PART 3 OF 3 3. BEST PRACTICES



Use the Right Scheme

- Choose the scheme that fits your security needs:
- basic - Simple username/password
 - digest - Enhanced password handling
 - oauth2_0 - OAuth 2.0 flows
 - x509 - Mutual TLS / Certificates
 - custom - Custom authentication mechanisms



Reuse Schemes

- Define schemes once in a security file and reuse across APIs.
- Ensures consistency and reduces duplication.
- Easier to maintain and update.



Document Clearly

- Provide clear descriptions for each scheme and property.
- Document required headers or parameters.
- Helps developers understand how to authenticate.



Secure by Default

- Apply security at the highest practical level.
- Override or remove only when necessary.



Test Your Security

- Verify that schemes are applied as expected.
- Test both authorized and unauthorized access.



Stay Consistent

- Use consistent naming conventions.
- Keep scheme definitions versioned and reviewed.
- Follow your organization's security standards.



Security schemes help you clearly define and apply authentication methods across your API.
Define once, reuse everywhere, and protect your APIs with confidence.



3.15 RAML BEST PRACTICES



Following best practices helps you build RAML APIs that are **consistent**, **reusable**, **secure** and easy to maintainable.

WHY BEST PRACTICES MATTER?

- ✓ Improve consistency across APIs and teams.
- ✓ Enhance readability and maintainability.
- ✓ Promote reuse and reduce duplication.
- ✓ Make onboarding and collaboration easier.
- ✓ Lead to better API quality and consumer experience.

CORE PRINCIPLES



QUICK SYNTAX REMINDERS

```
title: My API
version: v1
mediaType: application/json
types:
  User:
    type: object
    properties:
      id: integer
      name: string
traits:
  paginated:
    queryParams:
      page: integer
      size: integer
/users:
  get:
    is: [ paginated ]
    response:
      200:
        body:
          application/json:
            type: User[]
```

BEST PRACTICES CHECKLIST

- Organize with Consistent Structure**
 - Follow a clear folder and file structure.
 - Group related resources and types.
 - Keep RAML files focused and modular.
- Use Meaningful and Consistent Naming**
 - Use clear, descriptive names for resources, types, traits, and methods.
 - Follow naming conventions (e.g., kebab-case for resources).
 - Avoid abbreviations unless widely accepted.
- Reuse with Libraries, Includes, Types and Traits**
 - Extract reusable components to libraries.
 - Use !include and !include fragments to avoid duplication.
 - Define common patterns as traits and use them.
- Keep It DRY (Don't Repeat Yourself)**
 - Avoid repeating examples, responses, or headers.
 - Use types and traits to centralize common definitions.
- Write Clear and Helpful Documentation**
 - Provide a good description at API, resource and method level.
 - Use annotations and Markdown for detailed docs.
 - Include request/response examples.
- Design Consistent Responses**
 - Use consistent response codes and structures.
 - Define standard error responses and reuse them.
 - Document all possible responses.
- Apply Security Best Practices**
 - Define security schemes at the top level.
 - Use least-privilege access.
 - Do not expose sensitive data.
- Version Your API**
 - Include version in the title or baseUrl.
 - Follow semantic versioning (e.g., v1, v1.1, v2).
 - Maintain backward compatibility.
- Use Appropriate Resource Modeling**
 - Model resources as nouns, methods as actions.
 - Use proper HTTP methods (GET, POST, PUT, DELETE, etc.).
 - Use sub-resources and collections correctly.
- Validate and Test Your RAML**
 - Validate RAML using a linter (e.g., RAML CLI, API Designer).
 - Test examples and schemas.
 - Catch issues early.
- Collaborate and Review**
 - Use version control (Git) for RAML files.
 - Review changes and follow code review practices.
 - Maintain a changelog.
- Plan for Maintainability**
 - Keep files small and focused.
 - Document decisions and assumptions.
 - Refactor and clean up regularly.

GOOD VS AVOID

Good Practice	Avoid
✓ Use meaningful names	Using unclear or abbreviated names ✗
✓ Reuse types and traits	Duplicating schemas and definitions ✗
✓ Consistent response structures	Different structures for same scenarios ✗
✓ Secure by design	Exposing sensitive data or endpoints ✗
✓ Clear documentation and examples	Missing or unclear documentation ✗

EXAMPLE: GOOD PRACTICE

```
#RAML 1.0
title: User Management API
version: v1
baseUrl: https://api.example.com/{version}
mediaType: application/json

uses:
  common: libraries/common.raml

/users:
  description: Manage users
  get:
    is: [ common.paginated ]
    responses:
      200:
        body:
          application/json:
            type: common.User[]

  post:
    body:
      application/json:
        type: common.User
    responses:
      201:
        body:
          application/json:
            type: common.User
```

TOOLS THAT HELP

- API Designer**
Design and validate RAML visually.
- RAML CLI**
Validate, link and bundle RAML files.
- VS Code Extensions**
RAML language support and snippets.
- CI/CD Pipelines**
Automate validation and quality checks.

COMMON PITFALLS

- ⚠ Overusing includes making it hard to navigate.
- ⚠ Creating overly large RAML files.
- ⚠ Ignoring validation warnings and errors.
- ⚠ Inconsistent naming and formatting.
- ⚠ Not documenting error responses.

FINAL TAKEAWAY

★ Follow best practices to build RAML APIs that are consistent, secure, reusable, and a joy to maintain.
Good RAML today,
Great APIs tomorrow!



Good RAML is not just about valid syntax – it's about **clarity**, **reuse** and **value** for your consumers.
Follow best practices and build APIs that stand the test of time!

